

Niva source code

Review copy of the app program files. Open this PDF in VS Code (right-click the file -> Open With) or any PDF viewer.

Files included

- index.html (784 lines)
- app.js (2634 lines)
- core.js (286 lines)
- styles.css (2682 lines)
- visual.js (10 lines)
- sw.js (44 lines)
- manifest.json (18 lines)

Not included: tests.html, sample CSV, deploy helper, binary assets. Line numbers are on the left.

index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover" />
6     <meta name="mobile-web-app-capable" content="yes" />
7     <meta name="apple-mobile-web-app-capable" content="yes" />
8     <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" />
9     <meta name="description" content="Niva ? Know where it goes. Private spending tracker that stays on this device." />
10    <meta name="theme-color" content="#1a1016" />
11    <title>Niva</title>
12    <link rel="icon" href="icon.svg" type="image/svg+xml" />
13    <link rel="apple-touch-icon" href="icon.svg" />
14    <link rel="preconnect" href="https://fonts.googleapis.com" />
15    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
16    <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@400;500;600;700&family=Syne:wght@600;700;800&display=swap" rel="stylesheet" />
17    <link rel="manifest" href="manifest.json" />
18    <link rel="stylesheet" href="styles.css" />
19  </head>
20  <body>
21    <div class="bg-wallpaper" aria-hidden="true"></div>
22    <div class="bg-mesh" aria-hidden="true"></div>
23    <div class="bg-vignette" aria-hidden="true"></div>
24
25    <!-- WELCOME (first open) -->
26    <div id="screen-welcome" class="screen">
27      <div class="welcome-stage welcome-simple">
28        <div class="welcome-card">
29          <div class="brand-hero">
30            
31            <div>
32              <p class="brand-name">Niva</p>
33              <p class="brand-tag brand-motto">Know where it goes</p>
34            </div>
35          </div>
36          <h1>Track spending in under a minute</h1>
37          <p class="lede">
38            Log what you spend, then see where it went. Everything stays in this browser ? nothing is uploaded.
39          </p>
40          <ol class="how-steps">
41            <li><span>1</span> Set up once</li>
42            <li><span>2</span> Add spends</li>
43            <li><span>3</span> Check Insights</li>
44          </ol>
45          <div class="welcome-actions">
46            <button type="button" class="btn primary large" id="btn-start-setup" onclick="window.startSetup&&window.startSetup();return false;">
```

```
47         Get started
48         <span class="btn-sub">Name, PIN, and a quick setup</span>
49     </button>
50     <button type="button" class="btn ghost" id="btn-open-demo" onclick="window.openDemo&&window.openDemo();return false;">
51         Try a demo first
52     </button>
53     <button type="button" class="btn linkish" id="btn-goto-login" onclick="window.showLogin&&window.showLogin();return false;">
54         Already set up? Unlock
55     </button>
56 </div>
57 </div>
58 </div>
59 </div>
60
61 <!-- LOGIN (existing profile on this device) -->
62 <div id="screen-login" class="screen" hidden>
63     <div class="login-stage">
64         <div class="welcome-card login-card">
65             <div class="brand-hero">
66                 
67             <div>
68                 <p class="brand-name">Niva</p>
69                 <p class="brand-tag brand-motto">Know where it goes</p>
70             </div>
71         </div>
72         <h1>Unlock</h1>
73         <p class="lede" id="login-copy">
74             Enter the mobile number and PIN for this browser?s profile.
75         </p>
76         <form id="login-form" class="login-form" autocomplete="on">
77             <label>Mobile number
78                 <input type="tel" id="login-mobile" inputmode="tel" autocomplete="username" placeholder="10 digits or +91" required />
79             </label>
80             <label>PIN
81                 <input type="password" id="login-pin" inputmode="numeric" maxlength="8" autocomplete="current-password" placeholder="4?8 digits" required />
82             </label>
83             <div id="login-confirm-wrap" hidden>
84                 <label>Confirm PIN
85                 <input type="password" id="login-pin-confirm" inputmode="numeric" maxlength="8" autocomplete="new-password" placeholder="Re-enter PIN" />
86             </div>
87             <div>
88                 <button type="submit" class="btn primary large block" id="btn-login" onclick="window.submitLogin&&window.submitLogin();return false;">
89                     <span id="login-btn-label">Unlock</span>
90                     <span class="btn-sub" id="login-btn-sub">Stays unlocked until you close this tab</span>
91                 </button>
92                 <p class="form-status login-status" id="login-status" aria-live="polite"></p>
93             </div>
94         </form>
95     </div>
96 </div>
97 </div>
98 </div>
```

```
95         <button type="button" class="btn secondary" id="btn-create-profile" onclick="window.createNewProfile&&window.createNewProfile();return false;">
96             Create a new profile
97             <span class="btn-sub" id="create-profile-sub">For a new person on this browser</span>
98         </button>
99         <button type="button" class="btn ghost" id="btn-login-demo" onclick="window.openDemo&&window.openDemo();return false;">
100             Try demo
101         </button>
102     </div>
103     <div class="login-reset">
104         <button type="button" class="btn ghost tiny" id="btn-forgot-reset">Erase this browser?s data?</button>
105         <div id="login-reset-panel" hidden>
106             <p class="muted small" id="login-reset-copy">Type the mobile number on this profile to erase everything here.</p>
107             <label>Confirm reset
108                 <input type="text" id="login-reset-confirm" autocomplete="off" placeholder="Mobile number" />
109             </label>
110             <button type="button" class="btn danger" id="btn-confirm-device-reset">Erase everything</button>
111             <p class="form-status login-status" id="login-reset-status" aria-live="polite"></p>
112         </div>
113     </div>
114 </div>
115 </div>
116 </div>
117
118 <!-- SETUP WIZARD -->
119 <div id="screen-setup" class="screen" hidden>
120     <div class="wizard">
121         <header class="wizard-head">
122             <p class="eyebrow">Setup</p>
123             <h1>A few quick questions</h1>
124             <p class="muted" id="setup-step-label">Step 1 of 5 ? about you</p>
125             <div class="step-pills" aria-hidden="true">
126                 <span class="step-pill on" data-pill="1">You</span>
127                 <span class="step-pill" data-pill="2">PIN</span>
128                 <span class="step-pill" data-pill="3">Targets</span>
129                 <span class="step-pill" data-pill="4">Accounts</span>
130                 <span class="step-pill" data-pill="5">Budgets</span>
131             </div>
132             <div class="progress" aria-hidden="true"><div class="progress-fill" id="setup-progress"></div></div>
133         </header>
134
135         <section class="wizard-step panel" data-step="1">
136             <h2>Your name</h2>
137             <p class="muted">First name is enough.</p>
138             <label>Name <input type="text" id="setup-name" placeholder="e.g. Priya" required autocomplete="given-name" /></label>
139             <label>
140                 Currency
141                 <select id="setup-currency">
142                     <option value="INR">INR (?)</option>
```

```
143         <option value="USD">USD ($)</option>
144         <option value="EUR">EUR (?)</option>
145         <option value="GBP">GBP (?)</option>
146     </select>
147 </label>
148 </section>
149
150 <section class="wizard-step panel" data-step="2" hidden>
151     <h2>Mobile & PIN</h2>
152     <p class="muted">Unlocks Niva on this browser. PIN is never stored as plain text.</p>
153     <label>Mobile number <input type="tel" id="setup-mobile" required autocomplete="tel" inputmode="tel" placeholder="10 digits or +91" /></label>
154     <div class="form-row-2">
155         <label>PIN (4-8 digits) <input type="password" id="setup-pin" inputmode="numeric" maxlength="8" autocomplete="new-password" required /></label>
156         <label>Confirm PIN <input type="password" id="setup-pin-confirm" inputmode="numeric" maxlength="8" autocomplete="new-password" required /></label>
157     </div>
158 </section>
159
160 <section class="wizard-step panel" data-step="3" hidden>
161     <h2>Income & savings</h2>
162     <p class="muted">Approximate is fine ? used for your Insights score.</p>
163     <label>Usual monthly income <input type="number" id="setup-income" min="0" step="100" placeholder="85000" inputmode="numeric" /></label>
164     <label>Monthly savings goal <input type="number" id="setup-savings" min="0" step="100" placeholder="15000" inputmode="numeric" /></label>
165 </section>
166
167 <section class="wizard-step panel" data-step="4" hidden>
168     <h2>Accounts</h2>
169     <p class="muted">Nicknames like ?HDFC salary? or ?SBI UPI?. Opening balance is optional.</p>
170     <div id="setup-accounts" class="stack"></div>
171     <button type="button" class="btn ghost" id="btn-add-account-field">+ Add another account</button>
172 </section>
173
174 <section class="wizard-step panel" data-step="5" hidden>
175     <h2>Monthly budgets</h2>
176     <p class="muted">Soft limits ? Insights warns if you go over.</p>
177     <div id="setup-budgets" class="budget-grid"></div>
178 </section>
179
180 <footer class="wizard-foot">
181     <button type="button" class="btn ghost" id="btn-setup-back" hidden>Back</button>
182     <button type="button" class="btn primary" id="btn-setup-next">Continue</button>
183 </footer>
184 </div>
185 </div>
186
187 <!-- MAIN APP -->
188 <div id="screen-app" class="screen" hidden>
189     <header class="topbar">
190         <div class="brand">
```

```
191     <span class="brand-mark" aria-hidden="true">
192       
193       <img id="header-photo" alt="" hidden />
194     </span>
195   </div>
196   <p class="brand-name">Niva</p>
197   <p class="brand-tag brand-motto" id="app-subtitle">Know where it goes</p>
198 </div>
199 </div>
200 <nav class="main-tabs" role="tablist" aria-label="App sections">
201   <button type="button" class="tab active" data-view="track" role="tab" aria-selected="true">
202     <span class="tab-ico" aria-hidden="true">?</span>
203     <span class="tab-label">Home</span>
204   </button>
205   <button type="button" class="tab" data-view="results" role="tab" aria-selected="false">
206     <span class="tab-ico" aria-hidden="true">?</span>
207     <span class="tab-label">Insights</span>
208   </button>
209   <button type="button" class="tab" data-view="setup-edit" role="tab" aria-selected="false">
210     <span class="tab-ico" aria-hidden="true">?</span>
211     <span class="tab-label">Profile</span>
212   </button>
213 </nav>
214 <div class="save-cluster">
215   <p class="save-status" id="save-status">Auto-saved</p>
216   <button type="button" class="btn ghost tiny" id="btn-save-local">Backup</button>
217 </div>
218 </header>
219
220 <div id="demo-banner" class="demo-banner" hidden>
221   <strong>Demo mode</strong> ? sample data only. Your real list is untouched.
222   <button type="button" class="btn tiny secondary" id="btn-exit-demo">Exit demo</button>
223 </div>
224
225 <!-- TRACK -->
226 <main id="view-track" class="layout view">
227   <section class="home-glance" id="home-glance" aria-live="polite">
228     <div class="glance-card">
229       <span class="glance-label">Spent this month</span>
230       <strong id="glance-spent">?</strong>
231     </div>
232     <div class="glance-card">
233       <span class="glance-label">Received</span>
234       <strong id="glance-income" class="good">?</strong>
235     </div>
236     <div class="glance-card">
237       <span class="glance-label">Left / buffer</span>
238       <strong id="glance-net">?</strong>
```

```
239     </div>
240 </section>
241
242 <section class="panel intro-panel slim">
243   <div>
244     <p class="eyebrow" id="track-greet">Welcome</p>
245     <h1 id="track-heading">Add a spend</h1>
246     <p class="muted" id="next-action">Amount first, then who and category.</p>
247   </div>
248 </section>
249
250 <section class="grid-2-track">
251   <form id="quick-form" class="panel form-card add-sheet">
252     <h2>New entry</h2>
253
254     <label class="amount-field">Amount
255       <input type="number" id="q-amount" required min="0.01" step="0.01" placeholder="0" inputmode="decimal" autofocus />
256     </label>
257
258     <fieldset class="type-pick type-chips">
259       <legend class="sr-only">Type</legend>
260       <label class="chip-choice">
261         <input type="radio" name="q-type" value="expense" checked />
262         <span>Spent</span>
263       </label>
264       <label class="chip-choice">
265         <input type="radio" name="q-type" value="income" />
266         <span>Received</span>
267       </label>
268       <label class="chip-choice">
269         <input type="radio" name="q-type" value="transfer" />
270         <span>Transfer</span>
271       </label>
272     </fieldset>
273     <div id="transfer-fields" hidden>
274       <label>To account <select id="q-transfer-to"></select></label>
275     </div>
276
277     <label>Who / what <input type="text" id="q-merchant" required placeholder="Swiggy, Rent, Salary?" /></label>
278     <div class="form-row-2">
279       <label>Category <select id="q-category"></select></label>
280       <label>Account <select id="q-account" required></select></label>
281     </div>
282
283     <details class="more-fields">
284       <summary>Date & note</summary>
285       <label>Date <input type="date" id="q-date" /></label>
286       <label>Note <input type="text" id="q-note" placeholder="Optional" /></label>
```

```
287         </details>
288
289         <button type="submit" class="btn primary large block">Save</button>
290         <p class="form-status" id="form-status" aria-live="polite"></p>
291     </form>
292
293     <section class="panel">
294         <div class="panel-head row">
295             <div>
296                 <h2>Recent</h2>
297                 <p class="muted" id="track-count">0 saved</p>
298             </div>
299         </div>
300         <label class="list-search">
301             Search
302             <input type="search" id="track-search" placeholder="Merchant, category?" />
303         </label>
304         <div class="bulk-bar" id="bulk-bar" hidden>
305             <span id="bulk-count">0 selected</span>
306             <div class="bulk-actions">
307                 <button type="button" class="btn tiny secondary" id="btn-bulk-edit">Edit</button>
308                 <button type="button" class="btn tiny danger" id="btn-bulk-delete">Delete</button>
309                 <button type="button" class="btn tiny ghost" id="btn-bulk-clear">Clear</button>
310             </div>
311         </div>
312         <div class="table-wrap">
313             <table class="tx-table compact">
314                 <thead>
315                     <tr>
316                         <th class="check-col">
317                             <input type="checkbox" id="select-all" title="Select all visible" aria-label="Select all visible transactions" />
318                         </th>
319                         <th>Date</th>
320                         <th>Details</th>
321                         <th class="num">Amount</th>
322                         <th></th>
323                     </tr>
324                 </thead>
325                 <tbody id="track-tx-body"></tbody>
326             </table>
327         </div>
328         <p class="empty-inline" id="track-empty">Nothing yet ? add your first spend on the left.</p>
329         <div class="pager" id="track-pager" hidden>
330             <button type="button" class="btn ghost tiny" id="track-prev">Newer</button>
331             <span id="track-page-label">Page 1</span>
332             <button type="button" class="btn ghost tiny" id="track-next">Older</button>
333         </div>
334         <details class="more-fields import-fold">
```

```
335         <summary>Import bank CSV</summary>
336         <label class="btn secondary file-btn block">
337             Choose CSV file
338             <input type="file" id="csv-input" accept=".csv,text/csv" hidden />
339         </label>
340         <p class="muted small">Map columns and skip duplicates before importing.</p>
341     </details>
342 </section>
343 </section>
344 </main>
345
346 <!-- RESULTS -->
347 <main id="view-results" class="layout view" hidden>
348     <section class="panel intro-panel slim">
349         <div>
350             <p class="eyebrow">Overview</p>
351             <h1>Insights</h1>
352             <p class="muted">How you're doing this period ? based on what you logged.</p>
353         </div>
354     </section>
355
356     <section class="controls panel compact-controls">
357         <div class="control-group">
358             <div class="segmented" role="group" aria-label="Time period">
359                 <button type="button" class="seg active" data-period="day">Day</button>
360                 <button type="button" class="seg" data-period="month">Month</button>
361                 <button type="button" class="seg" data-period="year">Year</button>
362             </div>
363         </div>
364         <div class="control-group">
365             <div class="date-nav">
366                 <button type="button" class="icon-round" id="btn-prev" aria-label="Previous">?</button>
367                 <input type="date" id="anchor-date" />
368                 <button type="button" class="icon-round" id="btn-next" aria-label="Next">?</button>
369                 <button type="button" class="btn ghost tiny" id="btn-today">Today</button>
370             </div>
371         </div>
372         <div class="control-group">
373             <select id="account-filter" aria-label="Account"><option value="all">All accounts</option></select>
374         </div>
375         <div class="control-group grow">
376             <input type="search" id="search" placeholder="Search?" aria-label="Search" />
377         </div>
378         <div class="control-group">
379             <button type="button" class="btn ghost tiny" id="btn-export">Export</button>
380         </div>
381     </section>
382
```

```
383     <section class="hero-strip panel" id="health-strip">
384       <div>
385         <p class="eyebrow">Health</p>
386         <h2 id="health-title">This period</h2>
387         <p class="muted" id="health-sub">Add a few spends to unlock your score.</p>
388       </div>
389       <div class="health-meter">
390         <div class="ring-wrap">
391           <div class="ring" id="health-ring"><span id="health-score">?</span></div>
392         </div>
393         <p class="muted small" id="health-label">Score</p>
394       </div>
395     </section>
396
397     <section class="stats" id="stats-row"></section>
398
399     <div class="grid-2">
400       <section class="panel">
401         <div class="panel-head">
402           <h2>Spending over time</h2>
403           <p class="muted" id="chart-caption">Selected period</p>
404         </div>
405         <div class="chart" id="trend-chart"></div>
406       </section>
407       <section class="panel">
408         <div class="panel-head">
409           <h2>By category</h2>
410           <p class="muted">Where money went</p>
411         </div>
412         <div id="category-bars" class="category-bars"></div>
413       </section>
414     </div>
415
416     <div class="grid-2">
417       <section class="panel">
418         <div class="panel-head">
419           <h2>Budgets</h2>
420           <p class="muted">Vs your monthly limits</p>
421         </div>
422         <div id="budget-list" class="budget-list"></div>
423       </section>
424       <section class="panel">
425         <div class="panel-head">
426           <h2>Tips</h2>
427           <p class="muted">Based on your habits</p>
428         </div>
429         <ul id="insights-list" class="insights"></ul>
430       </section>
```

```
431     </div>
432
433     <details class="panel more-insights">
434       <summary>More detail ? merchants, accounts, recurring</summary>
435       <div class="grid-2 nested-grid">
436         <section>
437           <h3 class="subhead">Top merchants</h3>
438           <div id="merchant-list" class="merchant-list"></div>
439         </section>
440         <section>
441           <h3 class="subhead">Accounts</h3>
442           <div id="accounts-list" class="accounts"></div>
443           <h3 class="subhead">Likely recurring</h3>
444           <div id="recurring-list" class="recurring-list"></div>
445           <h3 class="subhead">Coming bills</h3>
446           <div id="forecast-list" class="recurring-list"></div>
447         </section>
448       </div>
449     </details>
450
451     <details class="panel more-insights">
452       <summary>Transactions in this period</summary>
453       <div class="panel-head row">
454         <p class="muted" id="tx-count">0 items</p>
455         <button type="button" class="btn ghost tiny" id="btn-print">Print</button>
456       </div>
457       <div class="bulk-bar" id="results-bulk-bar" hidden>
458         <span id="results-bulk-count">0 selected</span>
459         <div class="bulk-actions">
460           <button type="button" class="btn tiny secondary" id="btn-results-bulk-edit">Edit</button>
461           <button type="button" class="btn tiny danger" id="btn-results-bulk-delete">Delete</button>
462           <button type="button" class="btn tiny ghost" id="btn-results-bulk-clear">Clear</button>
463         </div>
464       </div>
465       <div class="table-wrap">
466         <table class="tx-table">
467           <thead>
468             <tr>
469               <th class="check-col">
470                 <input type="checkbox" id="results-select-all" title="Select all in this view" aria-label="Select all transactions in this view" />
471               </th>
472               <th>Date</th>
473               <th>Merchant</th>
474               <th>Category</th>
475               <th>Account</th>
476               <th class="num">Amount</th>
477             <th></th>
478           </thead>
```

```
479         </thead>
480         <tbody id="results-tx-body"></tbody>
481     </table>
482 </div>
483 <p class="empty-inline" id="results-empty" hidden>No data in this period ? add spends on the Home tab.</p>
484 <div class="pager" id="results-pager" hidden>
485     <button type="button" class="btn ghost tiny" id="results-prev">Newer</button>
486     <span id="results-page-label">Page 1</span>
487     <button type="button" class="btn ghost tiny" id="results-next">Older</button>
488 </div>
489 </details>
490 </main>
491
492 <!-- DEMO (same results UI, sample data) -->
493 <main id="view-demo" class="layout view" hidden>
494     <section class="panel intro-panel">
495         <h1>This is a tour, not your money</h1>
496         <p class="muted">
497             Sample payments so you can see Insights. Nothing here is saved to your real list.
498         </p>
499         <button type="button" class="btn primary" id="btn-demo-to-setup">Set up my own tracking</button>
500     </section>
501     <div id="demo-results-mount"></div>
502 </main>
503
504 <!-- EDIT DETAILS -->
505 <main id="view-setup-edit" class="layout view" hidden>
506     <section class="panel form-card narrow profile-page">
507         <h1>Profile</h1>
508         <p class="muted">Settings for this browser only.</p>
509
510         <details class="profile-section" open>
511             <summary>Security</summary>
512             <div class="pin-box" id="pin-gate">
513                 <p class="muted" id="pin-copy">Mobile + PIN unlock this profile. Closing the tab locks it again.</p>
514                 <p class="masked-mobile">Signed in as <strong id="profile-mobile-masked">?????????</strong></p>
515                 <h3 class="subhead">Change PIN</h3>
516                 <label>Current PIN <input type="password" id="pin-current" inputmode="numeric" maxlength="8" autocomplete="current-password" /></label>
517                 <div class="form-row-2" id="pin-set-row">
518                     <label>New PIN <input type="password" id="pin-new" inputmode="numeric" maxlength="8" autocomplete="new-password" /></label>
519                     <label>Confirm <input type="password" id="pin-confirm" inputmode="numeric" maxlength="8" autocomplete="new-password" /></label>
520                 </div>
521                 <div class="row-actions">
522                     <button type="button" class="btn primary" id="btn-change-pin">Update PIN</button>
523                     <button type="button" class="btn secondary" id="btn-logout" onclick="window.logoutSession()&&window.logoutSession();return false;">Lock</button>
524                 </div>
525                 <p class="form-status" id="pin-status"></p>
526             </div>
```

```
527     </details>
528
529     <details class="profile-section" open>
530       <summary>About you</summary>
531       <div id="sensitive-wrap">
532         <div class="profile-identity">
533           <div class="avatar-wrap">
534             <img id="profile-photo" alt="" hidden />
535             <span class="avatar-fallback" id="profile-photo-fallback" aria-hidden="true">+</span>
536           </div>
537           <div class="profile-identity-copy">
538             <p class="muted">Optional photo ? stays in this browser.</p>
539             <div class="row-actions">
540               <label class="btn secondary file-btn">
541                 Upload
542                 <input type="file" id="photo-input" accept="image/*" hidden />
543               </label>
544               <button type="button" class="btn ghost" id="btn-remove-photo">Remove</button>
545             </div>
546           </div>
547         </div>
548         <label>Name <input type="text" id="edit-name" autocomplete="name" placeholder="e.g. Priya" /></label>
549         <div class="form-row-2">
550           <label>Phone <input type="tel" id="edit-phone" autocomplete="tel" inputmode="tel" placeholder="+91 ?" /></label>
551           <label>Email <input type="email" id="edit-email" autocomplete="email" placeholder="you@email.com" /></label>
552         </div>
553         <details class="more-fields">
554           <summary>Optional details</summary>
555           <label>Date of birth <input type="date" id="edit-dob" /></label>
556           <div class="form-row-2">
557             <label>City <input type="text" id="edit-city" autocomplete="address-level2" /></label>
558             <label>Occupation <input type="text" id="edit-occupation" /></label>
559           </div>
560           <label>
561             Gender
562             <select id="edit-gender">
563               <option value="">Prefer not to say</option>
564               <option value="woman">Woman</option>
565               <option value="man">Man</option>
566               <option value="nonbinary">Non-binary</option>
567               <option value="other">Other</option>
568             </select>
569           </label>
570           <label>Notes <textarea id="edit-notes" rows="2" placeholder="Optional"></textarea></label>
571         </details>
572       </div>
573     </details>
574
```

```
575     <details class="profile-section" open>
576       <summary>Money</summary>
577       <label>
578         Currency
579         <select id="edit-currency">
580           <option value="INR">INR (?)</option>
581           <option value="USD">USD ($)</option>
582           <option value="EUR">EUR (?)</option>
583           <option value="GBP">GBP (?)</option>
584         </select>
585       </label>
586       <div class="form-row-2">
587         <label>Monthly income <input type="number" id="edit-income" min="0" step="100" /></label>
588         <label>Savings goal <input type="number" id="edit-savings" min="0" step="100" /></label>
589       </div>
590       <h3 class="subhead">Accounts</h3>
591       <div id="edit-accounts" class="stack"></div>
592       <button type="button" class="btn ghost" id="btn-edit-add-account">+ Add account</button>
593       <h3 class="subhead">Monthly budgets</h3>
594       <div id="edit-budgets" class="budget-grid"></div>
595     </details>
596
597     <div class="row-actions spaced">
598       <button type="button" class="btn primary" id="btn-save-details">Save changes</button>
599     </div>
600     <p class="form-status" id="edit-status" aria-live="polite"></p>
601
602     <details class="profile-section">
603       <summary>Backup & advanced</summary>
604       <p class="muted">Data already auto-saves here. Download a file if you want a copy on your computer.</p>
605       <div class="row-actions">
606         <button type="button" class="btn secondary" id="btn-save-file">Download backup</button>
607         <label class="btn ghost file-btn">
608           Restore file
609           <input type="file" id="backup-input" accept=".json,application/json" hidden />
610         </label>
611       </div>
612       <div class="row-actions spaced">
613         <button type="button" class="btn ghost" id="btn-goto-demo-from-profile">Try demo</button>
614         <button type="button" class="btn ghost danger" id="btn-reset-all">Erase all data?</button>
615       </div>
616     </details>
617   </section>
618 </main>
619 </div>
620
621 <dialog id="edit-dialog">
622   <form method="dialog" id="edit-form" class="dialog-form">
```

```
623     <h3 id="edit-dialog-title">Edit transaction</h3>
624     <p class="muted" id="edit-dialog-hint">Change any field, then save.</p>
625     <input type="hidden" id="e-id" />
626     <fieldset class="type-pick">
627       <legend>Type</legend>
628       <label class="chip"><input type="radio" name="e-type" value="expense" checked /> Spent</label>
629       <label class="chip"><input type="radio" name="e-type" value="income" /> Received</label>
630       <label class="chip"><input type="radio" name="e-type" value="transfer" /> Transfer</label>
631     </fieldset>
632     <label>Date <input type="date" id="e-date" required /></label>
633     <label>Where / who? <input type="text" id="e-merchant" required /></label>
634     <label>Amount <input type="number" id="e-amount" required min="0.01" step="0.01" /></label>
635     <label>Category <select id="e-category"></select></label>
636     <label>From account <select id="e-account"></select></label>
637     <div id="e-transfer-fields" hidden>
638       <label>Send to account <select id="e-transfer-to"></select></label>
639     </div>
640     <label>Note <input type="text" id="e-note" /></label>
641     <div class="dialog-actions">
642       <button type="button" class="btn ghost" id="btn-edit-cancel">Cancel</button>
643       <button type="submit" class="btn primary">Save changes</button>
644     </div>
645   </form>
646 </dialog>
647
648 <dialog id="bulk-edit-dialog">
649   <form method="dialog" id="bulk-edit-form" class="dialog-form">
650     <h3>Edit selected transactions</h3>
651     <p class="muted" id="bulk-edit-hint">Only filled fields are applied. Leave a field blank to keep each row as-is.</p>
652     <label>
653       Date
654       <input type="date" id="b-date" />
655     </label>
656     <label>
657       Type
658       <select id="b-type">
659         <option value="">Keep current</option>
660         <option value="expense">Money out</option>
661         <option value="income">Money in</option>
662         <option value="transfer">Transfer</option>
663       </select>
664     </label>
665     <label>
666       Category
667       <select id="b-category">
668         <option value="">Keep current</option>
669       </select>
670     </label>
```

```
671     <label>
672       Account
673       <select id="b-account">
674         <option value="">Keep current</option>
675       </select>
676     </label>
677     <div id="b-transfer-fields" hidden>
678       <label>Send to account <select id="b-transfer-to"></select></label>
679     </div>
680     <label>Note (optional replace) <input type="text" id="b-note" placeholder="Leave blank to keep notes" /></label>
681     <label class="chip keep-row">
682       <input type="checkbox" id="b-note-apply" />
683       Replace notes with the text above
684     </label>
685     <div class="dialog-actions">
686       <button type="button" class="btn ghost" id="btn-bulk-edit-cancel">Cancel</button>
687       <button type="submit" class="btn primary">Apply to selected</button>
688     </div>
689   </form>
690 </dialog>
691
692 <dialog id="save-dialog">
693   <div class="dialog-form">
694     <h3>Save your data locally</h3>
695     <p class="muted">
696       Everything you enter is already stored in this browser (local storage ? not encrypted). Download a file if you want a copy on your computer.
697     </p>
698     <div class="save-options">
699       <article class="save-option">
700         <strong>This browser</strong>
701         <p>Auto-saves name, accounts, budgets, and every payment on this device.</p>
702         <p class="muted small" id="save-dialog-status">Up to date</p>
703       </article>
704       <article class="save-option">
705         <strong>A file on your computer</strong>
706         <p>Best backup ? keep it in Downloads or Documents, then use Open a saved file anytime.</p>
707         <button type="button" class="btn primary block" id="btn-save-json-file">Save as JSON file</button>
708         <button type="button" class="btn secondary block" id="btn-save-csv-file">Save payments as CSV</button>
709       </article>
710     </div>
711     <div class="dialog-actions">
712       <button type="button" class="btn ghost" id="btn-save-dialog-close">Done</button>
713     </div>
714   </div>
715 </dialog>
716
717 <dialog id="export-dialog">
718   <div class="dialog-form">
```

```
719     <h3>Export</h3>
720     <p class="muted">Download your real data (not demo).</p>
721     <div class="export-grid">
722         <button type="button" class="btn secondary" id="btn-export-csv">CSV (current Results filter)</button>
723         <button type="button" class="btn secondary" id="btn-export-all-csv">CSV (all payments)</button>
724         <button type="button" class="btn primary" id="btn-export-json">Full JSON backup</button>
725     </div>
726     <div class="dialog-actions">
727         <button type="button" class="btn ghost" id="btn-export-close">Close</button>
728     </div>
729 </div>
730 </dialog>
731
732 <dialog id="csv-dialog">
733     <div class="dialog-form">
734         <h3>Import bank CSV</h3>
735         <p class="muted">Match your bank's columns, preview a few rows, then import. Duplicates can be skipped.</p>
736         <label class="chip keep-row"><input type="checkbox" id="csv-day-first" checked /> Dates are day-first (India: 18/08/2026)</label>
737         <label class="chip keep-row"><input type="checkbox" id="csv-skip-dupes" checked /> Skip rows that already exist</label>
738         <div class="form-row-2">
739             <label>Date column <select id="csv-col-date"></select></label>
740             <label>Description <select id="csv-col-description"></select></label>
741         </div>
742         <div class="form-row-2">
743             <label>Amount <select id="csv-col-amount"></select></label>
744             <label>Type <select id="csv-col-type"></select></label>
745         </div>
746         <div class="form-row-2">
747             <label>Debit / withdrawal <select id="csv-col-debit"></select></label>
748             <label>Credit / deposit <select id="csv-col-credit"></select></label>
749         </div>
750         <div class="form-row-2">
751             <label>Account <select id="csv-col-account"></select></label>
752             <label>Category <select id="csv-col-category"></select></label>
753         </div>
754         <p class="muted small" id="csv-preview-meta">Preview</p>
755         <div class="table-wrap">
756             <table class="tx-table compact">
757                 <thead>
758                     <tr>
759                         <th>Date</th>
760                         <th>Details</th>
761                         <th class="num">Amount</th>
762                     </tr>
763                 </thead>
764                 <tbody id="csv-preview-body"></tbody>
765             </table>
766         </div>
```

```
767     <p class="form-status" id="csv-status"></p>
768     <div class="dialog-actions">
769         <button type="button" class="btn ghost" id="btn-csv-cancel">Cancel</button>
770         <button type="button" class="btn primary" id="btn-csv-confirm">Import these rows</button>
771     </div>
772 </div>
773 </dialog>
774
775 <div class="toast" id="toast" hidden role="status" aria-live="polite">
776     <span id="toast-text">Saved</span>
777     <button type="button" class="btn tiny secondary" id="toast-action" hidden>Undo</button>
778 </div>
779
780 <script src="core.js"></script>
781 <script src="app.js"></script>
782 <script src="visual.js"></script>
783 </body>
784 </html>
```

app.js

```
1  (() => {
2    const STORAGE_KEY = "payment-modulator-v2";
3    const LEGACY_KEYS = ["payment-modulator-v1", "spend-insight-v1"];
4
5    const CATEGORIES = [
6      "Food & Dining",
7      "Groceries",
8      "Transport",
9      "Shopping",
10     "Entertainment",
11     "Bills & Utilities",
12     "Rent / Housing",
13     "Health",
14     "Subscriptions",
15     "Travel",
16     "Education",
17     "Transfers",
18     "Income",
19     "Other",
20   ];
21
22   const BUDGETABLE = CATEGORIES.filter((c) => c !== "Income" && c !== "Transfers");
23
24   const CATEGORY_COLORS = {
25     "Food & Dining": "#ff8b6a",
26     Groceries: "#4ade80",
27     Transport: "#5ec8f0",
28     Shopping: "#f0b429",
29     Entertainment: "#c084fc",
30     "Bills & Utilities": "#fbbf24",
31     "Rent / Housing": "#94a3b8",
32     Health: "#34d399",
33     Subscriptions: "#fb7185",
34     Travel: "#38bdf8",
35     Education: "#818cf8",
36     Transfers: "#a8a29e",
37     Income: "#6ee7b7",
38     Other: "#cbd5e1",
39   };
40
41   const DEFAULT_BUDGETS = {
42     "Food & Dining": 8000,
43     Groceries: 6000,
44     Transport: 4000,
45     Shopping: 5000,
46     Entertainment: 2500,
```

```
47     "Bills & Utilities": 4000,
48     "Rent / Housing": 22000,
49     Health: 2000,
50     Subscriptions: 1500,
51     Travel: 3000,
52     Education: 2000,
53     Other: 2000,
54 };
55
56 const MERCHANT_HINTS = [
57   { test: /swiggy|zomato|uber\s*eats|dominos|mcdonald|cafe|restaurant|dining/i, cat: "Food & Dining" },
58   { test: /bigbasket|blinkit|zepto|dmart|grocery|supermarket/i, cat: "Groceries" },
59   { test: /uber|ola|metro|irctc|petrol|fuel|rapido/i, cat: "Transport" },
60   { test: /amazon|flipkart|myntra|ajio|meesho/i, cat: "Shopping" },
61   { test: /netflix|spotify|prime|hotstar|youtube/i, cat: "Subscriptions" },
62   { test: /electricity|broadband|airtel|jio|bill/i, cat: "Bills & Utilities" },
63   { test: /rent|housing|society/i, cat: "Rent / Housing" },
64   { test: /pharmacy|apollo|hospital|clinic/i, cat: "Health" },
65   { test: /salary|payroll|payout|refund/i, cat: "Income" },
66   { test: /bookmyshow|pvr|cinema/i, cat: "Entertainment" },
67   { test: /makemytrip|goibibo|hotel|flight/i, cat: "Travel" },
68 ];
69
70 const CURRENCY_LOCALE = { INR: "en-IN", USD: "en-US", EUR: "de-DE", GBP: "en-GB" };
71
72 const DEFAULT_SETTINGS = {
73   name: "",
74   currency: "INR",
75   monthlyIncome: 85000,
76   savingsGoal: 15000,
77   photo: "",
78   phone: "",
79   email: "",
80   city: "",
81   occupation: "",
82   dob: "",
83   gender: "",
84   notes: "",
85   pinHash: "",
86   mobile: "",
87 };
88
89 const PAGE_SIZE = 25;
90 const SESSION_UNLOCK_KEY = "pm-unlocked";
91 const SETUP_LAST_STEP = 5;
92 const IDB_NAME = "payment-modulator";
93 const IDB_STORE = "kv";
94
```

```
95  const $ = (id) => document.getElementById(id);
96  const Core = window.PMCore || {};
97
98  function todayISO() {
99    return Core.toLocalISO ? Core.toLocalISO(new Date()) : new Date().toISOString().slice(0, 10);
100 }
101
102 const state = {
103   setupComplete: false,
104   accounts: ["Primary account"],
105   openingBalances: {},
106   transactions: [],
107   budgets: { ...DEFAULT_BUDGETS },
108   settings: { ...DEFAULT_SETTINGS },
109   setupStep: 1,
110   setupBudgetsDraft: null,
111   view: "track",
112   demoMode: false,
113   period: "month",
114   anchor: todayISO(),
115   account: "all",
116   search: "",
117   trackSearch: "",
118   selected: new Set(),
119   lastSavedAt: null,
120   pinUnlocked: false,
121   trackPage: 0,
122   resultsPage: 0,
123 };
124
125 let demoCache = null;
126 let pendingCsvText = "";
127 let undoTimer = null;
128 let undoPayload = null;
129 let loginBusy = false;
130
131 function uid() {
132   return crypto.randomUUID
133     ? crypto.randomUUID()
134     : `t_${Date.now()}_${Math.random().toString(16).slice(2)}`;
135 }
136
137 function money(n) {
138   const currency = activeSettings().currency || "INR";
139   const value = Core.roundMoney ? Core.roundMoney(n) : Number(n) || 0;
140   return new Intl.NumberFormat(CURRENCY_LOCALE[currency] || "en-IN", {
141     style: "currency",
142     currency,
```

```
143     minimumFractionDigits: 2,
144     maximumFractionDigits: 2,
145   }).format(value);
146 }
147
148 function parseAmount(v) {
149   return Core.parseAmount ? Core.parseAmount(v) : Number(String(v ?? "").replace(/^[^0-9.-]/g, "")) || 0;
150 }
151
152 function txFingerprint(row) {
153   if (Core.fingerprint) return Core.fingerprint(row);
154   const parts = [
155     row.date,
156     row.type,
157     Number(row.amount).toFixed(2),
158     String(row.merchant || "").trim().toLowerCase(),
159     String(row.account || "").trim().toLowerCase(),
160   ];
161   if (row.type === "transfer") {
162     parts.push(String(row.transferTo || "").trim().toLowerCase());
163   }
164   return parts.join("|");
165 }
166
167 function signedDisplay(t) {
168   return t.type === "income" ? t.amount : -t.amount;
169 }
170
171 function defaultBudgetsFor(currency) {
172   return Core.defaultBudgets ? Core.defaultBudgets(currency || "INR") : { ...DEFAULT_BUDGETS };
173 }
174
175 function guessCategory(merchant, type) {
176   if (type === "income") return "Income";
177   if (type === "transfer") return "Transfers";
178   for (const h of MERCHANT_HINTS) if (h.test.test(merchant)) return h.cat;
179   return "Other";
180 }
181
182 function escapeHtml(str) {
183   return String(str)
184     .replace(/&/g, "&amp;")
185     .replace(/</g, "&lt;")
186     .replace(/>/g, "&gt;")
187     .replace(/"/g, "&quot;");
188 }
189
190 function digitsOnly(raw) {
```

```
191     return String(raw || "").replace(/\D/g, "");
192 }
193
194 function normalizeMobile(raw) {
195     const digits = digitsOnly(raw);
196     if (digits.length === 10) return digits;
197     if (digits.length === 12 && digits.startsWith("91")) return digits.slice(2);
198     if (digits.length === 11 && digits.startsWith("0")) return digits.slice(1);
199     return "";
200 }
201
202 function isValidMobile(raw) {
203     return /^d{10}$/.test(normalizeMobile(raw));
204 }
205
206 function mobilesMatch(a, b) {
207     const na = normalizeMobile(a);
208     const nb = normalizeMobile(b);
209     return Boolean(na && nb && na === nb);
210 }
211
212 function formatMobile(raw) {
213     const n = normalizeMobile(raw);
214     if (!n) return String(raw || "").trim();
215     return `+91 ${n.slice(0, 5)} ${n.slice(5)}`;
216 }
217
218 function maskMobile(raw) {
219     const n = normalizeMobile(raw);
220     if (!n) return "?????????";
221     return `+91 ${n.slice(0, 2)}?? ??${n.slice(-3)}`;
222 }
223
224 function storedMobile() {
225     return state.settings.mobile || state.settings.phone || "";
226 }
227
228 function isSessionUnlocked() {
229     try {
230         if (sessionStorage.getItem(SESSION_UNLOCK_KEY) === "1") return true;
231     } catch {
232         /* file:// or blocked storage */
233     }
234     return Boolean(state.pinUnlocked);
235 }
236
237 function setSessionUnlocked(on) {
238     state.pinUnlocked = Boolean(on);
```

```
239     try {
240       if (on) sessionStorage.setItem(SESSION_UNLOCK_KEY, "1");
241       else sessionStorage.removeItem(SESSION_UNLOCK_KEY);
242     } catch {
243       /* in-memory session only */
244     }
245   }
246
247   function syncSettingsMobile(value) {
248     const formatted = formatMobile(value);
249     state.settings.phone = formatted;
250     state.settings.mobile = formatted;
251   }
252
253   function activeTxns() {
254     return state.demoMode ? getDemoData() : state.transactions;
255   }
256
257   function activeSettings() {
258     if (!state.demoMode) return state.settings;
259     return { name: "Demo User", currency: "INR", monthlyIncome: 85000, savingsGoal: 15000 };
260   }
261
262   function activeBudgets() {
263     return state.demoMode ? { ...DEFAULT_BUDGETS } : state.budgets;
264   }
265
266   function activeAccounts() {
267     if (!state.demoMode) return state.accounts;
268     return ["HDFC ****4210", "SBI ****8821"];
269   }
270
271   function backupPayload() {
272     const phone = state.settings.phone || state.settings.mobile || "";
273     return {
274       app: "Niva",
275       version: 4,
276       setupComplete: state.setupComplete,
277       savedAt: new Date().toISOString(),
278       settings: {
279         ...state.settings,
280         phone,
281         mobile: state.settings.mobile || phone,
282         pinHash: state.settings.pinHash || "",
283       },
284       accounts: state.accounts,
285       openingBalances: state.openingBalances,
286       budgets: state.budgets,
```

```
287     transactions: state.transactions,
288   };
289 }
290
291 function formatSavedAt(iso) {
292   if (!iso) return "Not saved yet";
293   const d = new Date(iso);
294   if (Number.isNaN(d.getTime())) return "Saved on this device";
295   return `Saved ${d.toLocaleString()}`;
296 }
297
298 function updateSaveStatus(flash) {
299   const text = state.lastSavedAt ? "Auto-saved" : "Not saved yet";
300   const detail = formatSavedAt(state.lastSavedAt);
301   const status = $("save-status");
302   const dialogStatus = $("save-dialog-status");
303   if (status) {
304     status.textContent = flash ? "Saved ?" : text;
305     status.title = detail;
306     status.classList.toggle("flash", Boolean(flash));
307   }
308   if (dialogStatus) dialogStatus.textContent = detail;
309 }
310
311 function showToast(message, actionLabel, onAction) {
312   const toast = $("toast");
313   const text = $("toast-text");
314   const btn = $("toast-action");
315   if (!toast || !text) {
316     if (message) alert(message);
317     return;
318   }
319   text.textContent = message;
320   toast.hidden = false;
321   if (undoTimer) clearTimeout(undoTimer);
322   if (actionLabel && onAction) {
323     btn.hidden = false;
324     btn.textContent = actionLabel;
325     btn.onclick = () => {
326       toast.hidden = true;
327       onAction();
328     };
329   } else {
330     btn.hidden = true;
331     btn.onclick = null;
332   }
333   undoTimer = setTimeout(() => {
334     toast.hidden = true;
```

```
335     undoPayload = null;
336   }, 8000);
337 }
338
339 function openIdb() {
340   return new Promise((resolve, reject) => {
341     if (!window.indexedDB) {
342       reject(new Error("IndexedDB unavailable"));
343       return;
344     }
345     const req = indexedDB.open(IDB_NAME, 1);
346     req.onupgradeneeded = () => {
347       const db = req.result;
348       if (!db.objectStoreNames.contains(IDB_STORE)) db.createObjectStore(IDB_STORE);
349     };
350     req.onsuccess = () => resolve(req.result);
351     req.onerror = () => reject(req.error);
352   });
353 }
354
355 async function idbSet(payload) {
356   const db = await openIdb();
357   return new Promise((resolve, reject) => {
358     const tx = db.transaction(IDB_STORE, "readwrite");
359     tx.objectStore(IDB_STORE).put(payload, "snapshot");
360     tx.oncomplete = () => resolve();
361     tx.onerror = () => reject(tx.error);
362   });
363 }
364
365 async function idbGet() {
366   const db = await openIdb();
367   return new Promise((resolve, reject) => {
368     const tx = db.transaction(IDB_STORE, "readonly");
369     const req = tx.objectStore(IDB_STORE).get("snapshot");
370     req.onsuccess = () => resolve(req.result || null);
371     req.onerror = () => reject(req.error);
372   });
373 }
374
375 async function idbClear() {
376   try {
377     const db = await openIdb();
378     await new Promise((resolve, reject) => {
379       const tx = db.transaction(IDB_STORE, "readwrite");
380       tx.objectStore(IDB_STORE).delete("snapshot");
381       tx.oncomplete = () => resolve();
382       tx.onerror = () => reject(tx.error);
```

```
383     });
384   } catch {
385     /* ignore */
386   }
387 }
388
389 function applyLoaded(data) {
390   if (!data || typeof data !== "object") return;
391   if (Array.isArray(data.transactions)) {
392     state.transactions = data.transactions.map((t) => ({
393       ...t,
394       amount: parseAmount(t.amount),
395       fingerprint: t.fingerprint || txFingerprint(t),
396     }));
397   }
398   if (data.budgets) state.budgets = { ...DEFAULT_BUDGETS, ...data.budgets };
399   if (data.settings) {
400     state.settings = { ...DEFAULT_SETTINGS, ...data.settings };
401     const m = state.settings.mobile || state.settings.phone || "";
402     state.settings.phone = m;
403     state.settings.mobile = m;
404   }
405   if (Array.isArray(data.accounts) && data.accounts.length) state.accounts = data.accounts;
406   else if (state.transactions.length) {
407     state.accounts = [...new Set(state.transactions.map((t) => t.account).filter(Boolean))];
408   }
409   if (data.openingBalances && typeof data.openingBalances === "object") {
410     state.openingBalances = data.openingBalances;
411   }
412   state.lastSavedAt = data.savedAt || null;
413   state.setupComplete = Boolean(data.setupComplete);
414 }
415
416 async function saveToBrowser() {
417   const savedAt = new Date().toISOString();
418   state.lastSavedAt = savedAt;
419   const payload = { ...backupPayload(), savedAt };
420   let stored = false;
421   try {
422     localStorage.setItem(STORAGE_KEY, JSON.stringify(payload));
423     stored = true;
424   } catch (err) {
425     const quota = err && (err.name === "QuotaExceededError" || err.code === 22);
426     if (quota && payload.settings?.photo) {
427       try {
428         const slim = JSON.parse(JSON.stringify(payload));
429         slim.settings.photo = "";
430         localStorage.setItem(STORAGE_KEY, JSON.stringify(slim));
```

```
431         stored = true;
432         showToast("Storage is full ? photo was not kept in browser storage. Download a file backup.");
433     } catch {
434         showToast("This browser is out of space. Download a JSON backup so you do not lose payments.");
435     }
436 } else {
437     showToast("Could not save in this browser. Download a file backup.");
438 }
439 }
440 try {
441     await idbSet(payload);
442     stored = true;
443 } catch {
444     /* localStorage may have worked */
445 }
446 if (stored) updateSaveStatus(true);
447 }
448
449 function save() {
450     saveToBrowser();
451 }
452
453 function saveToFile(kind) {
454     if (state.demoMode) {
455         alert("Exit demo first ? demo data is not your real list.");
456         return;
457     }
458     if (kind === "csv") {
459         download("niva-payments.csv", toCsv(state.transactions), "text/csv");
460         return;
461     }
462     download(
463         `niva-${todayISO()}.json`,
464         JSON.stringify(backupPayload(), null, 2),
465         "application/json"
466     );
467 }
468
469 function withTimeout(promise, ms) {
470     return Promise.race([
471         promise,
472         new Promise((_, reject) => setTimeout(() => reject(new Error("timeout")), ms)),
473     ]);
474 }
475
476 function loadFromLocalStorage() {
477     try {
478         let raw = localStorage.getItem(STORAGE_KEY);
```

```
479     if (!raw) {
480       for (const key of LEGACY_KEYS) {
481         const legacy = localStorage.getItem(key);
482         if (legacy) {
483           raw = legacy;
484           break;
485         }
486       }
487     }
488     if (raw) applyLoaded(JSON.parse(raw));
489   } catch {
490     /* ignore */
491   }
492 }
493
494 async function loadFromIdb() {
495   try {
496     const data = await withTimeout(idbGet(), 600);
497     if (data) {
498       applyLoaded(data);
499       return true;
500     }
501   } catch {
502     /* file:// or slow IDB */
503   }
504   return false;
505 }
506
507 function getDemoData() {
508   if (demoCache) return demoCache;
509   const year = new Date().getFullYear();
510   const month = String(new Date().getMonth() + 1).padStart(2, "0");
511   const prev = new Date();
512   prev.setMonth(prev.getMonth() - 1);
513   const pm = String(prev.getMonth() + 1).padStart(2, "0");
514   const py = prev.getFullYear();
515   const a = ["HDFC ****4210", "SBI ****8821"];
516   const rows = [
517     ["01", "Salary Credit", 85000, "income", "Income", a[0]],
518     ["02", "House Rent", 22000, "expense", "Rent / Housing", a[0]],
519     ["03", "BigBasket", 3200, "expense", "Groceries", a[0]],
520     ["04", "Swiggy", 540, "expense", "Food & Dining", a[1]],
521     ["05", "Uber", 280, "expense", "Transport", a[1]],
522     ["06", "Netflix", 649, "expense", "Subscriptions", a[0]],
523     ["07", "Amazon", 2499, "expense", "Shopping", a[0]],
524     ["08", "Spotify", 119, "expense", "Subscriptions", a[0]],
525     ["09", "Zomato", 780, "expense", "Food & Dining", a[1]],
526     ["10", "Electricity Bill", 2100, "expense", "Bills & Utilities", a[0]],
```

```
527     ["11", "To SBI pocket", 5000, "transfer", "Transfers", a[0], a[1]],
528     ["12", "Petrol Pump", 2500, "expense", "Transport", a[0]],
529     ["14", "Swiggy", 420, "expense", "Food & Dining", a[1]],
530     ["16", "BookMyShow", 900, "expense", "Entertainment", a[1]],
531     ["18", "Airtel Broadband", 999, "expense", "Bills & Utilities", a[0]],
532     ["20", "Zomato", 560, "expense", "Food & Dining", a[1]],
533     ["22", "Cafe Coffee Day", 370, "expense", "Food & Dining", a[1]],
534     ["24", "DMart", 2750, "expense", "Groceries", a[0]],
535     ["26", "Swiggy", 640, "expense", "Food & Dining", a[1]],
536     ["28", "Freelance Payout", 15000, "income", "Income", a[1]],
537 ];
538 const last = [
539     ["01", "Salary Credit", 85000, "income", "Income", a[0]],
540     ["02", "House Rent", 22000, "expense", "Rent / Housing", a[0]],
541     ["06", "Netflix", 649, "expense", "Subscriptions", a[0]],
542     ["08", "Swiggy", 1100, "expense", "Food & Dining", a[1]],
543     ["15", "Airtel Broadband", 999, "expense", "Bills & Utilities", a[0]],
544 ];
545 const map =
546     (y, m) =>
547     ([day, merchant, amount, type, category, account, transferTo]) => ({
548         id: `demo_${y}${m}${day}_${merchant}`,
549         date: `${y}-${m}-${day}`,
550         merchant,
551         amount,
552         type,
553         category,
554         account,
555         transferTo: transferTo || "",
556         note: "Demo sample",
557     });
558 demoCache = [...rows.map(map(year, month)), ...last.map(map(py, pm))];
559 return demoCache;
560 }
561
562 /* ----- screens / navigation ----- */
563
564 function showScreen(name) {
565     ["welcome", "setup", "login", "app"].forEach((n) => {
566         const el = $(`screen-${n}`);
567         if (!el) return;
568         const show = n === name;
569         el.hidden = !show;
570         if (show) el.removeAttribute("hidden");
571         else el.setAttribute("hidden", "");
572     });
573 }
574
```

```
575 function setLoginStatus(message) {
576     const status = $("login-status");
577     if (status) status.textContent = message || "";
578 }
579
580 function prepareLoginScreen(message) {
581     const hasPin = Boolean(state.settings.pinHash);
582     const hasMobile = Boolean(normalizeMobile(storedMobile()));
583     const btnLabel = $("login-btn-label");
584     const sub = $("login-btn-sub");
585     if (btnLabel) btnLabel.textContent = hasPin ? "Unlock" : "Save PIN and unlock";
586     else if ($("btn-login")) $("btn-login").textContent = hasPin ? "Unlock" : "Save PIN and unlock";
587     if (sub) {
588         sub.textContent = hasPin
589             ? "Stays unlocked until you close this tab"
590             : "Required next time you open Niva here";
591     }
592     if ($("login-confirm-wrap")) $("login-confirm-wrap").hidden = hasPin;
593     if ($("login-copy")) {
594         $("login-copy").textContent = !hasPin
595             ? "Set a mobile number and PIN to lock this browser?s profile."
596             : !hasMobile
597             ? "Enter your mobile number and PIN to continue."
598             : "Enter the mobile and PIN for this browser?s profile.";
599     }
600     if ($("login-reset-copy")) {
601         $("login-reset-copy").textContent = hasMobile
602             ? "Type the mobile number on this profile to erase everything here."
603             : "Type RESET THIS DEVICE to erase everything in this browser.";
604     }
605     if ($("login-reset-confirm")) {
606         $("login-reset-confirm").placeholder = hasMobile ? "Mobile number" : "RESET THIS DEVICE";
607     }
608     if ($("login-reset-panel")) $("login-reset-panel").hidden = true;
609     if ($("login-reset-confirm")) $("login-reset-confirm").value = "";
610     if ($("login-reset-status")) $("login-reset-status").textContent = "";
611     const createSub = $("create-profile-sub");
612     if (createSub) {
613         createSub.textContent = hasPin
614             ? "Erases the current profile on this browser"
615             : "First time here ? about a minute";
616     }
617     setLoginStatus(message || "");
618 }
619
620 function showLogin(message) {
621     showScreen("login");
622     if ($("login-mobile")) $("login-mobile").value = "";
```

```
623     if ($("#login-pin")) $("#login-pin").value = "";
624     if ($("#login-pin-confirm")) $("#login-pin-confirm").value = "";
625     prepareLoginScreen(message);
626 }
627
628 function logoutSession() {
629     setSessionUnlocked(false);
630     state.demomode = false;
631     showLogin();
632 }
633
634 async function wipeAllData() {
635     try {
636         localStorage.removeItem(STORAGE_KEY);
637         for (const key of LEGACY_KEYS) localStorage.removeItem(key);
638     } catch {
639         /* ignore */
640     }
641     await idbClear();
642     setSessionUnlocked(false);
643     state.setupComplete = false;
644     state.transactions = [];
645     state.accounts = ["Primary account"];
646     state.openingBalances = {};
647     state.budgets = { ...DEFAULT_BUDGETS };
648     state.settings = { ...DEFAULT_SETTINGS };
649     state.pinUnlocked = false;
650     state.demomode = false;
651     state.selected = new Set();
652     state.lastSavedAt = null;
653 }
654
655 async function submitLogin(e) {
656     if (e && typeof e.preventDefault === "function") e.preventDefault();
657     if (loginBusy) return;
658     loginBusy = true;
659     try {
660         const mobileRaw = ($("#login-mobile")?.value || "");
661         const pin = ($("#login-pin")?.value || "");
662         const confirmPin = ($("#login-pin-confirm")?.value || "");
663         const stored = storedMobile();
664         const hasStoredMobile = Boolean(normalizeMobile(stored));
665         const hasPin = Boolean(state.settings.pinHash);
666
667         if (!isValidMobile(mobileRaw)) {
668             setLoginStatus("Enter a valid 10-digit Indian mobile number (or +91).");
669             return;
670         }
```

```
671
672   if (hasStoredMobile && !mobilesMatch(mobileRaw, stored)) {
673       setLoginStatus(
674         "This device already has a profile. Log in with that mobile and PIN, or reset the app from login."
675       );
676       if ($("#login-pin")) $("#login-pin").value = "";
677       return;
678   }
679
680   if (hasPin) {
681       if (!(await verifyPin(pin, mobileRaw))) {
682           setLoginStatus("Wrong PIN. Try again.");
683           if ($("#login-pin")) $("#login-pin").value = "";
684           return;
685       }
686   } else {
687       if (!/^\d{4,8}$/.test(pin)) {
688           setLoginStatus("Create a 4-8 digit PIN.");
689           return;
690       }
691       if (pin !== confirmPin) {
692           setLoginStatus("PINs do not match.");
693           return;
694       }
695       state.settings.pinHash = await hashPin(pin, mobileRaw);
696   }
697
698   syncSettingsMobile(mobileRaw);
699   if (!hasStoredMobile || !hasPin) save();
700
701   setSessionUnlocked(true);
702   if ($("#login-pin")) $("#login-pin").value = "";
703   if ($("#login-pin-confirm")) $("#login-pin-confirm").value = "";
704   if ($("#login-mobile")) $("#login-mobile").value = "";
705   setLoginStatus("");
706   showScreen("app");
707   setView("track");
708 } catch (err) {
709   console.error(err);
710   setLoginStatus("Could not log in. Try again.");
711 } finally {
712   loginBusy = false;
713 }
714 }
715
716 async function confirmDeviceReset() {
717   const typed = ($("#login-reset-confirm")?.value || "");
718   const stored = storedMobile();
```

```
719     const status = $("login-reset-status");
720     const hasMobile = Boolean(normalizeMobile(stored));
721     const ok = hasMobile
722       ? mobilesMatch(typed, stored)
723       : String(typed).trim().toUpperCase() === "RESET THIS DEVICE";
724     if (!ok) {
725       if (status) {
726         status.textContent = hasMobile
727           ? "That number does not match this device?s profile."
728           : "Type RESET THIS DEVICE to confirm.";
729       }
730       return;
731     }
732     if (!confirm("Erase all payments and the profile on this device? This cannot be undone.)) return;
733     await wipeAllData();
734     showScreen("welcome");
735   }
736
737   function personalizeChrome() {
738     const name = state.settings.name?.trim();
739     const first = name ? name.split(" ")[0] : "";
740     if ($("app-subtitle")) {
741       $("app-subtitle").textContent = state.demoMode
742         ? "Demo"
743         : first
744         ? `Hi, ${first}`
745         : "Know where it goes";
746     }
747     if ($("track-greet")) {
748       $("track-greet").textContent = first ? `Hey ${first}` : "Welcome";
749     }
750     if ($("track-heading")) {
751       $("track-heading").textContent = state.transactions.length ? "Add another" : "Add your first spend";
752     }
753     if ($("next-action")) {
754       $("next-action").textContent = state.transactions.length
755         ? "Amount ? who ? Save. Tap Insights for charts."
756         : "Enter amount, pick Spent or Received, then Save.";
757     }
758     applyPhoto(state.demoMode ? "" : state.settings.photo);
759   }
760
761   function applyPhoto(src) {
762     const header = $("header-photo");
763     const mark = header?.parentElement;
764     const profile = $("profile-photo");
765     const fallback = $("profile-photo-fallback");
766     const has = Boolean(src);
```

```
767     if (header) {
768         header.hidden = !has;
769         header.src = src || "";
770         header.alt = has ? "Your photo" : "";
771     }
772     if (mark) mark.classList.toggle("has-photo", has);
773     if (profile) {
774         profile.hidden = !has;
775         profile.src = src || "";
776         profile.alt = has ? "Profile photo" : "";
777     }
778     if (fallback) fallback.hidden = has;
779 }
780
781 function resizePhoto(file) {
782     return new Promise((resolve, reject) => {
783         if (!file || !file.type.startsWith("image/")) {
784             reject(new Error("Please choose a photo file."));
785             return;
786         }
787         if (file.size > 8 * 1024 * 1024) {
788             reject(new Error("Please use a photo smaller than 8 MB."));
789             return;
790         }
791         const img = new Image();
792         const url = URL.createObjectURL(file);
793         img.onload = () => {
794             URL.revokeObjectURL(url);
795             const size = 320;
796             const canvas = document.createElement("canvas");
797             canvas.width = size;
798             canvas.height = size;
799             const ctx = canvas.getContext("2d");
800             const min = Math.min(img.width, img.height);
801             const sx = (img.width - min) / 2;
802             const sy = (img.height - min) / 2;
803             ctx.drawImage(img, sx, sy, min, min, 0, 0, size, size);
804             resolve(canvas.toDataURL("image/jpeg", 0.82));
805         };
806         img.onerror = () => {
807             URL.revokeObjectURL(url);
808             reject(new Error("Could not read that photo."));
809         };
810         img.src = url;
811     });
812 }
813
814 function setView(view) {
```

```
815     if (view !== "demo" && state.setupComplete && !isSessionUnlocked()) {
816         showLogin();
817         return;
818     }
819     const intro = $("view-results").querySelector(".intro-panel");
820
821     if (view === "demo") {
822         state.demoMode = true;
823         state.view = "demo";
824         document.querySelectorAll(".main-tabs .tab").forEach((tab) => {
825             const on = tab.dataset.view === "results";
826             tab.classList.toggle("active", on);
827             tab.setAttribute("aria-selected", on ? "true" : "false");
828         });
829         $("view-track").hidden = true;
830         $("view-setup-edit").hidden = true;
831         $("view-demo").hidden = false;
832         $("view-results").hidden = false;
833         $("demo-banner").hidden = false;
834         personalizeChrome();
835         if (intro) intro.hidden = true;
836         renderAllResults();
837         return;
838     }
839
840     if (state.demoMode) {
841         if (view === "results") {
842             state.view = "results";
843             document.querySelectorAll(".main-tabs .tab").forEach((tab) => {
844                 const on = tab.dataset.view === "results";
845                 tab.classList.toggle("active", on);
846                 tab.setAttribute("aria-selected", on ? "true" : "false");
847             });
848             $("view-track").hidden = true;
849             $("view-setup-edit").hidden = true;
850             $("view-demo").hidden = false;
851             $("view-results").hidden = false;
852             $("demo-banner").hidden = false;
853             personalizeChrome();
854             if (intro) intro.hidden = true;
855             renderAllResults();
856             return;
857         }
858         if (view === "track" || view === "setup-edit") {
859             alert("Exit demo first ? tap Exit demo on the banner.");
860             return;
861         }
862     }
```

```
863
864     state.demoMode = false;
865     state.view = view;
866     $("demo-banner").hidden = true;
867     personalizeChrome();
868     if (intro) intro.hidden = false;
869
870     ["track", "results", "demo", "setup-edit"].forEach((v) => {
871         const el = $(`view-${v}`);
872         if (el) el.hidden = v !== view;
873     });
874     document.querySelectorAll(".main-tabs .tab").forEach((tab) => {
875         const on = tab.dataset.view === view;
876         tab.classList.toggle("active", on);
877         tab.setAttribute("aria-selected", on ? "true" : "false");
878     });
879
880     if (view === "results") renderAllResults();
881     if (view === "track") renderTrack();
882     if (view === "setup-edit") {
883         fillEditForm();
884         syncAccountSecurityUI();
885     }
886 }
887
888 function boot() {
889     if (!state.setupComplete) {
890         showScreen("welcome");
891         return;
892     }
893     if (!isSessionUnlocked()) {
894         showLogin();
895         return;
896     }
897     showScreen("app");
898     setView("track");
899 }
900
901 /* ----- setup wizard ----- */
902
903 function renderAccountFields(containerId, values) {
904     const box = $(containerId);
905     box.innerHTML = "";
906     const list = values.length ? values : [""];
907     list.forEach((val, i) => {
908         const name = typeof val === "string" ? val : val.name || "";
909         const opening =
910             typeof val === "object" && val && "opening" in val
```

```
911     ? val.opening
912     : state.openingBalances[name] || "";
913     const row = document.createElement("div");
914     row.className = "account-row";
915     row.innerHTML = `
916       <input type="text" data-account-input value="${escapeHtml(name)}" placeholder="e.g. HDFC ****4210" ${
917         i === 0 ? "required" : ""
918       } />
919       <input type="number" data-opening step="0.01" min="0" inputmode="decimal" value="${
920         opening === "" ? "" : escapeHtml(String(opening))
921       }" placeholder="Opening ?" title="Opening balance" />
922       <button type="button" class="btn ghost tiny" data-remove-account ${list.length === 1 ? "disabled" : ""}>Remove</button>
923     `;
924     box.appendChild(row);
925   });
926 }
927
928 function readAccountFields(containerId) {
929   return [...$(containerId).querySelectorAll("[data-account-input]")]
930     .map((i) => i.value.trim())
931     .filter(Boolean);
932 }
933
934 function readOpeningBalances(containerId) {
935   const out = {};
936   $(containerId).querySelectorAll(".account-row").forEach((row) => {
937     const name = row.querySelector("[data-account-input]")?.value.trim();
938     if (!name) return;
939     out[name] = parseAmount(row.querySelector("[data-opening]")?.value);
940   });
941   return out;
942 }
943
944 function snapshotAccountRows(containerId) {
945   return [...$(containerId).querySelectorAll(".account-row")].map((row) => ({
946     name: row.querySelector("[data-account-input]")?.value || "",
947     opening: row.querySelector("[data-opening]")?.value ?? "",
948   }));
949 }
950
951 function renderBudgetFields(containerId, budgets) {
952   $(containerId).innerHTML = BUDGETABLE.map(
953     (c) => `
954       <label>
955         <span>${escapeHtml(c)}</span>
956         <input type="number" min="0" step="100" data-budget="${escapeHtml(c)}" value="${budgets[c] ?? 0}" />
957       </label>`
958   ).join("");

```

```
959   }
960
961   function readBudgets(containerId) {
962     const out = { ...DEFAULT_BUDGETS };
963     $(containerId).querySelectorAll("[data-budget]").forEach((input) => {
964       out[input.getAttribute("data-budget")] = parseAmount(input.value) || 0;
965     });
966     return out;
967   }
968
969   function updateSetupUI() {
970     const step = state.setupStep;
971     const labels = {
972       1: "Step 1 of 5 ? about you",
973       2: "Step 2 of 5 ? mobile & PIN",
974       3: "Step 3 of 5 ? income & savings",
975       4: "Step 4 of 5 ? accounts",
976       5: "Step 5 of 5 ? budgets",
977     };
978     if ($("#setup-step-label")) $("#setup-step-label").textContent = labels[step] || `Step ${step} of 5`;
979     if ($("#setup-progress")) $("#setup-progress").style.width = `${step * 20}%`;
980     document.querySelectorAll(".step-pill").forEach((el) => {
981       el.classList.toggle("on", Number(el.dataset.pill) === step);
982     });
983     document.querySelectorAll(".wizard-step").forEach((el) => {
984       el.hidden = Number(el.dataset.step) !== step;
985     });
986     if ($("#btn-setup-back")) $("#btn-setup-back").hidden = step === 1;
987     if ($("#btn-setup-next")) {
988       $("#btn-setup-next").textContent = step === SETUP_LAST_STEP ? "Done ? go to Home" : "Continue";
989     }
990   }
991
992   function beginSetupWizard() {
993     state.setupStep = 1;
994     state.setupBudgetsDraft = null;
995     showScreen("setup");
996     try {
997       if ($("#setup-name")) $("#setup-name").value = state.settings.name || "";
998       if ($("#setup-currency")) $("#setup-currency").value = state.settings.currency || "INR";
999       if ($("#setup-mobile")) $("#setup-mobile").value = storedMobile();
1000       if ($("#setup-pin")) $("#setup-pin").value = "";
1001       if ($("#setup-pin-confirm")) $("#setup-pin-confirm").value = "";
1002       if ($("#setup-income")) $("#setup-income").value = state.settings.monthlyIncome || "";
1003       if ($("#setup-savings")) $("#setup-savings").value = state.settings.savingsGoal || "";
1004       renderAccountFields("setup-accounts", state.accounts.length ? state.accounts : ["Primary account"]);
1005       renderBudgetFields(
1006         "setup-budgets",
```

```
1007         state.budgets.Other != null ? state.budgets : defaultBudgetsFor($"setup-currency").value)
1008     );
1009     updateSetupUI();
1010 } catch (err) {
1011     console.error(err);
1012 }
1013 }
1014
1015 function startSetup() {
1016     if (state.setupComplete) {
1017         showLogin(
1018             "This browser already has a profile. Log in with that mobile and PIN, or create a new profile below."
1019         );
1020         return;
1021     }
1022     beginSetupWizard();
1023 }
1024
1025 async function createNewProfile() {
1026     if (state.setupComplete) {
1027         const ok = confirm(
1028             "This browser already has a profile. Creating a new one will erase all payments and settings on this device. Continue?"
1029         );
1030         if (!ok) return;
1031         await wipeAllData();
1032         state.settings = { ...DEFAULT_SETTINGS };
1033         state.accounts = ["Primary account"];
1034         state.openingBalances = {};
1035         state.budgets = { ...DEFAULT_BUDGETS };
1036         state.transactions = [];
1037         state.selected = new Set();
1038         state.demoMode = false;
1039     }
1040     beginSetupWizard();
1041 }
1042
1043 function openDemo() {
1044     showScreen("app");
1045     setView("demo");
1046 }
1047
1048 function validateSetupStep() {
1049     if (state.setupStep === 1) {
1050         if (!$("setup-name").value.trim()) {
1051             alert("Please enter your name.");
1052             return false;
1053         }
1054     }
1055 }
```

```
1055     if (state.setupStep === 2) {
1056       if (!isValidMobile($("#setup-mobile")?.value)) {
1057         alert("Enter a valid 10-digit mobile number (or +91).");
1058         return false;
1059       }
1060       const a = ($("#setup-pin")?.value || "");
1061       const b = ($("#setup-pin-confirm")?.value || "");
1062       if (!state.settings.pinHash || a || b) {
1063         if (!/^d{4,8}$/.test(a)) {
1064           alert("Choose a 4-8 digit PIN.");
1065           return false;
1066         }
1067         if (a !== b) {
1068           alert("PINs do not match.");
1069           return false;
1070         }
1071       }
1072     }
1073     if (state.setupStep === 4) {
1074       const accounts = readAccountFields("setup-accounts");
1075       if (!accounts.length) {
1076         alert("Add at least one bank account name.");
1077         return false;
1078       }
1079     }
1080     return true;
1081   }
1082
1083   async function finishSetup() {
1084     const mobileRaw = ($("#setup-mobile")?.value || storedMobile());
1085     if (!isValidMobile(mobileRaw)) {
1086       alert("Enter a valid mobile number.");
1087       state.setupStep = 2;
1088       updateSetupUI();
1089       return;
1090     }
1091     const pin = ($("#setup-pin")?.value || "");
1092     const confirmPin = ($("#setup-pin-confirm")?.value || "");
1093     if (pin) {
1094       if (!/^d{4,8}$/.test(pin) || pin !== confirmPin) {
1095         alert("Check your PIN and confirmation.");
1096         state.setupStep = 2;
1097         updateSetupUI();
1098         return;
1099       }
1100       state.settings.pinHash = await hashPin(pin, mobileRaw);
1101       if ($("#setup-pin")) $("#setup-pin").value = "";
1102       if ($("#setup-pin-confirm")) $("#setup-pin-confirm").value = "";
```

```
1103     }
1104     if (!state.settings.pinHash) {
1105         alert("Set a 4?8 digit PIN to continue.");
1106         state.setupStep = 2;
1107         updateSetupUI();
1108         return;
1109     }
1110     syncSettingsMobile(mobileRaw);
1111     state.settings = {
1112         ...DEFAULT_SETTINGS,
1113         ...state.settings,
1114         name: $("setup-name").value.trim() || "",
1115         currency: $("setup-currency").value || "INR",
1116         monthlyIncome: parseAmount($("setup-income").value) || 0,
1117         savingsGoal: parseAmount($("setup-savings").value) || 0,
1118         phone: state.settings.phone,
1119         mobile: state.settings.mobile,
1120         pinHash: state.settings.pinHash,
1121     };
1122     state.accounts = readAccountFields("setup-accounts");
1123     state.openingBalances = readOpeningBalances("setup-accounts");
1124     state.budgets = readBudgets("setup-budgets");
1125     state.setupBudgetsDraft = null;
1126     state.setupComplete = true;
1127     state.demoMode = false;
1128     setSessionUnlocked(true);
1129     save();
1130     showScreen("app");
1131     setView("track");
1132     if ($("form-status")) $("form-status").textContent = "You're set. Add your first payment below.";
1133 }
1134
1135 /* ----- analytics helpers ----- */
1136
1137 function periodBounds(period, anchor) {
1138     const d = new Date(`${anchor}T12:00:00`);
1139     if (period === "day") {
1140         const start = new Date(d);
1141         start.setHours(0, 0, 0, 0);
1142         const end = new Date(d);
1143         end.setHours(23, 59, 59, 999);
1144         return { start, end };
1145     }
1146     if (period === "year") {
1147         return {
1148             start: new Date(d.getFullYear(), 0, 1),
1149             end: new Date(d.getFullYear(), 11, 31, 23, 59, 59, 999),
1150         };
1151     }
1152 }
```

```
1151     }
1152     return {
1153       start: new Date(d.getFullYear(), d.getMonth(), 1),
1154       end: new Date(d.getFullYear(), d.getMonth() + 1, 0, 23, 59, 59, 999),
1155     };
1156   }
1157
1158   function shiftAnchor(delta) {
1159     state.anchor = Core.shiftLocalISO
1160       ? Core.shiftLocalISO(state.anchor, state.period, delta)
1161       : todayISO();
1162     $("anchor-date").value = state.anchor;
1163   }
1164
1165   function previousAnchor() {
1166     return Core.shiftLocalISO ? Core.shiftLocalISO(state.anchor, state.period, -1) : state.anchor;
1167   }
1168
1169   function inRange(iso, start, end) {
1170     const t = new Date(`${iso}T12:00:00`).getTime();
1171     return t >= start.getTime() && t <= end.getTime();
1172   }
1173
1174   function filtered(extraAnchor = state.anchor) {
1175     const { start, end } = periodBounds(state.period, extraAnchor);
1176     const q = state.search.trim().toLowerCase();
1177     return activeTxns()
1178       .filter((t) => inRange(t.date, start, end))
1179       .filter((t) => (state.account === "all" ? true : t.account === state.account))
1180       .filter((t) => {
1181         if (!q) return true;
1182         return (
1183           t.merchant.toLowerCase().includes(q) ||
1184           (t.note || "").toLowerCase().includes(q) ||
1185           t.category.toLowerCase().includes(q)
1186         );
1187       })
1188       .sort((a, b) => (a.date < b.date ? 1 : -1));
1189   }
1190
1191   function categoryTotals(txns) {
1192     const map = {};
1193     for (const t of txns) {
1194       if (t.type !== "expense") continue;
1195       map[t.category] = (map[t.category] || 0) + t.amount;
1196     }
1197     return Object.entries(map)
1198       .map(([name, amount]) => ({ name, amount }));
```

```
1199     .sort((a, b) => b.amount - a.amount);
1200   }
1201
1202   function budgetCap(category) {
1203     const monthly = Number(activeBudgets()[category]) || 0;
1204     if (state.period === "year") return monthly * 12;
1205     if (state.period === "day") return monthly / 30;
1206     return monthly;
1207   }
1208
1209   function scaleIncome(monthly) {
1210     if (state.period === "year") return monthly * 12;
1211     if (state.period === "day") return monthly / 30;
1212     return monthly;
1213   }
1214
1215   function healthScore(txs) {
1216     const settings = activeSettings();
1217     const spent = txs.filter((t) => t.type === "expense").reduce((s, t) => s + t.amount, 0);
1218     const income =
1219       txs.filter((t) => t.type === "income").reduce((s, t) => s + t.amount, 0) ||
1220       scaleIncome(settings.monthlyIncome);
1221     let score = 100;
1222     if (income > 0) {
1223       const rate = spent / income;
1224       if (rate > 1) score -= 45;
1225       else if (rate > 0.85) score -= 30;
1226       else if (rate > 0.7) score -= 15;
1227     }
1228     for (const c of categoryTotals(txs)) {
1229       const cap = budgetCap(c.name);
1230       if (cap > 0 && c.amount > cap) score -= 8;
1231     }
1232     const savings = income - spent;
1233     const goal = scaleIncome(settings.savingsGoal || 0);
1234     if (goal && savings < goal) score -= 8;
1235     if (goal && savings >= goal) score += 6;
1236     return Math.max(0, Math.min(100, Math.round(score)));
1237   }
1238
1239   function trendBuckets(txs) {
1240     const { end } = periodBounds(state.period, state.anchor);
1241     const labels = [];
1242     if (state.period === "day") {
1243       // single day total as one bar for clarity
1244       labels.push("Day");
1245     } else if (state.period === "year") {
1246       for (let m = 0; m < 12; m++) labels.push(new Date(2024, m, 1).toLocaleString("en", { month: "short" }));
```

```
1247   } else {
1248     for (let i = 1; i <= end.getDate(); i++) labels.push(String(i));
1249   }
1250   const sums = Object.fromEntries(labels.map((l) => [l, 0]));
1251   for (const t of txs) {
1252     if (t.type !== "expense") continue;
1253     let key;
1254     if (state.period === "day") key = "Day";
1255     else if (state.period === "year") {
1256       key = new Date(`${t.date}T12:00:00`).toLocaleString("en", { month: "short" });
1257     } else key = String(new Date(`${t.date}T12:00:00`).getDate());
1258     if (key in sums) sums[key] += t.amount;
1259   }
1260   return labels.map((label) => ({ label, amount: sums[label] || 0 }));
1261 }
1262
1263 function buildInsights(txs) {
1264   const settings = activeSettings();
1265   const expenses = txs.filter((t) => t.type === "expense");
1266   const spent = expenses.reduce((s, t) => s + t.amount, 0);
1267   const income = txs.filter((t) => t.type === "income").reduce((s, t) => s + t.amount, 0);
1268   const cats = categoryTotals(txs);
1269   const tips = [];
1270   if (!expenses.length) {
1271     return [
1272       {
1273         kind: "tip",
1274         icon: "i",
1275         title: "No expenses in this view",
1276         body: "Open Home, save a few spends, then come back.",
1277       },
1278     ];
1279   }
1280   const prevSpent = filtered(previousAnchor())
1281     .filter((t) => t.type === "expense")
1282     .reduce((s, t) => s + t.amount, 0);
1283   if (prevSpent > 0) {
1284     const pct = Math.round(((spent - prevSpent) / prevSpent) * 100);
1285     tips.push({
1286       kind: pct > 10 ? "warn" : "tip",
1287       icon: "?",
1288       title: pct >= 0 ? `Spend up ${pct}% vs last ${state.period}` : `Spend down ${Math.abs(pct)}% vs last ${state.period}`,
1289       body: `Previous: ${money(prevSpent)}. Now: ${money(spent)}.`,
1290     });
1291   }
1292   if (cats[0]) {
1293     const share = Math.round((cats[0].amount / spent) * 100);
1294     tips.push({
```

```
1295     kind: share >= 35 ? "warn" : "tip",
1296     icon: "%",
1297     title: `${cats[0].name} is ${share}% of spending`,
1298     body: share >= 35 ? `Try capping this near 25% to free ~${money(cats[0].amount * 0.3)}.` : `~${money(cats[0].amount)} went here.`,
1299   });
1300 }
1301 for (const c of cats) {
1302   const cap = budgetCap(c.name);
1303   if (cap && c.amount > cap) {
1304     tips.push({
1305       kind: "warn",
1306       icon: "B",
1307       title: `${c.name} over budget`,
1308       body: `Spent ${money(c.amount)} vs ${money(cap)}. Cut ${money(c.amount - cap)}.`,
1309     });
1310     break;
1311   }
1312 }
1313 const food = cats.find((c) => c.name === "Food & Dining");
1314 const foodCap = budgetCap("Food & Dining");
1315 const foodThreshold =
1316   foodCap > 0 ? foodCap * 0.75 : Math.max(scaleIncome(settings.monthlyIncome) * 0.08, 500);
1317 if (food && food.amount > foodThreshold) {
1318   tips.push({
1319     kind: "warn",
1320     icon: "F",
1321     title: "Dining looks high",
1322     body: `Food & dining is ${money(food.amount)}${foodCap ? ` (budget ${money(foodCap)})` : ""}. Cooking more meals could save ~${money(food.amount * 0.25)}.`,
1323   });
1324 }
1325 const effIncome = income || scaleIncome(settings.monthlyIncome);
1326 if (effIncome > 0) {
1327   const rate = spent / effIncome;
1328   tips.push({
1329     kind: rate > 0.8 ? "warn" : "ok",
1330     icon: "?",
1331     title: "Income vs spend",
1332     body: `You used ${Math.round(rate * 100)}% of income capacity. Buffer: ${money(effIncome - spent)}.`,
1333   });
1334 }
1335 return tips.slice(0, 6);
1336 }
1337
1338 function detectRecurring(txs) {
1339   const map = {};
1340   for (const t of txs.filter((x) => x.type === "expense")) {
1341     const k = t.merchant.trim().toLowerCase();
1342     if (!map[k]) map[k] = [];
```

```
1343     map[k].push(t);
1344   }
1345   const out = [];
1346   for (const list of Object.values(map)) {
1347     if (list.length < 2) continue;
1348     list.sort((a, b) => (a.date < b.date ? -1 : 1));
1349     const amounts = list.map((t) => t.amount);
1350     const avg = amounts.reduce((s, t) => s + t, 0) / amounts.length;
1351     const similar = amounts.every((a) => Math.abs(a - avg) / Math.max(avg, 1) <= 0.15);
1352     if (!similar) continue;
1353     let intervalOk = false;
1354     for (let i = 1; i < list.length; i++) {
1355       const prev = Core.fromLocalISO ? Core.fromLocalISO(list[i - 1].date) : new Date(list[i - 1].date);
1356       const cur = Core.fromLocalISO ? Core.fromLocalISO(list[i].date) : new Date(list[i].date);
1357       const days = Math.round((cur - prev) / 86400000);
1358       if (days >= 6 && days <= 35) intervalOk = true;
1359     }
1360     if (!intervalOk && list.length < 3) continue;
1361     out.push({
1362       merchant: list[0].merchant,
1363       amount: Math.round(avg),
1364       count: list.length,
1365       category: list[0].category,
1366     });
1367   }
1368   return out.sort((a, b) => b.count - a.count).slice(0, 5);
1369 }
1370
1371 /* ----- render track ----- */
1372
1373 function fillCategorySelect() {
1374   const opts = CATEGORIES.map((c) => `<option value="${escapeHtml(c)}">${escapeHtml(c)}</option>`).join("");
1375   if ($("#q-category")) $("#q-category").innerHTML = opts;
1376   if ($("#e-category")) $("#e-category").innerHTML = opts;
1377   if ($("#b-category")) $("#b-category").innerHTML = `<option value="">Keep current</option>` + opts;
1378 }
1379
1380 function fillAccountSelect() {
1381   const accounts = activeAccounts();
1382   const opts = accounts.map((a) => `<option value="${escapeHtml(a)}">${escapeHtml(a)}</option>`).join("");
1383   if ($("#q-account")) $("#q-account").innerHTML = opts;
1384   if ($("#e-account")) $("#e-account").innerHTML = opts;
1385   if ($("#b-account")) $("#b-account").innerHTML = `<option value="">Keep current</option>` + opts;
1386   if ($("#q-transfer-to")) $("#q-transfer-to").innerHTML = opts;
1387   if ($("#e-transfer-to")) $("#e-transfer-to").innerHTML = opts;
1388   if ($("#b-transfer-to")) $("#b-transfer-to").innerHTML = `<option value="">Keep current</option>` + opts;
1389 }
1390
```

```
1391 function selectedType(name) {
1392     return document.querySelector(`input[name="${name}"]:checked`)?.value || "expense";
1393 }
1394
1395 function syncTransferFields() {
1396     const qType = selectedType("q-type");
1397     if ($("#transfer-fields")) $("#transfer-fields").hidden = qType !== "transfer";
1398     const eType = selectedType("e-type");
1399     if ($("#e-transfer-fields")) $("#e-transfer-fields").hidden = eType !== "transfer";
1400     const bType = $("#b-type)?.value;
1401     if ($("#b-transfer-fields")) $("#b-transfer-fields").hidden = bType !== "transfer";
1402     if (qType === "income") $("#q-category").value = "Income";
1403     if (qType === "transfer") $("#q-category").value = "Transfers";
1404 }
1405
1406 function pageSlice(list, page) {
1407     const pages = Math.max(1, Math.ceil(list.length / PAGE_SIZE));
1408     const safe = Math.min(Math.max(0, page), pages - 1);
1409     const start = safe * PAGE_SIZE;
1410     return { page: safe, pages, slice: list.slice(start, start + PAGE_SIZE), total: list.length };
1411 }
1412
1413 function bindPager(prefix, info, onPage) {
1414     const pager = `${prefix}-pager`;
1415     const label = `${prefix}-page-label`;
1416     if (!pager) return;
1417     pager.hidden = info.total <= PAGE_SIZE;
1418     if (label) label.textContent = `Page ${info.page + 1} of ${info.pages}`;
1419     const prev = `${prefix}-prev`;
1420     const next = `${prefix}-next`;
1421     if (prev) prev.disabled = info.page <= 0;
1422     if (next) next.disabled = info.page >= info.pages - 1;
1423     pager._onPage = onPage;
1424 }
1425
1426 function visibleTrackTxns() {
1427     const q = state.trackSearch.trim().toLowerCase();
1428     return [...state.transactions]
1429         .sort((a, b) => (a.date < b.date ? 1 : -1))
1430         .filter((t) => {
1431             if (!q) return true;
1432             return (
1433                 t.merchant.toLowerCase().includes(q) ||
1434                 t.category.toLowerCase().includes(q) ||
1435                 t.account.toLowerCase().includes(q) ||
1436                 (t.note || "").toLowerCase().includes(q)
1437             );
1438         });
1439 }
```

```
1439 }
1440
1441 function pruneSelection() {
1442   const ids = new Set(state.transactions.map((t) => t.id));
1443   state.selected = new Set([...state.selected].filter((id) => ids.has(id)));
1444 }
1445
1446 function selectedList() {
1447   pruneSelection();
1448   return state.transactions.filter((t) => state.selected.has(t.id));
1449 }
1450
1451 function updateBulkBars() {
1452   const selected = selectedList();
1453   const n = selected.length;
1454   const trackVisible = visibleTrackTxs();
1455   const resultsVisible = filtered();
1456
1457   $("bulk-bar").hidden = n === 0;
1458   $("bulk-count").textContent = n === 1 ? "1 selected" : `${n} selected`;
1459   const allTrack = $("select-all");
1460   if (allTrack) {
1461     const page = pageSlice(trackVisible, state.trackPage).slice;
1462     allTrack.checked = page.length > 0 && page.every((t) => state.selected.has(t.id));
1463     allTrack.indeterminate = page.some((t) => state.selected.has(t.id)) && !allTrack.checked;
1464   }
1465
1466   $("results-bulk-bar").hidden = n === 0 || state.demomode;
1467   $("results-bulk-count").textContent = n === 1 ? "1 selected" : `${n} selected`;
1468   const allRes = $("results-select-all");
1469   if (allRes) {
1470     const page = pageSlice(resultsVisible, state.resultsPage).slice;
1471     allRes.checked = page.length > 0 && page.every((t) => state.selected.has(t.id));
1472     allRes.indeterminate = page.some((t) => state.selected.has(t.id)) && !allRes.checked;
1473   }
1474 }
1475
1476 function monthBoundsISO(anchor = todayISO()) {
1477   const d = new Date(`${anchor}T12:00:00`);
1478   const start = new Date(d.getFullYear(), d.getMonth(), 1);
1479   const end = new Date(d.getFullYear(), d.getMonth() + 1, 0, 23, 59, 59, 999);
1480   return { start, end };
1481 }
1482
1483 function renderHomeGlance() {
1484   if (!$("glance-spent")) return;
1485   const { start, end } = monthBoundsISO();
1486   const txs = activeTxs().filter((t) => {
```

```
1487     const d = Core.fromLocalISO ? Core.fromLocalISO(t.date) : new Date(`${t.date}T12:00:00`);
1488     return d >= start && d <= end;
1489   });
1490   const spent = txs.filter((t) => t.type === "expense").reduce((s, t) => s + t.amount, 0);
1491   const income = txs.filter((t) => t.type === "income").reduce((s, t) => s + t.amount, 0);
1492   const net = income - spent;
1493   $("glance-spent").textContent = money(spent);
1494   $("glance-income").textContent = money(income);
1495   if ($("glance-net")) {
1496     $("glance-net").textContent = `${net >= 0 ? "+" : "?"}${money(Math.abs(net))}`;
1497     $("glance-net").classList.toggle("good", net >= 0);
1498     $("glance-net").classList.toggle("bad", net < 0);
1499   }
1500 }
1501
1502 function renderTrack() {
1503   fillAccountSelect();
1504   syncTransferFields();
1505   if (!$("q-date").value) $("q-date").value = todayISO();
1506   pruneSelection();
1507   renderHomeGlance();
1508   const txs = visibleTrackTxs();
1509   const paged = pageSlice(txs, state.trackPage);
1510   state.trackPage = paged.page;
1511   $("track-count").textContent =
1512     state.transactions.length === 1 ? "1 entry" : `${state.transactions.length} entries`;
1513   $("track-empty").hidden = txs.length > 0;
1514   $("track-tx-body").innerHTML = paged.slice
1515     .map((t) => {
1516       const signed = signedDisplay(t);
1517       const on = state.selected.has(t.id);
1518       const extra =
1519         t.type === "transfer" && t.transferTo
1520           ? ` ? to ${escapeHtml(t.transferTo)} `
1521           : "";
1522       return `<tr class="${on ? "selected" : ""}" data-row="${t.id}">
1523         <td class="check-col"><input type="checkbox" data-select="${t.id}" ${on ? "checked" : ""} aria-label="Select ${escapeHtml(t.merchant)}" /></td>
1524         <td>${escapeHtml(t.date)}</td>
1525         <td><strong>${escapeHtml(t.merchant)}</strong><div class="muted small">${escapeHtml(
1526           t.category
1527         )} ? ${escapeHtml(t.account)}${extra}</div></td>
1528         <td class="num ${t.type}">${signed >= 0 ? "+" : "?"}${money(Math.abs(signed))}</td>
1529         <td>
1530           <button type="button" class="icon-btn" data-edit="${t.id}">Edit</button>
1531           <button type="button" class="icon-btn" data-del="${t.id}">Delete</button>
1532         </td>
1533       </tr>`;
1534     })
  }
```

```
1535     .join("");
1536     bindPager("track", paged);
1537     updateBulkBars();
1538     personalizeChrome();
1539 }
1540
1541 /* ----- render results ----- */
1542
1543 function renderAllResults() {
1544     const txs = filtered();
1545     const settings = activeSettings();
1546     const score = txs.length ? healthScore(txs) : null;
1547
1548     $("health-title").textContent = settings.name ? `${settings.name.split(" ")[0]}'s month` : "This period";
1549     if (score == null) {
1550         $("health-score").textContent = "?";
1551         $("health-ring").style.setProperty("--score", "0%");
1552         $("health-sub").textContent = "Add a few spends on Home to see your score.";
1553         $("health-label").textContent = "Score";
1554     } else {
1555         $("health-score").textContent = String(score);
1556         $("health-ring").style.setProperty("--score", `${score}%`);
1557         $("health-label").textContent = score >= 80 ? "Strong" : score >= 60 ? "Fair" : "Needs cut";
1558         $("health-sub").textContent =
1559             score >= 80 ? "Spending looks controlled." : "Check budgets and tips below.";
1560     }
1561
1562     const spent = txs.filter((t) => t.type === "expense").reduce((s, t) => s + t.amount, 0);
1563     const income = txs.filter((t) => t.type === "income").reduce((s, t) => s + t.amount, 0);
1564     const net = income - spent;
1565     const prevSpent = filtered(previousAnchor())
1566         .filter((t) => t.type === "expense")
1567         .reduce((s, t) => s + t.amount, 0);
1568     let deltaLabel = "vs prior period";
1569     let deltaClass = "";
1570     if (prevSpent > 0) {
1571         const pct = Math.round(((spent - prevSpent) / prevSpent) * 100);
1572         deltaLabel = `${pct} >= 0 ? "+" : ""}${pct}% vs prior`;
1573         deltaClass = pct > 0 ? "up" : "down";
1574     }
1575     const days =
1576         state.period === "day" ? 1 : state.period === "year" ? 12 : Math.max(1, new Date(`${state.anchor}T12:00:00`).getDate());
1577     const avg = state.period === "year" ? spent / 12 : spent / days;
1578     const toward = Math.max(0, (income || scaleIncome(settings.monthlyIncome)) - spent);
1579
1580     $("stats-row").innerHTML = `
1581     <article class="stat lift-card"><p class="stat-label">Spent</p><p class="stat-value expense">${money(spent)}</p><p class="stat-hint ${deltaClass}">${deltaLabel}</p></ar
title>
```

```
1582     <article class="stat lift-card"><p class="stat-label">Income</p><p class="stat-value income">${money(income)}</p><p class="stat-hint">this view</p></article>
1583     <article class="stat lift-card"><p class="stat-label">Net</p><p class="stat-value ${net >= 0 ? "income" : "expense"}">${money(net)}</p><p class="stat-hint">income ? exp
enses</p></article>
1584     <article class="stat lift-card"><p class="stat-label">Pace</p><p class="stat-value">${money(avg)}</p><p class="stat-hint">${state.period === "year" ? "avg / month" : "a
vg / day"}</p></article>
1585     <article class="stat lift-card"><p class="stat-label">Toward savings</p><p class="stat-value">${money(toward)}</p><p class="stat-hint">goal ${money(scaleIncome(settings
.savingsGoal || 0))}</p></article>
1586     `;
1587
1588     const buckets = trendBuckets(txs);
1589     const max = Math.max(...buckets.map((b) => b.amount), 1);
1590     $("chart-caption").textContent =
1591       state.period === "year" ? "Monthly totals" : state.period === "day" ? "Selected day" : "Daily totals";
1592     $("trend-chart").innerHTML = buckets
1593       .map((b) => {
1594         const h = Math.max(4, Math.round((b.amount / max) * 150));
1595         return `<div class="bar-col" title="${escapeHtml(b.label)}: ${money(b.amount)}"><div class="bar-3d" style="--h:${h}px"><span class="face front"></span><span class="fa
ce top"></span><span class="face side"></span></div><span class="bar-label">${escapeHtml(b.label)}</span></div>`;
1596       })
1597       .join("");
1598
1599     const cats = categoryTotals(txs).slice(0, 8);
1600     const cmax = cats[0]?.amount || 1;
1601     $("category-bars").innerHTML = cats.length
1602       ? cats
1603         .map(
1604           (c) => `<div class="cat-row"><span class="cat-name">${escapeHtml(c.name)}</span><div class="track"><div class="fill" style="--cat:${CATEGORY_COLORS[c.name] || "#0
f6b5c"};width:${Math.round((c.amount / cmax) * 100)}%></div></div><span class="cat-amt">${money(c.amount)}</span></div>`
1605         )
1606         .join("")
1607       : `<p class="muted">No expenses in this view.</p>`;
1608
1609     const totals = Object.fromEntries(categoryTotals(txs).map((c) => [c.name, c.amount]));
1610     $("budget-list").innerHTML = BUDGETABLE.map((name) => {
1611       const spentCat = totals[name] || 0;
1612       const cap = budgetCap(name);
1613       if (!cap && !spentCat) return "";
1614       const pct = cap ? Math.min(100, Math.round((spentCat / cap) * 100)) : 100;
1615       const over = cap > 0 && spentCat > cap;
1616       return `<div class="budget-row"><div class="budget-meta"><span>${escapeHtml(name)}</span><span class="${over ? "over" : ""}">${money(spentCat)} / ${money(cap)}</span></
div><div class="track"><div class="fill ${over ? "warn" : ""}" style="width:${pct}%></div></div></div>`;
1617     })
1618     .filter(Boolean)
1619     .join("") || `<p class="muted">No budget activity yet.</p>`;
1620
1621     const merchants = {};
1622     for (const t of txs.filter((x) => x.type === "expense")) {
1623       merchants[t.merchant] = (merchants[t.merchant] || 0) + t.amount;
```

```
1624     }
1625     const topM = Object.entries(merchants)
1626       .map(([name, amount]) => ({ name, amount }))
1627       .sort((a, b) => b.amount - a.amount)
1628       .slice(0, 6);
1629     const mmax = topM[0]?.amount || 1;
1630     $("merchant-list").innerHTML = topM.length
1631       ? topM
1632         .map(
1633           (m) => `<div class="merchant-row"><span class="merchant-name">${escapeHtml(m.name)}</span><div class="track"><div class="fill" style="width:${Math.round((m.amount
/ mmax) * 100)}%"></div></div><span class="merchant-amt">${money(m.amount)}</span></div>`
1634         )
1635         .join("")
1636       : `<p class="muted">No merchants yet.</p>`;
1637
1638     $("insights-list").innerHTML = buildInsights(txs)
1639       .map(
1640         (t) => `<li class="insight ${t.kind}"><div class="insight-icon">${t.icon}</div><div><h3>${escapeHtml(t.title)}</h3><p>${escapeHtml(t.body)}</p></div></li>`
1641       )
1642       .join("");
1643
1644     const bal = Core.applyLedger
1645       ? Core.applyLedger(activeAccounts(), state.demomode ? {} : state.openingBalances, activeTxs())
1646       : (() => {
1647         const map = {};
1648         for (const t of activeTxs()) {
1649           map[t.account] = (map[t.account] || 0) + (t.type === "income" ? t.amount : -t.amount);
1650         }
1651         return map;
1652       })();
1653     const accounts = Object.entries(bal);
1654     $("accounts-list").innerHTML = accounts.length
1655       ? accounts
1656         .map(
1657           ([name, balance]) => `<div class="account"><div><strong>${escapeHtml(name)}</strong><span>Running balance</span></div><div class="bal">${money(balance)}</div></div>`
1658         )
1659         .join("")
1660       : `<p class="muted">No account activity yet.</p>`;
1661
1662     const recurring = detectRecurring(activeTxs());
1663     $("recurring-list").innerHTML = recurring.length
1664       ? recurring
1665         .map(
1666           (r) => `<div class="recurring-item"><div><strong>${escapeHtml(r.merchant)}</strong><span>${escapeHtml(r.category)} ? ${r.count}</span></div><div class="bal">~${m
oney(r.amount)}</div></div>`
1667         )
1668         .join("")
```

```
1669      : `
```

```
1717   if (!recurring.length) return `
```

```
1765     return digestPin(`pm-pin:${pin}`);
1766 }
1767
1768 async function hashPin(pin, mobile) {
1769     const salt = normalizeMobile(mobile || storedMobile()) || "niva";
1770     return digestPin(`pm-pin-v2:${salt}:${pin}`);
1771 }
1772
1773 async function verifyPin(pin, mobile) {
1774     const stored = state.settings.pinHash;
1775     if (!stored) return false;
1776     const salted = await hashPin(pin, mobile);
1777     if (salted === stored) return true;
1778     const legacy = await hashPinLegacy(pin);
1779     if (legacy === stored) {
1780         state.settings.pinHash = salted;
1781         save();
1782         return true;
1783     }
1784     return false;
1785 }
1786
1787 function syncAccountSecurityUI() {
1788     if ($("#profile-mobile-masked")) $("#profile-mobile-masked").textContent = maskMobile(storedMobile());
1789     if ($("#sensitive-wrap")) $("#sensitive-wrap").hidden = false;
1790     if ($("#pin-set-row")) $("#pin-set-row").hidden = false;
1791     if ($("#pin-copy")) {
1792         $("#pin-copy").textContent =
1793             "Mobile + PIN unlock this profile. Closing the tab locks it again. Data stays in this browser.";
1794     }
1795     applyPhoto(state.demoMode ? "" : state.settings.photo);
1796 }
1797
1798 /* ----- CSV ----- */
1799
1800 function csvColSelects() {
1801     return {
1802         date: $("csv-col-date"),
1803         description: $("csv-col-description"),
1804         amount: $("csv-col-amount"),
1805         type: $("csv-col-type"),
1806         debit: $("csv-col-debit"),
1807         credit: $("csv-col-credit"),
1808         account: $("csv-col-account"),
1809         category: $("csv-col-category"),
1810     };
1811 }
1812
```

```
1813 function fillCsvColumnSelects(headers, idx) {
1814   const none = `<option value="-1">(none)</option>`;
1815   const opts = headers.map((h, i) => `<option value="${i}">${escapeHtml(h || `Column ${i + 1}`)}</option>`).join("");
1816   const selects = csvColSelects();
1817   for (const [key, el] of Object.entries(selects)) {
1818     if (!el) continue;
1819     el.innerHTML = (key === "date" ? "" : none) + opts;
1820     const value = idx[key] ?? -1;
1821     el.value = String(value >= 0 ? value : -1);
1822   }
1823 }
1824
1825 function mappingFromSelects() {
1826   const idx = {};
1827   for (const [key, el] of Object.entries(csvColSelects())) {
1828     idx[key] = el ? Number(el.value) : -1;
1829   }
1830   return idx;
1831 }
1832
1833 function parseCsvWithOptions(text, mapping) {
1834   if (!Core.parseCsvRows) throw new Error("CSV helper missing.");
1835   return Core.parseCsvRows(text, {
1836     dayFirst: $("csv-day-first") ? $("csv-day-first").checked : true,
1837     defaultAccount: state.accounts[0] || "Imported bank",
1838     guessCategory,
1839     categories: CATEGORIES,
1840     uid,
1841     mapping,
1842   });
1843 }
1844
1845 function existingFingerprints() {
1846   return new Set(state.transactions.map((t) => t.fingerprint || txFingerprint(t)));
1847 }
1848
1849 function refreshCsvPreview() {
1850   const status = $("csv-status");
1851   const body = $("csv-preview-body");
1852   const meta = $("csv-preview-meta");
1853   if (!pendingCsvText) return;
1854   try {
1855     const parsed = parseCsvWithOptions(pendingCsvText, mappingFromSelects());
1856     const seen = existingFingerprints();
1857     const skip = $("csv-skip-dupes")?.checked !== false;
1858     let dupes = 0;
1859     const unique = [];
1860     for (const row of parsed.rows) {
```

```
1861     const fp = row.fingerprint || txFingerprint(row);
1862     if (skip && seen.has(fp)) dupes += 1;
1863     else unique.push(row);
1864   }
1865   if (body) {
1866     body.innerHTML = unique.slice(0, 8)
1867       .map(
1868         (t) =>
1869           `<tr><td>${escapeHtml(t.date)}</td><td><strong>${escapeHtml(t.merchant)}</strong><div class="muted small">${escapeHtml(t.account)} ? ${escapeHtml(t.type)}</div>
1870           </td><td class="num ${t.type}">${t.type === "income" ? "+" : "?"}${money(t.amount)}</td></tr>`
1871       )
1872       .join("") || `<tr><td colspan="3">No new rows with this mapping.</td></tr>`;
1873   }
1874   if (meta) {
1875     meta.textContent = `${parsed.rows.length} valid rows ? ${unique.length} new${dupes ? ` ? ${dupes} duplicates skipped` : ""}`;
1876   }
1877   if (status) status.textContent = "";
1878   return { parsed, unique, dupes };
1879 } catch (err) {
1880   if (body) body.innerHTML = "";
1881   if (status) status.textContent = err.message || "Could not parse this file.";
1882   return null;
1883 }
1884
1885 async function openCsvDialog(text) {
1886   pendingCsvText = text;
1887   const detected = Core.detectCsvMapping
1888     ? Core.detectCsvMapping(text.split(/\r?\n/).find((l) => l.trim()) || "")
1889     : { headers: [], idx: {} };
1890   fillCsvColumnSelects(detected.headers, detected.idx);
1891   refreshCsvPreview();
1892   $("csv-dialog").showModal();
1893 }
1894
1895 function download(filename, content, type) {
1896   const blob = new Blob([content], { type });
1897   const url = URL.createObjectURL(blob);
1898   const a = document.createElement("a");
1899   a.href = url;
1900   a.download = filename;
1901   a.click();
1902   URL.revokeObjectURL(url);
1903 }
1904
1905 function guardLive() {
1906   if (state.demoMode) {
1907     alert("Exit demo first ? demo data cannot be edited.");
```

```
1908     return false;
1909   }
1910   return true;
1911 }
1912
1913 function openSingleEdit(id) {
1914   if (!guardLive()) return;
1915   const tx = state.transactions.find((t) => t.id === id);
1916   if (!tx) return;
1917   fillAccountSelect();
1918   $("#edit-dialog-title").textContent = "Edit transaction";
1919   $("#edit-dialog-hint").textContent = "Change any field, then save.";
1920   $("#e-id").value = tx.id;
1921   $("#e-date").value = tx.date;
1922   $("#e-merchant").value = tx.merchant;
1923   $("#e-amount").value = tx.amount;
1924   $("#e-category").value = tx.category;
1925   $("#e-account").value = tx.account;
1926   $("#e-note").value = tx.note || "";
1927   const typeRadio = document.querySelector(`input[name="e-type"][value="${tx.type}"]`) || document.querySelector('input[name="e-type"][value="expense"]');
1928   if (typeRadio) typeRadio.checked = true;
1929   if ($("#e-transfer-to") && tx.transferTo) $("#e-transfer-to").value = tx.transferTo;
1930   syncTransferFields();
1931   $("#edit-dialog").showModal();
1932 }
1933
1934 function openBulkEdit() {
1935   if (!guardLive()) return;
1936   const n = selectedList().length;
1937   if (!n) {
1938     alert("Tick one or more transactions first.");
1939     return;
1940   }
1941   fillAccountSelect();
1942   $("#bulk-edit-hint").textContent = `Apply to ${n} selected payment${n === 1 ? "" : "s"}. Leave a field blank to keep it.`;
1943   $("#b-date").value = "";
1944   $("#b-type").value = "";
1945   $("#b-category").value = "";
1946   $("#b-account").value = "";
1947   $("#b-note").value = "";
1948   $("#b-note-apply").checked = false;
1949   syncTransferFields();
1950   $("#bulk-edit-dialog").showModal();
1951 }
1952
1953 function deleteIds(ids) {
1954   if (!guardLive()) return;
1955   if (!ids.length) return;
```

```
1956     const drop = new Set(ids);
1957     const removed = state.transactions.filter((t) => drop.has(t.id));
1958     state.transactions = state.transactions.filter((t) => !drop.has(t.id));
1959     ids.forEach((id) => state.selected.delete(id));
1960     undoPayload = removed;
1961     save();
1962     renderTrack();
1963     if (state.view === "results" || state.view === "demo") renderAllResults();
1964     const label = removed.length === 1 ? "1 payment deleted" : `${removed.length} payments deleted`;
1965     showToast(label, "Undo", () => {
1966         if (!undoPayload?.length) return;
1967         state.transactions = state.transactions.concat(undoPayload);
1968         undoPayload = null;
1969         save();
1970         renderTrack();
1971         if (state.view === "results" || state.view === "demo") renderAllResults();
1972         showToast("Delete undone");
1973     });
1974 }
1975
1976 function toggleSelect(id, checked) {
1977     if (checked) state.selected.add(id);
1978     else state.selected.delete(id);
1979     updateBulkBars();
1980     document.querySelectorAll(`tr[data-row="${id}"]`).forEach((row) => {
1981         row.classList.toggle("selected", checked);
1982         const box = row.querySelector("[data-select]");
1983         if (box) box.checked = checked;
1984     });
1985 }
1986
1987 function handleTableClick(e) {
1988     const editId = e.target.getAttribute("data-edit");
1989     const delId = e.target.getAttribute("data-del");
1990     if (editId) {
1991         e.stopPropagation();
1992         openSingleEdit(editId);
1993         return;
1994     }
1995     if (delId) {
1996         e.stopPropagation();
1997         deleteIds([delId]);
1998         return;
1999     }
2000     if (e.target.matches("[data-select]")) {
2001         toggleSelect(e.target.getAttribute("data-select"), e.target.checked);
2002         return;
2003     }
2004 }
```

```
2004     const row = e.target.closest("tr[data-row]");
2005     if (!row) return;
2006     const id = row.getAttribute("data-row");
2007     toggleSelect(id, !state.selected.has(id));
2008 }
2009
2010 function toCsv(txs) {
2011     const header = "Date,Description,Amount,Type,Category,Account,TransferTo,Note";
2012     const rows = txs.map((t) =>
2013         [t.date, t.merchant, t.amount, t.type, t.category, t.account, t.transferTo || "", t.note || ""]
2014         .map((v) => `_${String(v).replace(/"/g, '"')}_`)
2015         .join(",")
2016     );
2017     return [header, ...rows].join("\n");
2018 }
2019
2020 /* ----- bind ----- */
2021
2022 function on(id, evt, fn) {
2023     const el = $(id);
2024     if (!el) return;
2025     el.addEventListener(evt, fn);
2026 }
2027
2028 function bind() {
2029     on("btn-start-setup", "click", (e) => {
2030         e.preventDefault();
2031         startSetup();
2032     });
2033     on("btn-open-demo", "click", (e) => {
2034         e.preventDefault();
2035         openDemo();
2036     });
2037     on("btn-goto-login", "click", (e) => {
2038         e.preventDefault();
2039         showLogin();
2040     });
2041     on("btn-create-profile", "click", (e) => {
2042         e.preventDefault();
2043         createNewProfile();
2044     });
2045     on("login-form", "submit", (e) => {
2046         e.preventDefault();
2047         submitLogin(e);
2048     });
2049     on("btn-forgot-reset", "click", () => {
2050         const panel = $("login-reset-panel");
2051         if (!panel) return;
```

```
2052     panel.hidden = !panel.hidden;
2053   });
2054   on("btn-confirm-device-reset", "click", () => {
2055     confirmDeviceReset();
2056   });
2057
2058   try {
2059     on("btn-setup-back", "click", () => {
2060       if (state.setupStep > 1) {
2061         if (state.setupStep === SETUP_LAST_STEP) {
2062           state.setupBudgetsDraft = readBudgets("setup-budgets");
2063         }
2064         state.setupStep -= 1;
2065         updateSetupUI();
2066       }
2067     });
2068
2069     on("btn-setup-next", "click", async () => {
2070       if (!validateSetupStep()) return;
2071       if (state.setupStep < SETUP_LAST_STEP) {
2072         if (state.setupStep === 1) {
2073           state.settings.name = $("setup-name").value.trim() || "";
2074           state.settings.currency = $("setup-currency").value || "INR";
2075         }
2076         if (state.setupStep === 2) {
2077           syncSettingsMobile($("setup-mobile").value || "");
2078           const pin = $("setup-pin").value || "";
2079           if (pin) {
2080             state.settings.pinHash = await hashPin(pin, $("setup-mobile").value || "");
2081             if ($("setup-pin")) $("setup-pin").value = "";
2082             if ($("setup-pin-confirm")) $("setup-pin-confirm").value = "";
2083           }
2084         }
2085         if (state.setupStep === 3) {
2086           state.settings.monthlyIncome = parseAmount($("setup-income").value) || 0;
2087           state.settings.savingsGoal = parseAmount($("setup-savings").value) || 0;
2088         }
2089         if (state.setupStep === 4) {
2090           state.accounts = readAccountFields("setup-accounts");
2091           state.openingBalances = readOpeningBalances("setup-accounts");
2092         }
2093         state.setupStep += 1;
2094         if (state.setupStep === SETUP_LAST_STEP) {
2095           if (!state.setupBudgetsDraft) {
2096             state.setupBudgetsDraft = defaultBudgetsFor($("setup-currency").value || state.settings.currency);
2097           }
2098           renderBudgetFields("setup-budgets", state.setupBudgetsDraft);
2099         }
2100       }
2101     });
2102   }
2103 }
```

```
2100     updateSetupUI();
2101   } else {
2102     state.budgets = readBudgets("setup-budgets");
2103     await finishSetup();
2104   }
2105 });
2106
2107 on("btn-add-account-field", "click", () => {
2108   const current = snapshotAccountRows("setup-accounts");
2109   current.push({ name: "", opening: "" });
2110   renderAccountFields("setup-accounts", current);
2111 });
2112
2113 on("setup-accounts", "click", (e) => {
2114   if (!e.target.matches("[data-remove-account]")) return;
2115   const current = snapshotAccountRows("setup-accounts");
2116   const row = e.target.closest(".account-row");
2117   const inputs = [...$("setup-accounts").querySelectorAll(".account-row")];
2118   const idx = inputs.indexOf(row);
2119   if (idx >= 0 && current.length > 1) {
2120     current.splice(idx, 1);
2121     renderAccountFields("setup-accounts", current);
2122   }
2123 });
2124
2125 document.querySelectorAll(".main-tabs .tab").forEach((tab) => {
2126   tab.addEventListener("click", () => {
2127     if (state.setupComplete && !isSessionUnlocked() && tab.dataset.view !== "demo") {
2128       showLogin();
2129       return;
2130     }
2131     if (!state.setupComplete && tab.dataset.view !== "demo") {
2132       if (confirm("Finish the short setup first? It takes about a minute.")) {
2133         startSetup();
2134         return;
2135       }
2136     }
2137     setView(tab.dataset.view);
2138   });
2139 });
2140
2141 on("btn-exit-demo", "click", () => {
2142   if (!state.setupComplete) {
2143     showScreen("welcome");
2144     return;
2145   }
2146   if (!isSessionUnlocked()) {
2147     showLogin();
```

```
2148         return;
2149     }
2150     setView("track");
2151 });
2152
2153 on("btn-demo-to-setup", "click", startSetup);
2154 $("btn-goto-demo-from-profile").addEventListener("click", () => setView("demo"));
2155 on("btn-goto-track", "click", () => setView("track"));
2156 $("btn-jump-insights").addEventListener("click", () => setView("results"));
2157
2158 fillCategorySelect();
2159 $("q-date").value = todayISO();
2160
2161 document.querySelectorAll('input[name="q-type"], input[name="e-type"]').forEach((r) => {
2162     r.addEventListener("change", syncTransferFields);
2163 });
2164 $("b-type").addEventListener("change", syncTransferFields);
2165
2166 on("q-merchant", "change", () => {
2167     const type = selectedType("q-type");
2168     $("q-category").value = guessCategory($("q-merchant").value, type);
2169 });
2170
2171 on("quick-form", "submit", (e) => {
2172     e.preventDefault();
2173     if (state.demoMode) {
2174         alert("Exit demo first ? demo cannot save into your real data.");
2175         return;
2176     }
2177     const type = selectedType("q-type");
2178     let category = $("q-category").value;
2179     if (type === "income") category = "Income";
2180     if (type === "transfer") category = "Transfers";
2181     const account = $("q-account").value;
2182     const transferTo = type === "transfer" ? $("q-transfer-to").value : "";
2183     if (type === "transfer" && (!transferTo || transferTo === account)) {
2184         alert("Pick a different account to send money to.");
2185         return;
2186     }
2187     const payload = {
2188         id: uid(),
2189         date: $("q-date").value || todayISO(),
2190         merchant: $("q-merchant").value.trim() || (type === "transfer" ? `To ${transferTo}` : ""),
2191         amount: parseAmount($("q-amount").value),
2192         type,
2193         category,
2194         account,
2195         transferTo: transferTo || "",
```

```
2196     note: $("q-note").value.trim(),
2197   };
2198   payload.fingerprint = txFingerprint(payload);
2199   if (!payload.merchant || !payload.amount || !payload.date) {
2200     alert("Please fill date, where/who, and amount.");
2201     return;
2202   }
2203   state.transactions.push(payload);
2204   if (!state.accounts.includes(account)) state.accounts.push(account);
2205   save();
2206   $("quick-form").reset();
2207   document.querySelector('input[name="q-type"][value="expense"]').checked = true;
2208   $("q-date").value = todayISO();
2209   fillAccountSelect();
2210   syncTransferFields();
2211   $("q-account").value = account;
2212   $("form-status").textContent = "Saved on this device. Add another, or tap See insights.";
2213   renderTrack();
2214   });
2215
2216   on("track-tx-body", "click", handleTableClick);
2217   on("results-tx-body", "click", handleTableClick);
2218
2219   on("select-all", "change", (e) => {
2220     pageSlice(visibleTrackTxs(), state.trackPage).slice.forEach((t) => {
2221       if (e.target.checked) state.selected.add(t.id);
2222       else state.selected.delete(t.id);
2223     });
2224     renderTrack();
2225   });
2226
2227   on("results-select-all", "change", (e) => {
2228     if (!guardLive()) {
2229       e.target.checked = false;
2230       return;
2231     }
2232     pageSlice(filtered(), state.resultsPage).slice.forEach((t) => {
2233       if (e.target.checked) state.selected.add(t.id);
2234       else state.selected.delete(t.id);
2235     });
2236     renderAllResults();
2237   });
2238
2239   on("track-search", "input", () => {
2240     state.trackSearch = $("track-search").value;
2241     state.trackPage = 0;
2242     renderTrack();
2243   });
```

```
2244
2245   $("track-prev")?.addEventListener("click", () => {
2246     state.trackPage = Math.max(0, state.trackPage - 1);
2247     renderTrack();
2248   });
2249   $("track-next")?.addEventListener("click", () => {
2250     state.trackPage += 1;
2251     renderTrack();
2252   });
2253   $("results-prev")?.addEventListener("click", () => {
2254     state.resultsPage = Math.max(0, state.resultsPage - 1);
2255     renderAllResults();
2256   });
2257   $("results-next")?.addEventListener("click", () => {
2258     state.resultsPage += 1;
2259     renderAllResults();
2260   });
2261
2262   const clearTicks = () => {
2263     state.selected.clear();
2264     renderTrack();
2265     if (!$("view-results").hidden) renderAllResults();
2266   };
2267
2268   on("btn-bulk-clear", "click", clearTicks);
2269   on("btn-results-bulk-clear", "click", clearTicks);
2270   on("btn-bulk-edit", "click", openBulkEdit);
2271   on("btn-results-bulk-edit", "click", openBulkEdit);
2272   on("btn-bulk-delete", "click", () => deleteIds(selectedList().map((t) => t.id)));
2273   on("btn-results-bulk-delete", "click", () => deleteIds(selectedList().map((t) => t.id)));
2274
2275   on("btn-edit-cancel", "click", () => $("edit-dialog").close());
2276   on("btn-bulk-edit-cancel", "click", () => $("bulk-edit-dialog").close());
2277
2278   on("edit-form", "submit", (e) => {
2279     e.preventDefault();
2280     if (!guardLive()) return;
2281     const id = $("e-id").value;
2282     const tx = state.transactions.find((t) => t.id === id);
2283     if (!tx) return;
2284     const type = selectedType("e-type");
2285     tx.date = $("e-date").value;
2286     tx.merchant = $("e-merchant").value.trim();
2287     tx.amount = parseAmount($("e-amount").value);
2288     tx.type = type;
2289     tx.category = type === "income" ? "Income" : type === "transfer" ? "Transfers" : $("e-category").value;
2290     tx.account = $("e-account").value;
2291     tx.transferTo = type === "transfer" ? $("e-transfer-to")?.value || "" : "";
```

```
2292     tx.note = $("e-note").value.trim();
2293     tx.fingerprint = txFingerprint(tx);
2294     if (type === "transfer" && (!tx.transferTo || tx.transferTo === tx.account)) {
2295         alert("Pick a different account to send money to.");
2296         return;
2297     }
2298     if (!state.accounts.includes(tx.account)) state.accounts.push(tx.account);
2299     save();
2300     $("edit-dialog").close();
2301     $("form-status").textContent = "Transaction updated.";
2302     renderTrack();
2303     if (!$("view-results").hidden) renderAllResults();
2304 });
2305
2306 on("bulk-edit-form", "submit", (e) => {
2307     e.preventDefault();
2308     if (!guardLive()) return;
2309     const list = selectedList();
2310     if (!list.length) return;
2311     const date = $("b-date").value;
2312     const type = $("b-type").value;
2313     const category = $("b-category").value;
2314     const account = $("b-account").value;
2315     const note = $("b-note").value;
2316     const applyNote = $("b-note-apply").checked;
2317     const transferTo = $("b-transfer-to")?.value;
2318     if (!date && !type && !category && !account && !applyNote && !(type === "transfer" && transferTo)) {
2319         alert("Fill at least one field to change.");
2320         return;
2321     }
2322     for (const tx of list) {
2323         if (date) tx.date = date;
2324         if (type) {
2325             tx.type = type;
2326             if (type === "income") tx.category = "Income";
2327             if (type === "transfer") tx.category = "Transfers";
2328         }
2329         if (category) tx.category = category;
2330         if (account) {
2331             tx.account = account;
2332             if (!state.accounts.includes(account)) state.accounts.push(account);
2333         }
2334         if (type === "transfer" && transferTo) tx.transferTo = transferTo;
2335         if (applyNote) tx.note = note.trim();
2336         tx.fingerprint = txFingerprint(tx);
2337     }
2338     save();
2339     $("bulk-edit-dialog").close();
```

```
2340     $("form-status").textContent = `Updated ${list.length} payment${list.length === 1 ? "" : "s"}`;
2341     renderTrack();
2342     if (!$("view-results").hidden) renderAllResults();
2343   });
2344
2345   document.querySelectorAll(".seg").forEach((btn) => {
2346     btn.addEventListener("click", () => {
2347       document.querySelectorAll(".seg").forEach((b) => b.classList.remove("active"));
2348       btn.classList.add("active");
2349       state.period = btn.dataset.period;
2350       state.resultsPage = 0;
2351       renderAllResults();
2352     });
2353   });
2354
2355   $("anchor-date").value = state.anchor;
2356   on("anchor-date", "change", () => {
2357     state.anchor = $("anchor-date").value || todayISO();
2358     renderAllResults();
2359   });
2360   on("btn-prev", "click", () => {
2361     shiftAnchor(-1);
2362     renderAllResults();
2363   });
2364   on("btn-next", "click", () => {
2365     shiftAnchor(1);
2366     renderAllResults();
2367   });
2368   on("btn-today", "click", () => {
2369     state.anchor = todayISO();
2370     $("anchor-date").value = state.anchor;
2371     renderAllResults();
2372   });
2373   on("account-filter", "change", () => {
2374     state.account = $("account-filter").value;
2375     state.resultsPage = 0;
2376     renderAllResults();
2377   });
2378   on("search", "input", () => {
2379     state.search = $("search").value;
2380     state.resultsPage = 0;
2381     renderAllResults();
2382   });
2383   on("btn-print", "click", () => window.print());
2384
2385   on("csv-input", "change", async () => {
2386     const file = $("csv-input").files?.[0];
2387     $("csv-input").value = "";
```

```
2388     if (!file) return;
2389     try {
2390       await openCsvDialog(await file.text());
2391     } catch (err) {
2392       alert(err.message || "Import failed");
2393     }
2394   });
2395   ["csv-col-date", "csv-col-description", "csv-col-amount", "csv-col-type", "csv-col-debit", "csv-col-credit", "csv-col-account", "csv-col-category", "csv-day-first", "csv-
skip-dupes"].forEach((id) => {
2396     $(id)?.addEventListener("change", refreshCsvPreview);
2397   });
2398   $("btn-csv-cancel").addEventListener("click", () => {
2399     pendingCsvText = "";
2400     $("csv-dialog").close();
2401   });
2402   $("btn-csv-confirm").addEventListener("click", () => {
2403     const preview = refreshCsvPreview();
2404     if (!preview?.unique?.length) {
2405       alert("Nothing new to import with this mapping.");
2406       return;
2407     }
2408     for (const row of preview.unique) {
2409       if (!state.accounts.includes(row.account)) state.accounts.push(row.account);
2410     }
2411     state.transactions = state.transactions.concat(preview.unique);
2412     save();
2413     pendingCsvText = "";
2414     $("csv-dialog").close();
2415     renderTrack();
2416     $("form-status").textContent = `Imported ${preview.unique.length} rows${preview.dupes ? ` (${preview.dupes} duplicates skipped)` : ""}.`;
2417     showToast(`Imported ${preview.unique.length} payments`);
2418   });
2419
2420   on("btn-export", "click", () => {
2421     if (state.demomode) {
2422       alert("Exit demo first ? demo data is not your real list.");
2423       return;
2424     }
2425     $("export-dialog").showModal();
2426   });
2427   on("btn-export-close", "click", () => $("export-dialog").close());
2428   on("btn-save-local", "click", () => {
2429     saveToBrowser();
2430     $("save-dialog").showModal();
2431   });
2432   on("btn-save-dialog-close", "click", () => $("save-dialog").close());
2433   on("btn-save-json-file", "click", () => saveToFile("json"));
2434   on("btn-save-csv-file", "click", () => saveToFile("csv"));
```

```
2435     on("btn-save-file", "click", () => saveToFile("json"));
2436     on("btn-export-csv", "click", () => {
2437       if (state.demoMode) {
2438         alert("Exit demo first ? demo data is not your real list.");
2439         return;
2440       }
2441       download(`niva-${state.anchor}.csv`, toCsv(filtered()), "text/csv");
2442       $("export-dialog").close();
2443     });
2444     on("btn-export-all-csv", "click", () => {
2445       saveToFile("csv");
2446       $("export-dialog").close();
2447     });
2448     on("btn-export-json", "click", () => {
2449       saveToFile("json");
2450       $("export-dialog").close();
2451     });
2452
2453     on("btn-edit-add-account", "click", () => {
2454       const current = snapshotAccountRows("edit-accounts");
2455       current.push({ name: "", opening: "" });
2456       renderAccountFields("edit-accounts", current);
2457     });
2458     on("edit-accounts", "click", (e) => {
2459       if (!e.target.matches("[data-remove-account]")) return;
2460       const rows = [...$("edit-accounts").querySelectorAll(".account-row")];
2461       const vals = snapshotAccountRows("edit-accounts");
2462       const idx = rows.indexOf(e.target.closest(".account-row"));
2463       if (idx >= 0 && vals.length > 1) {
2464         vals.splice(idx, 1);
2465         renderAccountFields("edit-accounts", vals);
2466       }
2467     });
2468
2469     on("photo-input", "change", async () => {
2470       const file = $("photo-input").files?.[0];
2471       $("photo-input").value = "";
2472       if (!file) return;
2473       try {
2474         state.settings.photo = await resizePhoto(file);
2475         applyPhoto(state.settings.photo);
2476         save();
2477         $("edit-status").textContent = "Photo saved on this device.";
2478         personalizeChrome();
2479       } catch (err) {
2480         alert(err.message || "Could not upload photo.");
2481       }
2482     });
```

```
2483
2484   on("btn-remove-photo", "click", () => {
2485     state.settings.photo = "";
2486     applyPhoto("");
2487     save();
2488     $("edit-status").textContent = "Photo removed.";
2489     personalizeChrome();
2490   });
2491
2492   $("setup-currency")?.addEventListener("change", () => {
2493     if (state.setupStep === SETUP_LAST_STEP) {
2494       state.setupBudgetsDraft = defaultBudgetsFor($("setup-currency").value);
2495       renderBudgetFields("setup-budgets", state.setupBudgetsDraft);
2496     }
2497   });
2498
2499   $("btn-change-pin")?.addEventListener("click", async () => {
2500     const status = $("pin-status");
2501     const current = $("pin-current")?.value || "";
2502     const a = $("pin-new")?.value || "";
2503     const b = $("pin-confirm")?.value || "";
2504     if (!state.settings.pinHash) {
2505       if (status) status.textContent = "No PIN is set on this profile.";
2506       return;
2507     }
2508     if (!(await verifyPin(current, storedMobile()))) {
2509       if (status) status.textContent = "Current PIN is wrong.";
2510       return;
2511     }
2512     if (!/^\d{4,8}$/.test(a)) {
2513       if (status) status.textContent = "Use 4 to 8 digits.";
2514       return;
2515     }
2516     if (a !== b) {
2517       if (status) status.textContent = "New PINs do not match.";
2518       return;
2519     }
2520     state.settings.pinHash = await hashPin(a, storedMobile());
2521     if ($("pin-current")) $("pin-current").value = "";
2522     if ($("pin-new")) $("pin-new").value = "";
2523     if ($("pin-confirm")) $("pin-confirm").value = "";
2524     save();
2525     if (status) status.textContent = "PIN updated on this device.";
2526   });
2527
2528   $("btn-logout")?.addEventListener("click", (e) => {
2529     e.preventDefault();
2530     logoutSession();
2531   });
```

```
2531   });
2532
2533   on("btn-save-details", "click", () => {
2534     const accounts = readAccountFields("edit-accounts");
2535     if (!accounts.length) {
2536       alert("Keep at least one account.");
2537       return;
2538     }
2539     const phoneRaw = $("edit-phone")?.value.trim() || "";
2540     if (!isValidMobile(phoneRaw)) {
2541       alert("Keep a valid 10-digit mobile number (or +91) ? it is used to log in.");
2542       return;
2543     }
2544     syncSettingsMobile(phoneRaw);
2545     state.settings = {
2546       ...state.settings,
2547       name: $("edit-name")?.value.trim() || "",
2548       phone: state.settings.phone,
2549       mobile: state.settings.mobile,
2550       email: $("edit-email")?.value.trim() || "",
2551       city: $("edit-city")?.value.trim() || "",
2552       occupation: $("edit-occupation")?.value.trim() || "",
2553       dob: $("edit-dob")?.value || "",
2554       gender: $("edit-gender")?.value || "",
2555       notes: $("edit-notes")?.value.trim() || "",
2556       currency: $("edit-currency")?.value || "INR",
2557       monthlyIncome: parseAmount($("edit-income")?.value) || 0,
2558       savingsGoal: parseAmount($("edit-savings")?.value) || 0,
2559     };
2560     state.accounts = accounts;
2561     state.openingBalances = readOpeningBalances("edit-accounts");
2562     state.budgets = readBudgets("edit-budgets");
2563     state.setupComplete = true;
2564     save();
2565     if ($("edit-status")) $("edit-status").textContent = "Profile saved.";
2566     syncAccountSecurityUI();
2567     personalizeChrome();
2568   });
2569
2570   on("btn-reset-all", "click", async () => {
2571     if (!confirm("Clear all data and return to welcome screen?")) return;
2572     await wipeAllData();
2573     showScreen("welcome");
2574   });
2575
2576   on("backup-input", "change", async () => {
2577     const file = $("backup-input").files?.[0];
2578     if ($("backup-input")) $("backup-input").value = "";
```

```
2579     if (!file) return;
2580     try {
2581         const data = JSON.parse(await file.text());
2582         if (!Array.isArray(data.transactions)) throw new Error("Invalid backup.");
2583         if (!confirm(`Restore ${data.transactions.length} payments?`)) return;
2584         applyLoaded({ ...data, setupComplete: true });
2585         save();
2586         setSessionUnlocked(false);
2587         showLogin("Backup restored. Log in with that profile's mobile and PIN.");
2588     } catch (err) {
2589         alert(err.message || "Restore failed");
2590     }
2591 });
2592 } catch (err) {
2593     console.error("bindApp failed", err);
2594 }
2595 }
2596
2597 window.startSetup = startSetup;
2598 window.openDemo = openDemo;
2599 window.submitLogin = submitLogin;
2600 window.showLogin = showLogin;
2601 window.createNewProfile = createNewProfile;
2602 window.logoutSession = logoutSession;
2603
2604 try {
2605     loadFromLocalStorage();
2606     updateSaveStatus();
2607     bind();
2608     boot();
2609 } catch (err) {
2610     console.error(err);
2611     showScreen("welcome");
2612 }
2613
2614 loadFromIdb().then((updated) => {
2615     if (!updated) return;
2616     updateSaveStatus();
2617     if (state.demoMode) return;
2618     if (state.setupComplete && !isSessionUnlocked()) {
2619         const appOpen = $("screen-app") && !$("screen-app").hidden;
2620         const welcomeOpen = $("screen-welcome") && !$("screen-welcome").hidden;
2621         if (appOpen || welcomeOpen) showLogin();
2622         return;
2623     }
2624     if (state.setupComplete && $("screen-app") && !$("screen-app").hidden) {
2625         personalizeChrome();
2626         if (state.view === "results") renderAllResults();
```

```
2627     else renderTrack();
2628   }
2629 });
2630
2631 if (location.protocol !== "file:" && "serviceWorker" in navigator) {
2632   navigator.serviceWorker.register("sw.js").catch(() => {});
2633 }
2634 }());
```

core.js

```
1  /* Niva ? shared date, money, and CSV helpers (also used by tests). */
2  (function (root) {
3      function pad(n) {
4          return String(n).padStart(2, "0");
5      }
6
7      function toLocalISO(d) {
8          const dt = d instanceof Date ? d : new Date();
9          return `${dt.getFullYear()}-${pad(dt.getMonth() + 1)}-${pad(dt.getDate())}`;
10     }
11
12     function fromLocalISO(iso) {
13         const m = String(iso || "").match(/^(\\d{4})-(\\d{2})-(\\d{2})$/);
14         if (!m) return new Date();
15         return new Date(Number(m[1]), Number(m[2]) - 1, Number(m[3]), 12, 0, 0);
16     }
17
18     function shiftLocalISO(iso, period, delta) {
19         const d = fromLocalISO(iso);
20         if (period === "day") d.setDate(d.getDate() + delta);
21         else if (period === "year") d.setFullYear(d.getFullYear() + delta);
22         else d.setMonth(d.getMonth() + delta);
23         return toLocalISO(d);
24     }
25
26     function roundMoney(n) {
27         const x = Number(n);
28         if (!Number.isFinite(x)) return 0;
29         return Math.round(x * 100) / 100;
30     }
31
32     function parseAmount(v) {
33         if (typeof v === "number") return roundMoney(v);
34         const cleaned = String(v ?? "").replace(/^[0-9.-]/g, "");
35         const n = Number(cleaned);
36         return Number.isFinite(n) ? roundMoney(n) : 0;
37     }
38
39     function fingerprint(row) {
40         const parts = [
41             row.date || "",
42             row.type || "",
43             roundMoney(row.amount).toFixed(2),
44             String(row.merchant || "")
45                 .trim()
46                 .toLowerCase(),
```

```
47     String(row.account || "")
48     .trim()
49     .toLowerCase(),
50 ];
51 if (row.type === "transfer") {
52     parts.push(
53         String(row.transferTo || "")
54         .trim()
55         .toLowerCase()
56     );
57 }
58 return parts.join("|");
59 }
60
61 function splitCsvLine(line) {
62     const out = [];
63     let cur = "";
64     let q = false;
65     for (let i = 0; i < line.length; i++) {
66         const ch = line[i];
67         if (ch === '"') {
68             if (q && line[i + 1] === '"') {
69                 cur += '"';
70                 i++;
71             } else q = !q;
72         } else if ((ch === "," || ch === "\t" || ch === ";") && !q) {
73             out.push(cur);
74             cur = "";
75         } else cur += ch;
76     }
77     out.push(cur);
78     return out.map((s) => s.trim().replace(/^"|"$/g, ""));
79 }
80
81 function normalizeDate(raw, dayFirst) {
82     const s = String(raw).trim();
83     if (/^\d{4}-\d{2}-\d{2}$/.test(s)) return s;
84     const m = s.match(/^(\\d{1,2})[\\/-\\.](\\d{1,2})[\\/-\\.](\\d{2,4})$/);
85     if (m) {
86         let [, a, b, c] = m;
87         if (c.length === 2) c = `20${c}`;
88         const preferDay = dayFirst !== false;
89         let day;
90         let month;
91         if (Number(a) > 12) {
92             day = a;
93             month = b;
94         } else if (Number(b) > 12) {
```

```
95     month = a;
96     day = b;
97   } else if (preferDay) {
98     day = a;
99     month = b;
100   } else {
101     month = a;
102     day = b;
103   }
104   return `${c.padStart(4, "0")}-${pad(month)}-${pad(day)}`;
105 }
106 const d = new Date(s);
107 return Number.isNaN(d.getTime()) ? null : toLocalISO(d);
108 }
109
110 const HEADER_ALIASES = {
111   date: [
112     "date",
113     "txn date",
114     "txn. date",
115     "transaction date",
116     "value date",
117     "value dt",
118     "posted date",
119     "tran date",
120     "transactiondate",
121   ],
122   description: [
123     "description",
124     "narration",
125     "particulars",
126     "merchant",
127     "details",
128     "remarks",
129     "transaction remarks",
130     "narration/description",
131   ],
132   amount: ["amount", "txn amount", "transaction amount", "amt"],
133   debit: [
134     "debit",
135     "withdrawal",
136     "debit amount",
137     "withdrawals",
138     "withdrawal amt",
139     "withdrawal amt.",
140     "dr",
141     "debit amt",
142   ],
```

```
143     credit: ["credit", "deposit", "credit amount", "deposits", "deposit amt", "deposit amt.", "cr", "credit amt"],
144     type: ["type", "dr/cr", "transaction type", "cr/dr"],
145     account: ["account", "account name", "account number", "a/c no", "account no"],
146     category: ["category"],
147     balance: ["balance", "closing balance", "running balance"],
148 };
149
150 function findHeader(headers, aliases) {
151     for (const a of aliases) {
152         const i = headers.indexOf(a);
153         if (i >= 0) return i;
154     }
155     return -1;
156 }
157
158 function detectCsvMapping(headerLine) {
159     const headers = splitCsvLine(headerLine).map((h) => h.trim().toLowerCase());
160     const idx = {};
161     for (const [key, aliases] of Object.entries(HEADER_ALIASES)) {
162         idx[key] = findHeader(headers, aliases);
163     }
164     return { headers, idx };
165 }
166
167 function parseCsvRows(text, options) {
168     const dayFirst = options?.dayFirst !== false;
169     const defaultAccount = options?.defaultAccount || "Imported bank";
170     const guessCategory = options?.guessCategory || (() => "Other");
171     const categories = options?.categories || [];
172     const uid = options?.uid || (() => `t_${Date.now()}`);
173     const mapping = options?.mapping;
174
175     const lines = text
176         .replace(/\uFEFF/, "")
177         .split(/\r?\n/)
178         .map((l) => l.trim())
179         .filter(Boolean);
180     if (lines.length < 2) throw new Error("CSV needs a header and rows.");
181     const detected = detectCsvMapping(lines[0]);
182     const idx = mapping || detected.idx;
183     if (idx.date < 0) throw new Error("Could not find a Date column.");
184
185     const imported = [];
186     for (let i = 1; i < lines.length; i++) {
187         const cols = splitCsvLine(lines[i]);
188         const date = normalizeDate(cols[idx.date] || "", dayFirst);
189         if (!date) continue;
190         const merchant = (idx.description >= 0 ? cols[idx.description] : "Imported")?.trim() || "Imported";
```

```
191     let amount = 0;
192     let type = "expense";
193     if (idx.debit >= 0 || idx.credit >= 0) {
194         const debit = parseAmount(cols[idx.debit] || 0);
195         const credit = parseAmount(cols[idx.credit] || 0);
196         if (credit > 0 && debit <= 0) {
197             amount = credit;
198             type = "income";
199         } else {
200             amount = debit || Math.abs(parseAmount(cols[idx.amount] || 0));
201             type = "expense";
202         }
203     } else {
204         const raw = parseAmount(cols[idx.amount] || 0);
205         const typeRaw = (idx.type >= 0 ? cols[idx.type] : "").toLowerCase();
206         amount = Math.abs(raw);
207         type = /credit|cr|income|deposit/.test(typeRaw) ? "income" : "expense";
208         if (raw < 0) type = "expense";
209     }
210     if (!amount) continue;
211     const account = (idx.account >= 0 && cols[idx.account] ? cols[idx.account] : defaultAccount).trim();
212     let category =
213         idx.category >= 0 && cols[idx.category] ? cols[idx.category].trim() : guessCategory(merchant, type);
214     if (categories.length && !categories.includes(category)) category = guessCategory(merchant, type);
215     imported.push({
216         id: uid(),
217         date,
218         merchant,
219         amount,
220         type,
221         category,
222         account,
223         note: "CSV import",
224         fingerprint: fingerprint({ date, type, amount, merchant, account }),
225     });
226 }
227 if (!imported.length) throw new Error("No valid rows found.");
228 return { rows: imported, headers: detected.headers, idx: detected.idx };
229 }
230
231 function defaultBudgets(currency) {
232     const inr = {
233         "Food & Dining": 8000,
234         Groceries: 6000,
235         Transport: 4000,
236         Shopping: 5000,
237         Entertainment: 2500,
238         "Bills & Utilities": 4000,
```

```
239     "Rent / Housing": 22000,
240     Health: 2000,
241     Subscriptions: 1500,
242     Travel: 3000,
243     Education: 2000,
244     Other: 2000,
245   };
246   const scale = { INR: 1, USD: 0.012, EUR: 0.011, GBP: 0.0095 }[currency] || 1;
247   if (scale === 1) return { ...inr };
248   const out = {};
249   for (const [k, v] of Object.entries(inr)) out[k] = Math.round(v * scale);
250   return out;
251 }
252
253 function applyLedger(accounts, openingBalances, transactions) {
254   const bal = {};
255   for (const name of accounts) bal[name] = roundMoney(openingBalances[name] || 0);
256   for (const t of transactions) {
257     if (!bal[t.account] && bal[t.account] !== 0) bal[t.account] = roundMoney(openingBalances[t.account] || 0);
258     if (t.type === "income") bal[t.account] = roundMoney(bal[t.account] + t.amount);
259     else if (t.type === "transfer") {
260       bal[t.account] = roundMoney(bal[t.account] - t.amount);
261       const dest = t.transferTo;
262       if (dest) {
263         if (!bal[dest] && bal[dest] !== 0) bal[dest] = roundMoney(openingBalances[dest] || 0);
264         bal[dest] = roundMoney(bal[dest] + t.amount);
265       }
266     } else bal[t.account] = roundMoney(bal[t.account] - t.amount);
267   }
268   return bal;
269 }
270
271 root.PMCore = {
272   toLocalISO,
273   fromLocalISO,
274   shiftLocalISO,
275   roundMoney,
276   parseAmount,
277   fingerprint,
278   splitCsvLine,
279   normalizeDate,
280   detectCsvMapping,
281   parseCsvRows,
282   defaultBudgets,
283   applyLedger,
284   HEADER_ALIASES,
285 };
286 })(typeof window !== "undefined" ? window : globalThis);
```

styles.css

```
1 :root {
2   --ink: #f6f1ec;
3   --ink-soft: #d8cfc8;
4   --muted: #a89890;
5   --paper: #1a1016;
6   --panel: rgba(36, 22, 28, 0.9);
7   --panel-solid: #2a1b22;
8   --line: rgba(255, 214, 210, 0.12);
9   --teal: #e11d2e;
10  --teal-deep: #b0101e;
11  --mint: rgba(225, 29, 46, 0.22);
12  --coral: #ff7a72;
13  --amber: #f0b429;
14  --sky: #c9d4dc;
15  --good: #6ee7b7;
16  --shadow: 0 22px 50px rgba(12, 4, 8, 0.45);
17  --radius: 20px;
18  --font: "Outfit", system-ui, sans-serif;
19  --display: "Syne", "Outfit", sans-serif;
20  --mx: 60vw;
21  --my: 18vh;
22  --tilt-x: 0deg;
23  --tilt-y: 0deg;
24  --parx: 0px;
25  --pary: 0px;
26 }
27
28 *,
29 *::before,
30 *::after {
31   box-sizing: border-box;
32 }
33
34 html {
35   color-scheme: dark;
36   -webkit-text-size-adjust: 100%;
37   background: #1a1016;
38 }
39
40 body {
41   margin: 0;
42   min-height: 100vh;
43   min-height: 100dvh;
44   font-family: var(--font);
45   color: var(--ink);
46   background: transparent;
```

```
47   line-height: 1.45;
48   padding-left: env(safe-area-inset-left);
49   padding-right: env(safe-area-inset-right);
50 }
51
52 .bg-wallpaper,
53 .bg-mesh,
54 .bg-scene,
55 .bg-texture,
56 .bg-grain,
57 .bg-spot,
58 .bg-vignette {
59   position: fixed;
60   inset: 0;
61   pointer-events: none;
62   z-index: 0;
63 }
64
65 .bg-wallpaper {
66   z-index: 0;
67   background:
68     radial-gradient(1200px 700px at 12% -8%, #5a1a28 0%, transparent 55%),
69     radial-gradient(900px 620px at 92% 8%, #1e3a4c 0%, transparent 48%),
70     radial-gradient(800px 500px at 70% 100%, #3a1840 0%, transparent 50%),
71     linear-gradient(165deg, #241018 0%, #16141f 42%, #1c1218 100%);
72 }
73
74 .bg-mesh {
75   z-index: 1;
76   opacity: 0.55;
77   background:
78     radial-gradient(circle at 20% 30%, rgba(255, 90, 90, 0.22), transparent 28%),
79     radial-gradient(circle at 78% 18%, rgba(80, 140, 180, 0.18), transparent 26%),
80     radial-gradient(circle at 60% 72%, rgba(180, 70, 120, 0.16), transparent 30%);
81   filter: blur(8px);
82   transform: translate3d(var(--parx), var(--pary), 0) scale(1.08);
83   animation: mesh-drift 22s ease-in-out infinite alternate;
84 }
85
86 .bg-scene {
87   z-index: 1;
88   isolation: isolate;
89 }
90
91 .orb {
92   position: absolute;
93   border-radius: 50%;
94   filter: blur(2px);
```

```
95     transform-style: preserve-3d;
96 }
97
98 .orb-a {
99     width: 280px;
100    height: 280px;
101    left: 6%;
102    top: 12%;
103    background: radial-gradient(circle at 35% 30%, #ff8a93, #e11d2e 42%, rgba(80, 10, 20, 0) 70%);
104    opacity: 0.45;
105    animation: float-3d 14s ease-in-out infinite;
106 }
107
108 .orb-b {
109     width: 220px;
110     height: 220px;
111     right: 8%;
112     top: 18%;
113     background: radial-gradient(circle at 40% 32%, #7ec8e3, #2a5f78 46%, transparent 70%);
114     opacity: 0.28;
115     animation: float-3d 18s ease-in-out infinite reverse;
116 }
117
118 .orb-c {
119     width: 340px;
120     height: 340px;
121     left: 42%;
122     bottom: -8%;
123     background: radial-gradient(circle at 40% 30%, #c45c8a, #4a1838 50%, transparent 72%);
124     opacity: 0.22;
125     animation: float-3d 20s ease-in-out infinite 1s;
126 }
127
128 .shard {
129     display: none;
130 }
131
132 .shard.s1 {
133     top: 22%;
134     left: 18%;
135     animation: shard-spin 16s linear infinite;
136 }
137 .shard.s2 {
138     top: 58%;
139     right: 14%;
140     width: 110px;
141     animation: shard-spin 22s linear infinite reverse;
142 }
```

```
143 .shard.s3 {
144   bottom: 12%;
145   left: 8%;
146   width: 90px;
147   height: 90px;
148   border-radius: 50%;
149   animation: shard-spin 19s linear infinite;
150 }
151
152 .bg-texture {
153   z-index: 3;
154   opacity: 0.22;
155   mix-blend-mode: overlay;
156   background-image:
157     url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='180' height='180'%3E%3Cfilter id='n'%3E%3CfeTurbulence type='fractalNoise' baseFrequency='0.8' numOctaves='4' stitchTiles='stitch'/%3E%3C/filter%3E%3Crect width='180' height='180' filter='url(%23n)'/%3E%3C/svg%3E"),
158     repeating-linear-gradient(90deg, rgba(255, 220, 210, 0.04) 0 1px, transparent 1px 4px),
159     repeating-linear-gradient(0deg, rgba(40, 20, 24, 0.18) 0 1px, transparent 1px 5px);
160   background-size: 180px 180px, 5px 5px, 6px 6px;
161 }
162
163 .bg-grain {
164   z-index: 4;
165   opacity: 0.12;
166   mix-blend-mode: soft-light;
167   background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='220' height='220'%3E%3Cfilter id='g'%3E%3CfeTurbulence type='fractalNoise' baseFrequency='1.2' numOctaves='3'/%3E%3C/filter%3E%3Crect width='100%25' height='100%25' filter='url(%23g)'/%3E%3C/svg%3E");
168   background-size: 220px 220px;
169   animation: grain 1.2s steps(4) infinite;
170 }
171
172 .bg-spot {
173   z-index: 5;
174   background: radial-gradient(520px circle at var(--mx) var(--my), rgba(255, 90, 100, 0.18), transparent 46%);
175 }
176
177 .bg-vignette {
178   z-index: 6;
179   background: radial-gradient(ellipse at center, transparent 62%, rgba(10, 4, 8, 0.28) 100%);
180 }
181
182 @keyframes mesh-drift {
183   from {
184     transform: translate3d(var(--parx), var(--pary), 0) scale(1.04) rotate(0deg);
185   }
186   to {
187     transform: translate3d(calc(var(--parx) + 20px), calc(var(--pary) - 16px), 0) scale(1.12) rotate(4deg);
188   }
```

```
189 }
190
191 @keyframes float-3d {
192     0%,
193     100% {
194         transform: translate3d(0, 0, 40px) rotateX(8deg);
195     }
196     50% {
197         transform: translate3d(18px, -22px, 80px) rotateX(-6deg);
198     }
199 }
200
201 @keyframes shard-spin {
202     from {
203         transform: rotateX(58deg) rotateY(0deg) rotateZ(-18deg);
204     }
205     to {
206         transform: rotateX(58deg) rotateY(360deg) rotateZ(-18deg);
207     }
208 }
209
210 @keyframes grain {
211     0%,
212     100% {
213         transform: translate(0, 0);
214     }
215     25% {
216         transform: translate(-1.5%, 1%);
217     }
218     50% {
219         transform: translate(1%, -1.2%);
220     }
221     75% {
222         transform: translate(-0.6%, -0.8%);
223     }
224 }
225
226 .screen,
227 .topbar,
228 .layout,
229 .demo-banner {
230     position: relative;
231     z-index: 20;
232 }
233
234 .bg-grid,
235 .bg-orbs {
236     display: none;
```

```
237 }
238
239 .screen[hidden],
240 .view[hidden],
241 .wizard-step[hidden],
242 [hidden] {
243     display: none !important;
244 }
245
246 #screen-welcome:not([hidden]),
247 #screen-setup:not([hidden]),
248 #screen-login:not([hidden]),
249 #screen-app:not([hidden]) {
250     display: grid !important;
251     position: relative;
252     z-index: 30;
253 }
254
255 /* Welcome */
256 #screen-welcome {
257     min-height: 100vh;
258     display: grid;
259     place-items: center;
260     padding: 1.5rem;
261 }
262
263 .welcome-stage {
264     width: min(1080px, 100%);
265     display: grid;
266     grid-template-columns: minmax(0, 1fr) minmax(240px, 0.72fr);
267     gap: 2rem;
268     align-items: center;
269     perspective: 1200px;
270 }
271
272 .welcome-card {
273     width: 100%;
274     position: relative;
275     z-index: 2;
276     background: linear-gradient(165deg, rgba(52, 24, 32, 0.92), rgba(22, 14, 20, 0.9));
277     border: 1px solid rgba(255, 180, 180, 0.18);
278     border-radius: 28px;
279     box-shadow:
280         0 0 0 1px rgba(255, 255, 255, 0.04),
281         0 30px 80px rgba(80, 10, 20, 0.35),
282         inset 0 1px 0 rgba(255, 255, 255, 0.16);
283     padding: 2rem 1.75rem;
284     backdrop-filter: blur(22px) saturate(1.2);
```

```
285 }
286
287 .live-pill {
288   display: inline-flex;
289   align-items: center;
290   gap: 0.45rem;
291   margin: 1.1rem 0 0;
292   padding: 0.28rem 0.7rem;
293   border-radius: 999px;
294   border: 1px solid rgba(225, 29, 46, 0.35);
295   color: #ffb3ba;
296   font-size: 0.75rem;
297   font-weight: 700;
298   letter-spacing: 0.04em;
299   text-transform: uppercase;
300 }
301
302 .privacy-note {
303   margin: 0.45rem 0 0.85rem;
304   max-width: 42ch;
305 }
306
307 .pulse {
308   width: 8px;
309   height: 8px;
310   border-radius: 50%;
311   background: #e11d2e;
312   box-shadow: 0 0 0 0 rgba(225, 29, 46, 0.7);
313   animation: pulse 1.8s infinite;
314 }
315
316 @keyframes pulse {
317   70% {
318     box-shadow: 0 0 0 8px rgba(225, 29, 46, 0);
319   }
320 }
321
322 .how-steps {
323   display: grid;
324   grid-template-columns: repeat(3, 1fr);
325   gap: 0.55rem;
326   list-style: none;
327   padding: 0;
328   margin: 0 0 1.35rem;
329 }
330
331 .how-steps li {
332   display: grid;
```

```
333   gap: 0.35rem;
334   padding: 0.7rem 0.65rem;
335   border-radius: 14px;
336   border: 1px solid rgba(255, 255, 255, 0.08);
337   background: rgba(255, 255, 255, 0.03);
338   font-size: 0.82rem;
339   font-weight: 600;
340   color: var(--ink-soft);
341 }
342
343 .how-steps span {
344   width: 22px;
345   height: 22px;
346   display: grid;
347   place-items: center;
348   border-radius: 7px;
349   background: var(--teal);
350   color: #fff;
351   font-size: 0.72rem;
352 }
353
354 .step-pills {
355   display: flex;
356   flex-wrap: wrap;
357   gap: 0.4rem;
358   margin: 0.85rem 0 0.4rem;
359 }
360
361 .step-pill {
362   padding: 0.28rem 0.7rem;
363   border-radius: 999px;
364   border: 1px solid var(--line);
365   font-size: 0.75rem;
366   font-weight: 700;
367   color: var(--muted);
368 }
369
370 .step-pill.on {
371   border-color: var(--teal);
372   background: rgba(225, 29, 46, 0.18);
373   color: #fff;
374   box-shadow: 0 0 16px rgba(225, 29, 46, 0.25);
375 }
376
377 .hud-card {
378   border-color: rgba(225, 29, 46, 0.22);
379   box-shadow: var(--shadow), inset 0 1px 0 rgba(255, 255, 255, 0.1), 0 0 40px rgba(225, 29, 46, 0.08);
380 }
```

```
381
382 .panel-kicker {
383   font-size: 0.7rem;
384   font-weight: 800;
385   letter-spacing: 0.14em;
386   text-transform: uppercase;
387   color: #ff8a93;
388 }
389
390 .choice-grid {
391   display: grid;
392   grid-template-columns: 1fr 1fr 1fr;
393   gap: 0.55rem;
394   border: 0;
395   padding: 0;
396 }
397
398 .choice-grid legend {
399   padding: 0;
400   margin-bottom: 0.45rem;
401 }
402
403 .choice {
404   display: grid;
405   justify-items: start;
406   gap: 0.15rem;
407   padding: 0.85rem 0.9rem;
408   border-radius: 16px;
409   border: 1px solid var(--line);
410   background: rgba(255, 255, 255, 0.03);
411   cursor: pointer;
412   position: relative;
413 }
414
415 .choice input {
416   position: absolute;
417   opacity: 0;
418   pointer-events: none;
419 }
420
421 .choice-ico {
422   width: 28px;
423   height: 28px;
424   border-radius: 8px;
425   display: grid;
426   place-items: center;
427   background: rgba(225, 29, 46, 0.2);
428   color: #ff8a93;
```

```
429   font-weight: 800;
430   margin-bottom: 0.2rem;
431 }
432
433 .choice-ico.in {
434   background: rgba(125, 206, 160, 0.18);
435   color: var(--good);
436 }
437
438 .choice-title {
439   font-weight: 700;
440   font-size: 1rem;
441 }
442
443 .choice-sub {
444   font-size: 0.78rem;
445   color: var(--muted);
446 }
447
448 .choice:has(input:checked) {
449   border-color: var(--teal);
450   background: rgba(225, 29, 46, 0.16);
451   box-shadow: 0 10px 24px rgba(225, 29, 46, 0.22), inset 0 1px 0 rgba(255, 255, 255, 0.12);
452   transform: translateZ(10px);
453 }
454
455 .amount-field input {
456   font-family: var(--display);
457   font-size: 1.85rem;
458   font-weight: 700;
459   letter-spacing: -0.04em;
460   padding: 0.9rem 1rem;
461   background: radial-gradient(120% 140% at 0% 0%, rgba(255, 90, 100, 0.16), rgba(20, 10, 14, 0.7));
462   box-shadow: inset 0 2px 10px rgba(0, 0, 0, 0.28);
463 }
464
465 .form-row-2 {
466   display: grid;
467   grid-template-columns: 1fr 1fr;
468   gap: 0.7rem;
469 }
470
471 .stack.tight {
472   gap: 0.45rem;
473   margin: 0.75rem 0 1rem;
474 }
475
476 .welcome-card h1 {
```

```
477   margin: 1.25rem 0 0.5rem;
478   font-family: var(--display);
479   font-size: clamp(1.6rem, 4vw, 2.1rem);
480   letter-spacing: -0.03em;
481 }
482
483 .lede {
484   color: var(--ink-soft);
485   margin: 0 0 1.5rem;
486 }
487
488 .welcome-actions {
489   display: grid;
490   gap: 0.75rem;
491   position: relative;
492   z-index: 5;
493 }
494
495 .welcome-actions .btn {
496   pointer-events: auto;
497   position: relative;
498   z-index: 5;
499 }
500
501 .hero-orbit {
502   position: relative;
503   min-height: 380px;
504   pointer-events: none;
505   z-index: 1;
506 }
507
508 .welcome-card {
509   position: relative;
510   z-index: 2;
511 }
512
513 .float-card {
514   position: absolute;
515   width: min(220px, 80%);
516   padding: 1rem 1.1rem;
517   border-radius: 18px;
518   background: linear-gradient(160deg, rgba(255, 255, 255, 0.14), rgba(40, 16, 22, 0.55));
519   border: 1px solid rgba(255, 255, 255, 0.18);
520   box-shadow: 16px 28px 40px rgba(0, 0, 0, 0.32), inset 0 1px 0 rgba(255, 255, 255, 0.28);
521   backdrop-filter: blur(16px);
522   transform-style: preserve-3d;
523 }
524
```

```
525 .float-card strong {
526   display: block;
527   font-family: var(--display);
528   font-size: 1.7rem;
529   letter-spacing: -0.04em;
530   margin: 0.15rem 0;
531 }
532
533 .fc-kicker {
534   font-size: 0.68rem;
535   font-weight: 800;
536   letter-spacing: 0.14em;
537   text-transform: uppercase;
538   color: #ffb3ba;
539 }
540
541 .float-card em {
542   font-style: normal;
543   color: var(--muted);
544   font-size: 0.82rem;
545 }
546
547 .mini-bar {
548   height: 8px;
549   border-radius: 999px;
550   background: rgba(255, 255, 255, 0.08);
551   overflow: hidden;
552   margin: 0.55rem 0 0.35rem;
553 }
554
555 .mini-bar i {
556   display: block;
557   height: 100%;
558   background: linear-gradient(90deg, #ff8a93, var(--teal));
559 }
560
561 .fc-a {
562   top: 18px;
563   left: 8%;
564   transform: rotateY(-18deg) rotateX(8deg) translateZ(40px);
565   animation: card-float 7s ease-in-out infinite;
566 }
567
568 .fc-b {
569   top: 132px;
570   right: 0;
571   transform: rotateY(16deg) rotateX(-6deg) translateZ(70px);
572   animation: card-float 8.5s ease-in-out infinite 0.4s;
```

```
573 }
574
575 .fc-c {
576   bottom: 28px;
577   left: 18%;
578   transform: rotateY(-10deg) rotateX(12deg) translateZ(24px);
579   animation: card-float 9s ease-in-out infinite 0.8s;
580 }
581
582 .coin {
583   position: absolute;
584   width: 54px;
585   height: 54px;
586   border-radius: 50%;
587   background:
588     radial-gradient(circle at 32% 28%, #ffe08a, #d4a017 42%, #8a5a10 70%);
589   box-shadow: inset -6px -8px 0 rgba(0, 0, 0, 0.18), 8px 12px 18px rgba(0, 0, 0, 0.3);
590   transform: rotateY(40deg);
591 }
592
593 .coin-1 {
594   top: 8%;
595   right: 18%;
596   animation: coin-spin 6s linear infinite;
597 }
598
599 .coin-2 {
600   bottom: 12%;
601   right: 8%;
602   width: 36px;
603   height: 36px;
604   animation: coin-spin 8s linear infinite reverse;
605 }
606
607 @keyframes card-float {
608   0%,
609   100% {
610     translate: 0 0 0;
611   }
612   50% {
613     translate: 0 -12px 12px;
614   }
615 }
616
617 @keyframes coin-spin {
618   from {
619     transform: rotateY(0deg) rotateX(18deg);
620   }
```

```
621   to {
622     transform: rotateY(360deg) rotateX(18deg);
623   }
624 }
625
626 @keyframes ring-hover {
627   0%,
628   100% {
629     transform: rotateX(12deg) rotateY(-8deg);
630   }
631   50% {
632     transform: rotateX(-6deg) rotateY(10deg);
633   }
634 }
635
636 .brand {
637   display: flex;
638   gap: 0.85rem;
639   align-items: center;
640 }
641
642 .brand-hero {
643   display: flex;
644   gap: 1rem;
645   align-items: center;
646   margin-bottom: 0.15rem;
647 }
648
649 .brand-hero-logo {
650   width: 88px;
651   height: 88px;
652   border-radius: 22px;
653   object-fit: cover;
654   flex-shrink: 0;
655   box-shadow: 0 14px 32px rgba(225, 29, 46, 0.35), inset 0 1px 0 rgba(255, 255, 255, 0.25);
656   border: 1px solid rgba(255, 255, 255, 0.12);
657 }
658
659 .brand.big .brand-mark {
660   width: 48px;
661   height: 48px;
662 }
663
664 .brand-mark {
665   width: 42px;
666   height: 42px;
667   border-radius: 14px;
668   background: transparent;
```

```
669   position: relative;
670   flex-shrink: 0;
671   overflow: hidden;
672   box-shadow: 0 8px 16px rgba(225, 29, 46, 0.35);
673 }
674
675 .brand-logo {
676   width: 100%;
677   height: 100%;
678   object-fit: cover;
679   display: block;
680   border-radius: inherit;
681 }
682
683 .brand-mark.has-photo .brand-logo {
684   display: none;
685 }
686
687 .brand-mark::before {
688   content: "";
689   position: absolute;
690   inset: 0;
691   z-index: 2;
692   pointer-events: none;
693   background: linear-gradient(110deg, transparent 40%, rgba(255, 255, 255, 0.28) 50%, transparent 60%);
694   animation: sheen 4.5s ease-in-out infinite;
695 }
696
697 @keyframes sheen {
698   0% {
699     transform: translateX(-120%);
700   }
701   40%,
702   100% {
703     transform: translateX(120%);
704   }
705 }
706
707 .brand-mark::after {
708   display: none;
709 }
710
711 .brand-name {
712   margin: 0;
713   font-family: var(--display);
714   font-size: 1.35rem;
715   font-weight: 700;
716   letter-spacing: -0.03em;
```

```
717 }
718
719 .brand-hero .brand-name {
720   font-size: clamp(1.85rem, 4vw, 2.35rem);
721 }
722
723 .brand-tag {
724   margin: 0.15rem 0 0;
725   color: var(--muted);
726   font-size: 0.85rem;
727 }
728
729 .brand-motto {
730   font-weight: 600;
731   letter-spacing: 0.02em;
732   color: #ffb3ba;
733 }
734
735 .brand-hero .brand-motto {
736   font-size: 0.95rem;
737 }
738
739 /* Wizard */
740 #screen-setup {
741   min-height: 100vh;
742   display: grid;
743   place-items: start center;
744   padding: 2rem 1rem 3rem;
745   position: relative;
746   z-index: 30;
747 }
748
749 .wizard {
750   width: min(640px, 100%);
751 }
752
753 .wizard-head h1 {
754   margin: 0.35rem 0;
755   font-family: var(--display);
756   letter-spacing: -0.02em;
757 }
758
759 .eyebrow {
760   margin: 0;
761   font-size: 0.75rem;
762   font-weight: 700;
763   letter-spacing: 0.08em;
764   text-transform: uppercase;
```

```
765   color: var(--teal);
766 }
767
768 .progress {
769   height: 8px;
770   border-radius: 999px;
771   background: rgba(225, 29, 46, 0.12);
772   margin: 1rem 0 1.25rem;
773   overflow: hidden;
774 }
775
776 .progress-fill {
777   height: 100%;
778   width: 20%;
779   background: var(--teal);
780   border-radius: inherit;
781   transition: width 0.3s ease;
782 }
783
784 .wizard-step {
785   display: grid;
786   gap: 0.85rem;
787 }
788
789 .wizard-step h2 {
790   margin: 0;
791   font-family: var(--display);
792   font-size: 1.35rem;
793 }
794
795 .wizard-foot {
796   display: flex;
797   justify-content: space-between;
798   gap: 0.75rem;
799   margin-top: 1rem;
800 }
801
802 /* App chrome */
803 .topbar {
804   position: sticky;
805   top: 0;
806   z-index: 30;
807   display: flex;
808   flex-wrap: wrap;
809   gap: 0.85rem 1rem;
810   align-items: center;
811   justify-content: space-between;
812   padding: 0.9rem 1.25rem;
```

```
813   backdrop-filter: blur(22px) saturate(1.3);
814   background: rgba(22, 12, 16, 0.62);
815   border-bottom: 1px solid rgba(255, 180, 180, 0.14);
816 }
817
818 .save-cluster {
819   display: flex;
820   align-items: center;
821   gap: 0.65rem;
822   margin-left: auto;
823 }
824
825 .save-status.flash {
826   animation: status-flash 0.8s ease;
827 }
828
829 @keyframes status-flash {
830   0% {
831     color: #fff;
832   }
833   100% {
834     color: var(--good);
835   }
836 }
837
838 .save-options {
839   display: grid;
840   gap: 0.75rem;
841 }
842
843 .save-option {
844   padding: 0.9rem;
845   border-radius: 14px;
846   border: 1px solid var(--line);
847   background: rgba(255, 255, 255, 0.03);
848   display: grid;
849   gap: 0.45rem;
850 }
851
852 .save-option p {
853   margin: 0;
854   color: var(--muted);
855   font-size: 0.86rem;
856 }
857
858 dialog#save-dialog {
859   width: min(460px, calc(100% - 2rem));
860 }
```

```
861
862 .main-tabs {
863   display: flex;
864   flex-wrap: wrap;
865   gap: 0.35rem;
866   padding: 0.25rem;
867   background: rgba(225, 29, 46, 0.08);
868   border-radius: 999px;
869 }
870
871 .tab {
872   border: 0;
873   background: transparent;
874   font: inherit;
875   font-weight: 600;
876   font-size: 0.9rem;
877   padding: 0.5rem 0.95rem;
878   border-radius: 999px;
879   cursor: pointer;
880   color: var(--ink-soft);
881   display: inline-flex;
882   align-items: center;
883   gap: 0.4rem;
884   min-height: 42px;
885 }
886
887 .tab-ico {
888   font-size: 0.85rem;
889   line-height: 1;
890 }
891
892 .tab-label {
893   white-space: nowrap;
894 }
895
896 .tab.active {
897   background: var(--teal);
898   color: #fff;
899   box-shadow: 0 4px 14px rgba(225, 29, 46, 0.35);
900 }
901
902 .demo-banner {
903   display: flex;
904   flex-wrap: wrap;
905   gap: 0.75rem;
906   align-items: center;
907   justify-content: space-between;
908   padding: 0.75rem 1.25rem;
```

```
909   background: #2a1214;
910   border-bottom: 1px solid rgba(225, 29, 46, 0.45);
911   color: #f3c4c8;
912   font-size: 0.92rem;
913 }
914
915 .layout {
916   width: min(1100px, calc(100% - 2rem));
917   margin: 1.25rem auto 3rem;
918   display: grid;
919   gap: 1rem;
920 }
921
922 .panel {
923   background: var(--panel);
924   border: 1px solid var(--line);
925   border-radius: var(--radius);
926   box-shadow: var(--shadow), inset 0 1px 0 rgba(255, 255, 255, 0.08);
927   padding: 1.15rem 1.25rem;
928   backdrop-filter: blur(16px) saturate(1.15);
929 }
930
931 @keyframes rise {
932   from {
933     opacity: 0;
934     transform: translateY(8px);
935   }
936   to {
937     opacity: 1;
938     transform: none;
939   }
940 }
941
942 .intro-panel h1 {
943   margin: 0;
944   font-family: var(--display);
945   font-size: clamp(1.45rem, 3vw, 1.9rem);
946   letter-spacing: -0.02em;
947 }
948
949 .intro-panel.rowish {
950   display: flex;
951   flex-wrap: wrap;
952   justify-content: space-between;
953   gap: 1rem;
954   align-items: center;
955 }
956
```

```
957 .muted {
958   margin: 0.35rem 0 0;
959   color: var(--muted);
960   font-size: 0.92rem;
961 }
962
963 .small {
964   font-size: 0.8rem;
965 }
966
967 .grid-2-track {
968   display: grid;
969   grid-template-columns: minmax(280px, 400px) 1fr;
970   gap: 1rem;
971   align-items: start;
972 }
973
974 .grid-2 {
975   display: grid;
976   grid-template-columns: 1.1fr 1fr;
977   gap: 1rem;
978 }
979
980 .form-card {
981   display: grid;
982   gap: 0.75rem;
983 }
984
985 .form-card.narrow {
986   width: min(640px, 100%);
987 }
988
989 .profile-identity {
990   display: grid;
991   grid-template-columns: auto 1fr;
992   gap: 1rem;
993   align-items: center;
994   padding: 0.9rem;
995   border-radius: 16px;
996   border: 1px solid var(--line);
997   background: rgba(225, 29, 46, 0.08);
998 }
999
1000 .avatar-wrap {
1001   width: 96px;
1002   height: 96px;
1003   border-radius: 50%;
1004   overflow: hidden;
```

```
1005 border: 2px solid rgba(225, 29, 46, 0.55);
1006 background: #1a1a1a;
1007 display: grid;
1008 place-items: center;
1009 flex-shrink: 0;
1010 position: relative;
1011 }
1012
1013 .avatar-wrap img {
1014   width: 100%;
1015   height: 100%;
1016   object-fit: cover;
1017   display: block;
1018 }
1019
1020 .avatar-fallback {
1021   font-size: 2rem;
1022   font-weight: 700;
1023   color: #ff8a93;
1024 }
1025
1026 .brand-mark img {
1027   position: absolute;
1028   inset: 0;
1029   width: 100%;
1030   height: 100%;
1031   object-fit: cover;
1032   border-radius: inherit;
1033 }
1034
1035 #header-photo {
1036   z-index: 1;
1037 }
1038
1039 .brand-mark.has-photo::after {
1040   display: none;
1041 }
1042
1043 textarea {
1044   resize: vertical;
1045   min-height: 84px;
1046 }
1047
1048 .form-card h2,
1049 .panel-head h2,
1050 .subhead {
1051   margin: 0;
1052   font-family: var(--display);
```

```
1053     font-size: 1.25rem;
1054 }
1055
1056 .step-hint {
1057     margin: -0.25rem 0 0.25rem;
1058     color: #ff8a93;
1059     font-size: 0.88rem;
1060     font-weight: 600;
1061 }
1062
1063 label,
1064 .field-label {
1065     display: grid;
1066     gap: 0.35rem;
1067     font-size: 0.82rem;
1068     font-weight: 600;
1069     color: var(--ink-soft);
1070 }
1071
1072 input,
1073 select,
1074 textarea {
1075     width: 100%;
1076     border: 1px solid var(--line);
1077     border-radius: 12px;
1078     padding: 0.75rem 0.85rem;
1079     font: inherit;
1080     font-size: 16px;
1081     background: linear-gradient(180deg, rgba(48, 28, 34, 0.95), rgba(24, 14, 18, 0.92));
1082     color: var(--ink);
1083     max-width: 100%;
1084     box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.06);
1085 }
1086
1087 /* Native dropdown lists often ignore page theme ? force readable contrast */
1088 select {
1089     background-color: var(--panel-solid);
1090     color: var(--ink);
1091 }
1092
1093 select option,
1094 select optgroup {
1095     background-color: #2a1b22;
1096     color: #f6f1ec;
1097 }
1098
1099 select option:checked,
1100 select option:hover,
```

```
1101 select option:focus {
1102   background-color: #3d2430;
1103   color: #fff;
1104 }
1105
1106 input:focus,
1107 select:focus,
1108 .btn:focus-visible,
1109 .tab:focus-visible,
1110 .seg:focus-visible {
1111   outline: 2px solid rgba(225, 29, 46, 0.35);
1112   outline-offset: 1px;
1113 }
1114
1115 fieldset.type-pick {
1116   border: 1px solid var(--line);
1117   border-radius: 14px;
1118   padding: 0.75rem;
1119   margin: 0;
1120   display: grid;
1121   gap: 0.45rem;
1122 }
1123
1124 fieldset.type-pick legend {
1125   padding: 0 0.35rem;
1126   font-size: 0.82rem;
1127   font-weight: 700;
1128 }
1129
1130 .chip {
1131   display: flex;
1132   align-items: center;
1133   gap: 0.5rem;
1134   padding: 0.55rem 0.65rem;
1135   border-radius: 10px;
1136   background: rgba(225, 29, 46, 0.06);
1137   cursor: pointer;
1138   font-size: 0.92rem;
1139 }
1140
1141 .chip input {
1142   width: auto;
1143 }
1144
1145 .btn {
1146   appearance: none;
1147   border: 1px solid transparent;
1148   border-radius: 999px;
```

```
1149 padding: 0.7rem 1.05rem;
1150 font: inherit;
1151 font-weight: 600;
1152 font-size: 0.95rem;
1153 cursor: pointer;
1154 touch-action: manipulation;
1155 min-height: 44px;
1156 transition: transform 0.18s ease, background 0.15s ease, box-shadow 0.18s ease;
1157 }
1158
1159 .btn:hover {
1160   transform: translateY(-2px) translateZ(8px);
1161 }
1162
1163 .btn:active {
1164   transform: translateY(1px);
1165 }
1166
1167 .btn.primary {
1168   background: linear-gradient(180deg, #ff5a66 0%, var(--teal) 62%, #9b0d18 100%);
1169   color: #fff;
1170   box-shadow: 0 10px 24px rgba(225, 29, 46, 0.34), inset 0 1px 0 rgba(255, 255, 255, 0.28);
1171 }
1172
1173 .btn.primary:hover {
1174   background: var(--teal-deep);
1175 }
1176
1177 .btn.secondary {
1178   background: var(--panel-solid);
1179   border-color: var(--line);
1180   color: var(--ink);
1181 }
1182
1183 .btn.ghost {
1184   background: transparent;
1185   border-color: var(--line);
1186   color: var(--ink-soft);
1187 }
1188
1189 .btn.danger {
1190   color: var(--coral);
1191   border-color: rgba(196, 92, 62, 0.35);
1192 }
1193
1194 .btn.large {
1195   padding: 0.95rem 1.15rem;
1196   text-align: left;
```

```
1197   border-radius: 16px;
1198 }
1199
1200 .btn.block {
1201   width: 100%;
1202   display: block;
1203   text-align: center;
1204 }
1205
1206 .btn.large .btn-sub {
1207   display: block;
1208   font-weight: 500;
1209   font-size: 0.82rem;
1210   opacity: 0.85;
1211   margin-top: 0.25rem;
1212 }
1213
1214 .btn.tiny {
1215   padding: 0.35rem 0.7rem;
1216   font-size: 0.8rem;
1217 }
1218
1219 .file-btn {
1220   display: inline-flex;
1221   align-items: center;
1222   justify-content: center;
1223 }
1224
1225 .form-status {
1226   min-height: 1.25rem;
1227   margin: 0;
1228   color: var(--good);
1229   font-size: 0.88rem;
1230   font-weight: 600;
1231 }
1232
1233 .stack {
1234   display: grid;
1235   gap: 0.55rem;
1236 }
1237
1238 .stack.gap {
1239   gap: 0.85rem;
1240 }
1241
1242 .account-row {
1243   display: grid;
1244   grid-template-columns: 1fr 8.5rem auto;
```

```
1245     gap: 0.45rem;
1246     align-items: center;
1247 }
1248
1249 .budget-grid {
1250     display: grid;
1251     gap: 0.55rem;
1252     max-height: 320px;
1253     overflow: auto;
1254 }
1255
1256 .budget-grid label {
1257     grid-template-columns: 1fr 120px;
1258     align-items: center;
1259     display: grid;
1260     gap: 0.65rem;
1261 }
1262
1263 .spaced {
1264     margin-top: 1.25rem !important;
1265 }
1266
1267 .soft {
1268     border: 0;
1269     border-top: 1px solid var(--line);
1270     margin: 0.5rem 0;
1271 }
1272
1273 .controls {
1274     display: flex;
1275     flex-wrap: wrap;
1276     gap: 1rem;
1277     align-items: end;
1278 }
1279
1280 .control-group {
1281     display: grid;
1282     gap: 0.35rem;
1283     min-width: 140px;
1284 }
1285
1286 .control-group.grow {
1287     flex: 1;
1288     min-width: 160px;
1289 }
1290
1291 .segmented {
1292     display: inline-flex;
```

```
1293 padding: 0.2rem;
1294 background: rgba(225, 29, 46, 0.08);
1295 border-radius: 999px;
1296 gap: 0.15rem;
1297 }
1298
1299 .seg {
1300 border: 0;
1301 background: transparent;
1302 color: var(--ink-soft);
1303 font: inherit;
1304 font-weight: 600;
1305 font-size: 0.88rem;
1306 padding: 0.45rem 0.85rem;
1307 border-radius: 999px;
1308 cursor: pointer;
1309 }
1310
1311 .seg.active {
1312 background: var(--teal);
1313 color: #fff;
1314 box-shadow: 0 4px 12px rgba(225, 29, 46, 0.35);
1315 }
1316
1317 .date-nav {
1318 display: flex;
1319 align-items: center;
1320 gap: 0.35rem;
1321 }
1322
1323 .date-nav input[type="date"] {
1324 width: auto;
1325 }
1326
1327 .icon-round {
1328 width: 44px;
1329 height: 44px;
1330 border-radius: 50%;
1331 border: 1px solid var(--line);
1332 background: var(--panel-solid);
1333 font-size: 1.15rem;
1334 cursor: pointer;
1335 color: var(--ink);
1336 }
1337
1338 .hero-strip {
1339 display: flex;
1340 justify-content: space-between;
```

```
1341   gap: 1.25rem;
1342   align-items: center;
1343   background: linear-gradient(135deg, rgba(225, 29, 46, 0.22), rgba(20, 20, 20, 0.9));
1344 }
1345
1346 .hero-strip h2 {
1347   margin: 0.25rem 0 0;
1348   font-family: var(--display);
1349   font-size: 1.5rem;
1350 }
1351
1352 .health-meter {
1353   text-align: center;
1354   perspective: 420px;
1355 }
1356
1357 .ring-wrap {
1358   transform-style: preserve-3d;
1359   animation: ring-hover 7s ease-in-out infinite;
1360 }
1361
1362 .ring {
1363   width: 96px;
1364   height: 96px;
1365   border-radius: 50%;
1366   display: grid;
1367   place-items: center;
1368   background:
1369     radial-gradient(circle at 35% 28%, rgba(255, 255, 255, 0.18), transparent 28%),
1370     radial-gradient(circle at center, #24151b 56%, transparent 57%),
1371     conic-gradient(#fff6b75 var(--score, 0%), rgba(225, 29, 46, 0.16) 0);
1372   font-family: var(--display);
1373   font-size: 1.45rem;
1374   font-weight: 700;
1375   color: #fff;
1376   margin: 0 auto;
1377   box-shadow: 0 16px 30px rgba(0, 0, 0, 0.35), inset 0 0 0 8px rgba(20, 8, 12, 0.55);
1378 }
1379
1380 .stats {
1381   display: grid;
1382   grid-template-columns: repeat(5, 1fr);
1383   gap: 0.75rem;
1384 }
1385
1386 .stat {
1387   background: linear-gradient(180deg, rgba(48, 28, 34, 0.88), rgba(28, 16, 22, 0.78));
1388   border: 1px solid var(--line);
```

```
1389   border-radius: var(--radius);
1390   padding: 0.95rem 1rem;
1391   box-shadow: var(--shadow), inset 0 1px 0 rgba(255, 255, 255, 0.1);
1392 }
1393
1394 .stat:nth-child(2) {
1395   animation-delay: 0.04s;
1396 }
1397 .stat:nth-child(3) {
1398   animation-delay: 0.08s;
1399 }
1400 .stat:nth-child(4) {
1401   animation-delay: 0.12s;
1402 }
1403 .stat:nth-child(5) {
1404   animation-delay: 0.16s;
1405 }
1406
1407 .stat-label {
1408   margin: 0;
1409   color: var(--muted);
1410   font-size: 0.8rem;
1411   font-weight: 600;
1412 }
1413
1414 .stat-value {
1415   margin: 0.3rem 0 0;
1416   font-family: var(--display);
1417   font-size: clamp(1.15rem, 2vw, 1.55rem);
1418   font-weight: 600;
1419 }
1420
1421 .stat-value.expense {
1422   color: var(--coral);
1423 }
1424 .stat-value.income {
1425   color: var(--good);
1426 }
1427
1428 .stat-hint {
1429   margin: 0.3rem 0 0;
1430   font-size: 0.75rem;
1431   color: var(--muted);
1432 }
1433 .stat-hint.up {
1434   color: var(--coral);
1435 }
1436 .stat-hint.down {
```

```
1437   color: var(--good);
1438 }
1439
1440 .chart {
1441   min-height: 200px;
1442   display: flex;
1443   align-items: end;
1444   gap: 0.45rem;
1445   perspective: 640px;
1446   padding: 0.4rem 0.2rem 0;
1447 }
1448
1449 .bar-col {
1450   flex: 1;
1451   min-width: 0;
1452   display: flex;
1453   flex-direction: column;
1454   align-items: center;
1455   gap: 0.45rem;
1456 }
1457
1458 .bar,
1459 .bar-3d {
1460   width: 100%;
1461   max-width: 36px;
1462   min-height: 4px;
1463 }
1464
1465 .bar {
1466   border-radius: 10px 10px 4px 4px;
1467   background: linear-gradient(180deg, #ff4d5c, var(--teal));
1468 }
1469
1470 .bar-3d {
1471   height: var(--h, 40px);
1472   position: relative;
1473   transform-style: preserve-3d;
1474   transform: rotateX(18deg) rotateY(-22deg);
1475   animation: bar-in 0.6s cubic-bezier(0.2, 0.8, 0.2, 1) both;
1476 }
1477
1478 .bar-3d .face {
1479   position: absolute;
1480   left: 0;
1481   bottom: 0;
1482   width: 100%;
1483   height: 100%;
1484 }
```

```
1485
1486 .bar-3d .front {
1487   background: linear-gradient(180deg, #ff7a84, #c41224);
1488   border-radius: 6px 6px 2px 2px;
1489   box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.35);
1490 }
1491
1492 .bar-3d .top {
1493   height: 10px;
1494   top: -8px;
1495   background: linear-gradient(90deg, #ffb3b8, #ff6b75);
1496   transform: rotateX(90deg);
1497   transform-origin: bottom;
1498   border-radius: 4px;
1499 }
1500
1501 .bar-3d .side {
1502   width: 10px;
1503   left: 100%;
1504   background: linear-gradient(180deg, #a30f1c, #6b0810);
1505   transform: rotateY(90deg);
1506   transform-origin: left;
1507   border-radius: 0 4px 2px 0;
1508 }
1509
1510 @keyframes bar-in {
1511   from {
1512     transform: rotateX(18deg) rotateY(-22deg) scaleY(0.2);
1513     opacity: 0;
1514   }
1515   to {
1516     transform: rotateX(18deg) rotateY(-22deg) scaleY(1);
1517     opacity: 1;
1518   }
1519 }
1520
1521 .bar-label {
1522   font-size: 0.65rem;
1523   color: var(--muted);
1524   max-width: 100%;
1525   overflow: hidden;
1526   text-overflow: ellipsis;
1527   white-space: nowrap;
1528 }
1529
1530 .category-bars,
1531 .budget-list,
1532 .merchant-list,
```

```
1533 .recurring-list,
1534 .accounts,
1535 .insights {
1536   display: grid;
1537   gap: 0.75rem;
1538 }
1539
1540 .insights {
1541   list-style: none;
1542   margin: 0;
1543   padding: 0;
1544 }
1545
1546 .cat-row,
1547 .merchant-row {
1548   display: grid;
1549   grid-template-columns: 110px 1fr auto;
1550   gap: 0.65rem;
1551   align-items: center;
1552 }
1553
1554 .budget-row {
1555   display: grid;
1556   gap: 0.3rem;
1557   padding-bottom: 0.55rem;
1558   border-bottom: 1px solid var(--line);
1559 }
1560
1561 .budget-meta {
1562   display: flex;
1563   justify-content: space-between;
1564   font-size: 0.85rem;
1565   font-weight: 600;
1566 }
1567
1568 .budget-meta .over {
1569   color: var(--coral);
1570 }
1571
1572 .track {
1573   height: 10px;
1574   border-radius: 999px;
1575   background: rgba(225, 29, 46, 0.12);
1576   overflow: hidden;
1577   box-shadow: inset 0 1px 3px rgba(0, 0, 0, 0.35);
1578 }
1579
1580 .fill {
```

```
1581   height: 100%;
1582   border-radius: inherit;
1583   background: var(--cat, var(--teal));
1584   transition: width 0.45s ease;
1585 }
1586
1587 .fill.warn {
1588   background: linear-gradient(90deg, var(--amber), var(--coral));
1589 }
1590
1591 .cat-name,
1592 .merchant-name,
1593 .cat-amt,
1594 .merchant-amt {
1595   font-size: 0.85rem;
1596   font-weight: 600;
1597 }
1598
1599 .insight {
1600   display: grid;
1601   grid-template-columns: auto 1fr;
1602   gap: 0.7rem;
1603   padding: 0.8rem;
1604   border-radius: 14px;
1605   border: 1px solid var(--line);
1606   background: rgba(225, 29, 46, 0.08);
1607 }
1608
1609 .insight-icon {
1610   width: 34px;
1611   height: 34px;
1612   border-radius: 11px;
1613   display: grid;
1614   place-items: center;
1615   font-weight: 700;
1616   background: var(--mint);
1617   color: #fff;
1618 }
1619
1620 .insight.warn .insight-icon {
1621   background: rgba(196, 92, 62, 0.15);
1622   color: var(--coral);
1623 }
1624
1625 .insight h3 {
1626   margin: 0;
1627   font-size: 0.92rem;
1628 }
```

```
1629
1630 .insight p {
1631     margin: 0.2rem 0 0;
1632     color: var(--muted);
1633     font-size: 0.84rem;
1634 }
1635
1636 .account,
1637 .recurring-item {
1638     display: flex;
1639     justify-content: space-between;
1640     gap: 0.75rem;
1641     padding: 0.75rem 0.85rem;
1642     border-radius: 12px;
1643     border: 1px solid var(--line);
1644     background: rgba(225, 29, 46, 0.08);
1645 }
1646
1647 .account strong,
1648 .recurring-item strong {
1649     display: block;
1650 }
1651
1652 .account span,
1653 .recurring-item span {
1654     color: var(--muted);
1655     font-size: 0.78rem;
1656 }
1657
1658 .bal {
1659     font-family: var(--display);
1660     font-weight: 600;
1661 }
1662
1663 .table-wrap {
1664     overflow-x: auto;
1665 }
1666
1667 .tx-table {
1668     width: 100%;
1669     border-collapse: collapse;
1670     font-size: 0.9rem;
1671 }
1672
1673 .tx-table th,
1674 .tx-table td {
1675     text-align: left;
1676     padding: 0.65rem 0.4rem;
```

```
1677 border-bottom: 1px solid var(--line);
1678 vertical-align: top;
1679 }
1680
1681 .tx-table th {
1682   color: var(--muted);
1683   font-size: 0.72rem;
1684   text-transform: uppercase;
1685   letter-spacing: 0.04em;
1686 }
1687
1688 .tx-table .num {
1689   text-align: right;
1690   font-weight: 600;
1691   font-variant-numeric: tabular-nums;
1692 }
1693
1694 .tx-table .expense {
1695   color: var(--coral);
1696 }
1697 .tx-table .income {
1698   color: var(--good);
1699 }
1700 .tx-table .transfer {
1701   color: #c4c4c4;
1702 }
1703
1704 .tx-table.compact td {
1705   padding: 0.55rem 0.35rem;
1706 }
1707
1708 .pill {
1709   display: inline-block;
1710   padding: 0.15rem 0.5rem;
1711   border-radius: 999px;
1712   background: rgba(225, 29, 46, 0.18);
1713   color: #ffc4c8;
1714   font-size: 0.75rem;
1715   font-weight: 600;
1716 }
1717
1718 .icon-btn {
1719   border: 0;
1720   background: transparent;
1721   color: var(--muted);
1722   cursor: pointer;
1723   font: inherit;
1724   font-size: 0.82rem;
```

```
1725     padding: 0.2rem 0.35rem;
1726     border-radius: 8px;
1727 }
1728
1729 .icon-btn:hover {
1730     background: rgba(196, 92, 62, 0.1);
1731     color: var(--coral);
1732 }
1733
1734 .empty-inline {
1735     color: var(--muted);
1736     text-align: center;
1737     padding: 1rem 0.5rem;
1738 }
1739
1740 .panel-head {
1741     margin-bottom: 0.85rem;
1742 }
1743
1744 .panel-head.row {
1745     display: flex;
1746     justify-content: space-between;
1747     gap: 1rem;
1748     align-items: flex-start;
1749 }
1750
1751 .row-actions {
1752     display: flex;
1753     flex-wrap: wrap;
1754     gap: 0.5rem;
1755 }
1756
1757 .subhead {
1758     font-size: 1.05rem;
1759     margin: 1rem 0 0.55rem;
1760 }
1761
1762 dialog {
1763     border: 1px solid rgba(255, 180, 180, 0.18);
1764     border-radius: 18px;
1765     padding: 0;
1766     width: min(420px, calc(100% - 2rem));
1767     background: linear-gradient(180deg, rgba(48, 26, 32, 0.96), rgba(24, 14, 18, 0.96));
1768     color: var(--ink);
1769     box-shadow: 0 24px 70px rgba(12, 4, 8, 0.55), inset 0 1px 0 rgba(255, 255, 255, 0.08);
1770     backdrop-filter: blur(18px);
1771 }
1772
```

```
1773 dialog::backdrop {
1774   background: rgba(12, 6, 10, 0.62);
1775 }
1776
1777 .dialog-form {
1778   display: grid;
1779   gap: 0.75rem;
1780   padding: 1.2rem;
1781 }
1782
1783 .dialog-form h3 {
1784   margin: 0;
1785   font-family: var(--display);
1786 }
1787
1788 .export-grid {
1789   display: grid;
1790   gap: 0.5rem;
1791 }
1792
1793 .dialog-actions {
1794   display: flex;
1795   justify-content: flex-end;
1796   gap: 0.5rem;
1797 }
1798
1799 .check-col {
1800   width: 2.2rem;
1801 }
1802
1803 .check-col input,
1804 .tx-table input[type="checkbox"] {
1805   width: 1.05rem;
1806   height: 1.05rem;
1807   accent-color: var(--teal);
1808   cursor: pointer;
1809 }
1810
1811 .list-search {
1812   margin: 0.65rem 0 0.75rem;
1813 }
1814
1815 .bulk-bar {
1816   display: flex;
1817   flex-wrap: wrap;
1818   gap: 0.65rem;
1819   align-items: center;
1820   justify-content: space-between;
```

```
1821 padding: 0.7rem 0.85rem;
1822 margin: 0.65rem 0;
1823 border-radius: 14px;
1824 border: 1px solid rgba(225, 29, 46, 0.45);
1825 background: rgba(225, 29, 46, 0.12);
1826 font-weight: 700;
1827 }
1828
1829 .bulk-actions {
1830   display: flex;
1831   flex-wrap: wrap;
1832   gap: 0.4rem;
1833 }
1834
1835 .tx-table tbody tr {
1836   cursor: pointer;
1837 }
1838
1839 .tx-table tbody tr.selected {
1840   background: rgba(225, 29, 46, 0.14);
1841 }
1842
1843 .btn.danger {
1844   background: var(--teal);
1845   color: #fff;
1846   border-color: transparent;
1847 }
1848
1849 .btn.tiny.danger:hover {
1850   background: var(--teal-deep);
1851 }
1852
1853 .keep-row {
1854   display: flex;
1855   align-items: center;
1856   gap: 0.5rem;
1857 }
1858
1859 .keep-row input {
1860   width: auto;
1861 }
1862
1863 input[type="date"],
1864 input[type="number"],
1865 input[type="search"],
1866 input[type="tel"],
1867 input[type="password"],
1868 input[type="text"],
```

```
1869 input[type="email"],
1870 textarea,
1871 select {
1872   color-scheme: dark;
1873   accent-color: var(--teal);
1874 }
1875
1876 code {
1877   font-size: 0.85em;
1878   background: rgba(225, 29, 46, 0.08);
1879   padding: 0.1rem 0.35rem;
1880   border-radius: 6px;
1881 }
1882
1883 @media (max-width: 1100px) {
1884   .stats {
1885     grid-template-columns: repeat(2, 1fr);
1886   }
1887 }
1888
1889 @media (max-width: 960px) {
1890   .grid-2-track,
1891   .grid-2 {
1892     grid-template-columns: 1fr;
1893   }
1894
1895   .layout {
1896     width: min(100% - 1.25rem, 1100px);
1897   }
1898
1899   .date-nav {
1900     flex-wrap: wrap;
1901   }
1902
1903   .date-nav input[type="date"] {
1904     width: 100%;
1905     min-width: 0;
1906   }
1907 }
1908
1909 @media (max-width: 760px) {
1910   .brand-hero-logo {
1911     width: 64px;
1912     height: 64px;
1913     border-radius: 16px;
1914   }
1915
1916   .brand-hero .brand-name {
```

```
1917     font-size: 1.55rem;
1918 }
1919
1920 .topbar {
1921     padding: 0.7rem 0.85rem;
1922     padding-top: max(0.7rem, env(safe-area-inset-top));
1923     gap: 0.65rem;
1924 }
1925
1926 .shard,
1927 .orb-c {
1928     display: none;
1929 }
1930
1931 .brand-name {
1932     font-size: 1.05rem;
1933 }
1934
1935 .save-cluster {
1936     margin-left: 0;
1937     width: 100%;
1938     justify-content: space-between;
1939 }
1940
1941 .save-status {
1942     font-size: 0.7rem;
1943     white-space: normal;
1944 }
1945
1946 .main-tabs {
1947     position: fixed;
1948     left: 0;
1949     right: 0;
1950     bottom: 0;
1951     z-index: 40;
1952     display: grid;
1953     grid-template-columns: repeat(3, 1fr);
1954     gap: 0;
1955     width: 100%;
1956     border-radius: 16px 16px 0 0;
1957     padding: 0.35rem 0.35rem max(0.45rem, env(safe-area-inset-bottom));
1958     background: rgba(24, 12, 16, 0.92);
1959     border-top: 1px solid rgba(225, 29, 46, 0.3);
1960     backdrop-filter: blur(16px);
1961     box-shadow: 0 -12px 30px rgba(0, 0, 0, 0.45);
1962 }
1963
1964 .tab {
```

```
1965     flex-direction: column;
1966     justify-content: center;
1967     gap: 0.15rem;
1968     border-radius: 12px;
1969     padding: 0.4rem 0.2rem;
1970     min-height: 56px;
1971     font-size: 0.7rem;
1972 }
1973
1974 .tab-ico {
1975     font-size: 1.05rem;
1976 }
1977
1978 #screen-app {
1979     padding-bottom: calc(5.4rem + env(safe-area-inset-bottom));
1980 }
1981
1982 .layout {
1983     margin: 0.85rem auto 1.25rem;
1984     width: min(100% - 1rem, 1100px);
1985     gap: 0.75rem;
1986 }
1987
1988 .panel {
1989     padding: 1rem;
1990     border-radius: 16px;
1991 }
1992
1993 .intro-panel.rowish,
1994 .panel-head.row,
1995 .hero-strip {
1996     flex-direction: column;
1997     align-items: stretch;
1998 }
1999
2000 .intro-panel .btn,
2001 .hero-strip .btn {
2002     width: 100%;
2003     text-align: center;
2004 }
2005
2006 .profile-identity {
2007     grid-template-columns: 1fr;
2008     justify-items: center;
2009     text-align: center;
2010 }
2011 .cat-row,
2012 .merchant-row,
```

```
2013 .budget-grid label,
2014 .stats {
2015   grid-template-columns: 1fr;
2016 }
2017
2018 .amount-field input {
2019   font-size: 1.45rem;
2020 }
2021
2022 .wizard {
2023   width: min(100%, 640px);
2024   padding-bottom: 5.5rem;
2025 }
2026
2027 .wizard-foot {
2028   position: sticky;
2029   bottom: 0.5rem;
2030   background: rgba(5, 5, 5, 0.92);
2031   padding: 0.65rem;
2032   border-radius: 16px;
2033   border: 1px solid var(--line);
2034 }
2035
2036 .wizard-foot .btn {
2037   flex: 1;
2038 }
2039
2040 .controls {
2041   gap: 0.75rem;
2042 }
2043
2044 .control-group,
2045 .control-group.grow {
2046   min-width: 100%;
2047   width: 100%;
2048 }
2049
2050 .segmented {
2051   width: 100%;
2052 }
2053
2054 .seg {
2055   flex: 1;
2056   text-align: center;
2057   min-height: 42px;
2058 }
2059
2060 .account-row {
```

```
2061     grid-template-columns: 1fr;
2062 }
2063
2064 .row-actions,
2065 .bulk-actions,
2066 .bulk-bar {
2067     flex-direction: column;
2068     align-items: stretch;
2069 }
2070
2071 .bulk-actions .btn,
2072 .row-actions .btn,
2073 .file-btn {
2074     width: 100%;
2075     text-align: center;
2076 }
2077
2078 .icon-btn {
2079     min-height: 40px;
2080     min-width: 44px;
2081     padding: 0.45rem 0.6rem;
2082     font-size: 0.88rem;
2083 }
2084
2085 .tx-table thead {
2086     display: none;
2087 }
2088
2089 .tx-table,
2090 .tx-table tbody,
2091 .tx-table tr,
2092 .tx-table td {
2093     display: block;
2094     width: 100%;
2095 }
2096
2097 .tx-table tr {
2098     border: 1px solid var(--line);
2099     border-radius: 14px;
2100     padding: 0.7rem 0.75rem 0.55rem;
2101     margin-bottom: 0.65rem;
2102     background: rgba(255, 255, 255, 0.03);
2103 }
2104
2105 .tx-table td {
2106     border: 0;
2107     padding: 0.28rem 0;
2108     display: flex;
```

```
2109     justify-content: space-between;
2110     gap: 0.75rem;
2111     text-align: left;
2112 }
2113
2114 .tx-table .num {
2115     text-align: right;
2116     font-size: 1.05rem;
2117 }
2118
2119 .tx-table .check-col {
2120     width: auto;
2121     justify-content: flex-start;
2122 }
2123
2124 .chart {
2125     overflow-x: auto;
2126     min-height: 160px;
2127 }
2128
2129 .bar-col {
2130     min-width: 28px;
2131 }
2132
2133 dialog {
2134     width: calc(100% - 1rem);
2135     max-height: calc(100dvh - 1.5rem);
2136     margin: auto 0.5rem;
2137     border-radius: 16px;
2138 }
2139
2140 .dialog-form {
2141     max-height: calc(100dvh - 2rem);
2142     overflow: auto;
2143 }
2144
2145 .dialog-actions {
2146     position: sticky;
2147     bottom: 0;
2148     background: var(--panel-solid);
2149     padding-top: 0.5rem;
2150 }
2151
2152 .welcome-card {
2153     padding: 1.35rem 1.1rem;
2154 }
2155
2156 .demo-banner {
```

```
2157     flex-direction: column;
2158     align-items: stretch;
2159   }
2160 }
2161
2162 @media (min-width: 761px) {
2163   .tab-ico {
2164     display: none;
2165   }
2166 }
2167
2168 dialog#bulk-edit-dialog,
2169 dialog#csv-dialog {
2170   width: min(520px, calc(100% - 2rem));
2171 }
2172
2173 .btn.ghost.danger {
2174   background: transparent;
2175   color: var(--coral);
2176 }
2177
2178 .pager {
2179   display: flex;
2180   align-items: center;
2181   justify-content: center;
2182   gap: 0.75rem;
2183   margin-top: 0.85rem;
2184   font-size: 0.85rem;
2185 }
2186
2187 .pin-box {
2188   margin: 1rem 0 1.25rem;
2189   padding: 1rem 1.1rem;
2190 }
2191
2192 .masked-mobile {
2193   margin: 0.35rem 0 0.85rem;
2194   font-size: 0.95rem;
2195   color: var(--ink-soft);
2196 }
2197
2198 .masked-mobile strong {
2199   color: var(--ink);
2200   letter-spacing: 0.04em;
2201 }
2202
2203 /* Login */
2204 #screen-login {
```

```
2205   min-height: 100vh;
2206   display: grid;
2207   place-items: center;
2208   padding: 1.5rem;
2209 }
2210
2211 .login-stage {
2212   width: min(440px, 100%);
2213 }
2214
2215 .login-card h1 {
2216   margin: 0.85rem 0 0.4rem;
2217   font-family: var(--display);
2218   letter-spacing: -0.03em;
2219 }
2220
2221 .login-form {
2222   display: grid;
2223   gap: 0.85rem;
2224   margin: 1rem 0 0.35rem;
2225 }
2226
2227 .login-extra {
2228   margin-top: 0.85rem;
2229 }
2230
2231 .login-reset {
2232   margin-top: 1.15rem;
2233   padding-top: 1rem;
2234   border-top: 1px solid rgba(255, 214, 210, 0.12);
2235   display: grid;
2236   gap: 0.7rem;
2237 }
2238
2239 #login-reset-panel {
2240   display: grid;
2241   gap: 0.7rem;
2242 }
2243
2244 .form-status.login-status,
2245 .form-status.is-error {
2246   color: #ff8b8b;
2247 }
2248
2249 .toast {
2250   position: fixed;
2251   left: 50%;
2252   bottom: calc(1.2rem + env(safe-area-inset-bottom));
```

```
2253   transform: translateX(-50%);
2254   z-index: 80;
2255   display: flex;
2256   align-items: center;
2257   gap: 0.75rem;
2258   padding: 0.7rem 0.9rem 0.7rem 1.05rem;
2259   border-radius: 999px;
2260   background: #2a161c;
2261   border: 1px solid rgba(225, 29, 46, 0.45);
2262   box-shadow: 0 12px 32px rgba(0, 0, 0, 0.45);
2263   color: #fff;
2264   font-size: 0.9rem;
2265 }
2266
2267 @media (max-width: 760px) {
2268   .choice-grid {
2269     grid-template-columns: 1fr;
2270   }
2271
2272   .toast {
2273     bottom: calc(5.6rem + env(safe-area-inset-bottom));
2274     width: calc(100% - 1.5rem);
2275     max-width: 420px;
2276     border-radius: 16px;
2277     justify-content: space-between;
2278   }
2279 }
2280
2281 @media (max-width: 900px) {
2282   .welcome-stage {
2283     grid-template-columns: 1fr;
2284   }
2285
2286   .hero-orbit {
2287     min-height: 260px;
2288     order: -1;
2289   }
2290 }
2291
2292 @media (prefers-reduced-motion: reduce) {
2293   *,
2294   *::before,
2295   *::after {
2296     animation: none !important;
2297     transition: none !important;
2298   }
2299
2300   .bg-mesh,
```

```
2301 .orb,
2302 .shard,
2303 .float-card,
2304 .coin,
2305 .ring-wrap,
2306 .bar-3d,
2307 .welcome-card,
2308 .panel,
2309 .stat {
2310     transform: none !important;
2311 }
2312
2313 .bg-grain {
2314     animation: none !important;
2315 }
2316 }
2317
2318 @media print {
2319     .topbar,
2320     .demo-banner,
2321     .controls,
2322     .main-tabs,
2323     .dialog-actions,
2324     .icon-btn,
2325     .check-col,
2326     .bg-wallpaper,
2327     .bg-mesh,
2328     .bg-scene,
2329     .bg-texture,
2330     .bg-grain,
2331     .bg-spot,
2332     .bg-vignette,
2333     .hero-orbit,
2334     .toast {
2335         display: none !important;
2336     }
2337
2338     body,
2339     .screen,
2340     .panel,
2341     .glass-3d,
2342     .card,
2343     .stat,
2344     .intro-panel,
2345     .results-grid,
2346     .table-wrap {
2347         background: #fff !important;
2348         color: #111 !important;
```

```
2349     box-shadow: none !important;
2350     border-color: #ccc !important;
2351 }
2352
2353 body {
2354     background: #fff !important;
2355 }
2356
2357 .muted,
2358 .small {
2359     color: #444 !important;
2360 }
2361
2362 .num.expense {
2363     color: #9b1c1c !important;
2364 }
2365
2366 .num.income {
2367     color: #0a6b3a !important;
2368 }
2369
2370 #view-track,
2371 #view-setup-edit,
2372 #view-demo,
2373 #screen-welcome,
2374 #screen-login,
2375 #screen-setup {
2376     display: none !important;
2377 }
2378
2379 #screen-app,
2380 #view-results {
2381     display: block !important;
2382 }
2383
2384 #view-results[hidden] {
2385     display: none !important;
2386 }
2387
2388 .tx-table th,
2389 .tx-table td {
2390     border: 1px solid #ccc !important;
2391     color: #111 !important;
2392 }
2393 }
2394
2395 .home-glance {
2396     display: grid;
```

```
2397   grid-template-columns: repeat(3, 1fr);
2398   gap: 0.65rem;
2399   margin-bottom: 0.85rem;
2400 }
2401
2402 .glance-card {
2403   padding: 0.85rem 0.9rem;
2404   border-radius: 14px;
2405   border: 1px solid var(--line);
2406   background: rgba(36, 22, 28, 0.88);
2407   display: grid;
2408   gap: 0.25rem;
2409 }
2410
2411 .glance-label {
2412   font-size: 0.72rem;
2413   font-weight: 600;
2414   letter-spacing: 0.04em;
2415   text-transform: uppercase;
2416   color: var(--muted);
2417 }
2418
2419 .glance-card strong {
2420   font-family: var(--display);
2421   font-size: clamp(1.05rem, 2.8vw, 1.35rem);
2422   letter-spacing: -0.03em;
2423 }
2424
2425 .glance-card strong.good {
2426   color: var(--good);
2427 }
2428
2429 .glance-card strong.bad {
2430   color: #ff8b8b;
2431 }
2432
2433 @media (max-width: 720px) {
2434   .home-glance {
2435     grid-template-columns: 1fr;
2436   }
2437
2438   .home-glance {
2439     grid-template-columns: repeat(3, 1fr);
2440     gap: 0.45rem;
2441   }
2442
2443   .glance-card {
2444     padding: 0.65rem 0.55rem;
```

```
2445     }
2446
2447     .glance-card strong {
2448         font-size: 0.95rem;
2449     }
2450
2451     .glance-label {
2452         font-size: 0.62rem;
2453     }
2454 }
2455
2456 /* ??? UX polish: calmer, clearer hierarchy ??? */
2457
2458 body {
2459     background: #161016;
2460 }
2461
2462 .bg-wallpaper {
2463     opacity: 0.92;
2464 }
2465
2466 .bg-mesh {
2467     opacity: 0.28;
2468     animation: none;
2469     filter: blur(24px);
2470 }
2471
2472 .bg-vignette {
2473     background: radial-gradient(ellipse at center, transparent 55%, rgba(10, 4, 8, 0.45) 100%);
2474 }
2475
2476 .welcome-simple {
2477     display: grid;
2478     place-items: center;
2479     min-height: 100dvh;
2480     padding: 1.5rem;
2481 }
2482
2483 .welcome-simple .welcome-card {
2484     width: min(420px, 100%);
2485     transform: none;
2486     box-shadow: 0 18px 48px rgba(0, 0, 0, 0.35);
2487 }
2488
2489 .btn.linkish {
2490     background: transparent;
2491     border: 0;
2492     color: var(--ink-soft);
```

```
2493   text-decoration: underline;
2494   text-underline-offset: 3px;
2495   font-weight: 500;
2496   padding: 0.35rem;
2497 }
2498
2499 .btn.linkish:hover {
2500   color: #fff;
2501 }
2502
2503 .intro-panel.slim {
2504   display: block;
2505   padding: 1rem 1.15rem;
2506 }
2507
2508 .intro-panel.slim h1 {
2509   margin: 0.15rem 0 0.35rem;
2510   font-size: clamp(1.35rem, 3vw, 1.75rem);
2511 }
2512
2513 .add-sheet h2 {
2514   margin-top: 0;
2515 }
2516
2517 .type-chips {
2518   display: flex;
2519   flex-wrap: wrap;
2520   gap: 0.5rem;
2521   border: 0;
2522   padding: 0;
2523   margin: 0 0 0.75rem;
2524 }
2525
2526 .chip-choice {
2527   cursor: pointer;
2528 }
2529
2530 .chip-choice input {
2531   position: absolute;
2532   opacity: 0;
2533   pointer-events: none;
2534 }
2535
2536 .chip-choice span {
2537   display: inline-flex;
2538   align-items: center;
2539   padding: 0.55rem 0.95rem;
2540   border-radius: 999px;
```

```
2541 border: 1px solid var(--line);
2542 background: rgba(255, 255, 255, 0.04);
2543 font-weight: 600;
2544 font-size: 0.92rem;
2545 }
2546
2547 .chip-choice input:checked + span {
2548   border-color: var(--teal);
2549   background: rgba(225, 29, 46, 0.22);
2550   color: #fff;
2551 }
2552
2553 .sr-only {
2554   position: absolute;
2555   width: 1px;
2556   height: 1px;
2557   padding: 0;
2558   margin: -1px;
2559   overflow: hidden;
2560   clip: rect(0, 0, 0, 0);
2561   border: 0;
2562 }
2563
2564 .more-fields {
2565   margin: 0.65rem 0 0.85rem;
2566   border-radius: 12px;
2567   border: 1px solid var(--line);
2568   padding: 0.35rem 0.75rem 0.55rem;
2569   background: rgba(0, 0, 0, 0.15);
2570 }
2571
2572 .more-fields summary,
2573 .more-insights summary,
2574 .profile-section summary {
2575   cursor: pointer;
2576   font-weight: 600;
2577   color: var(--ink-soft);
2578   padding: 0.45rem 0;
2579   list-style: none;
2580 }
2581
2582 .more-fields summary::-webkit-details-marker,
2583 .more-insights summary::-webkit-details-marker,
2584 .profile-section summary::-webkit-details-marker {
2585   display: none;
2586 }
2587
2588 .more-fields summary::before,
```

```
2589 .more-insights summary::before,
2590 .profile-section summary::before {
2591   content: "? ";
2592   color: var(--muted);
2593 }
2594
2595 .more-fields[open] summary::before,
2596 .more-insights[open] summary::before,
2597 .profile-section[open] summary::before {
2598   content: "? ";
2599 }
2600
2601 .more-fields label {
2602   margin-top: 0.55rem;
2603 }
2604
2605 .import-fold {
2606   margin-top: 1rem;
2607 }
2608
2609 .compact-controls {
2610   display: flex;
2611   flex-wrap: wrap;
2612   gap: 0.65rem;
2613   align-items: center;
2614 }
2615
2616 .compact-controls .field-label {
2617   display: none;
2618 }
2619
2620 .more-insights {
2621   margin-top: 0.85rem;
2622 }
2623
2624 .more-insights .nested-grid {
2625   margin-top: 0.75rem;
2626 }
2627
2628 .profile-section {
2629   margin: 0.85rem 0;
2630   padding: 0.35rem 0.85rem 0.75rem;
2631   border: 1px solid var(--line);
2632   border-radius: 14px;
2633   background: rgba(255, 255, 255, 0.03);
2634 }
2635
2636 .profile-section .pin-box {
```

```
2637     border: 0;
2638     background: transparent;
2639     padding: 0;
2640     box-shadow: none;
2641 }
2642
2643 .profile-page h1 {
2644     margin-bottom: 0.25rem;
2645 }
2646
2647 .file-btn.block {
2648     display: inline-flex;
2649     width: 100%;
2650     justify-content: center;
2651 }
2652
2653 .save-cluster .btn.ghost.tiny {
2654     opacity: 0.85;
2655 }
2656
2657 .hud-card {
2658     box-shadow: none;
2659 }
2660
2661 .panel {
2662     backdrop-filter: blur(8px);
2663 }
2664
2665 @media (max-width: 720px) {
2666     .welcome-actions .btn.large {
2667         width: 100%;
2668     }
2669
2670     .type-chips {
2671         justify-content: stretch;
2672     }
2673
2674     .chip-choice {
2675         flex: 1;
2676     }
2677
2678     .chip-choice span {
2679         width: 100%;
2680         justify-content: center;
2681     }
2682 }
```

visual.js

```
1  (() => {
2    /* Calm motion: spotlight disabled so the UI stays readable */
3    const root = document.documentElement;
4    root.style.setProperty("--mx", "50vw");
5    root.style.setProperty("--my", "18vh");
6    root.style.setProperty("--parx", "0px");
7    root.style.setProperty("--pary", "0px");
8    root.style.setProperty("--tilt-x", "0deg");
9    root.style.setProperty("--tilt-y", "0deg");
10  })();
```

sw.js

```
1  const CACHE = "niva-v4";
2  const ASSETS = [
3    "./",
4    "./index.html",
5    "./styles.css",
6    "./core.js",
7    "./app.js",
8    "./visual.js",
9    "./icon.svg",
10   "./manifest.json",
11   "./sample-bank-export.csv",
12 ];
13
14 self.addEventListener("install", (event) => {
15   event.waitUntil(
16     caches.open(CACHE).then(async (cache) => {
17       await Promise.allSettled(ASSETS.map((url) => cache.add(url)));
18       await self.skipWaiting();
19     })
20   );
21 });
22
23 self.addEventListener("activate", (event) => {
24   event.waitUntil(
25     caches.keys().then((keys) => Promise.all(keys.filter((k) => k !== CACHE).map((k) => caches.delete(k)))).then(() => self.clients.claim())
26   );
27 });
28
29 self.addEventListener("fetch", (event) => {
30   if (event.request.method !== "GET") return;
31   event.respondWith(
32     caches.match(event.request).then((cached) => {
33       if (cached) return cached;
34       return fetch(event.request)
35         .then((res) => {
36           const copy = res.clone();
37           caches.open(CACHE).then((cache) => cache.put(event.request, copy)).catch(() => {});
38           return res;
39         })
40         .catch(() => caches.match("./index.html"));
41     })
42   );
43 });
44
```

manifest.json

```
1 {
2   "name": "Niva",
3   "short_name": "Niva",
4   "description": "Niva ? Know where it goes. Private spending tracker that stays on this device.",
5   "start_url": "./index.html",
6   "scope": "./",
7   "display": "standalone",
8   "background_color": "#1a1016",
9   "theme_color": "#1a1016",
10  "icons": [
11    {
12      "src": "icon.svg",
13      "sizes": "any",
14      "type": "image/svg+xml",
15      "purpose": "any"
16    }
17  ]
18 }
```